

UNIT – 1

ROBOT LOCOMOTION

Introduction to AI and Robotics – Robot Locomotion – Legged Mobile Robots – Wheeled Mobile Robots – Aerial Mobile Robots.

Introduction to AI and Robotics

AI:

- Artificial intelligence (AI) is the ability of machines to replicate or enhance human intellect, such as reasoning and learning from experience.
- Artificial intelligence has been used in computer programs for years, but it is now applied to many other products and services.

Application of AI:

- Astronomy
- Healthcare
- Gaming
- Social Media
- Robotics

Robotics:

- Robotics is a branch of engineering and computer science that involves the conception, design, manufacture and operation of robots.
- The objective of the robotics field is to create intelligent machines that can assist humans in a variety of ways.

Applications of Robotics:

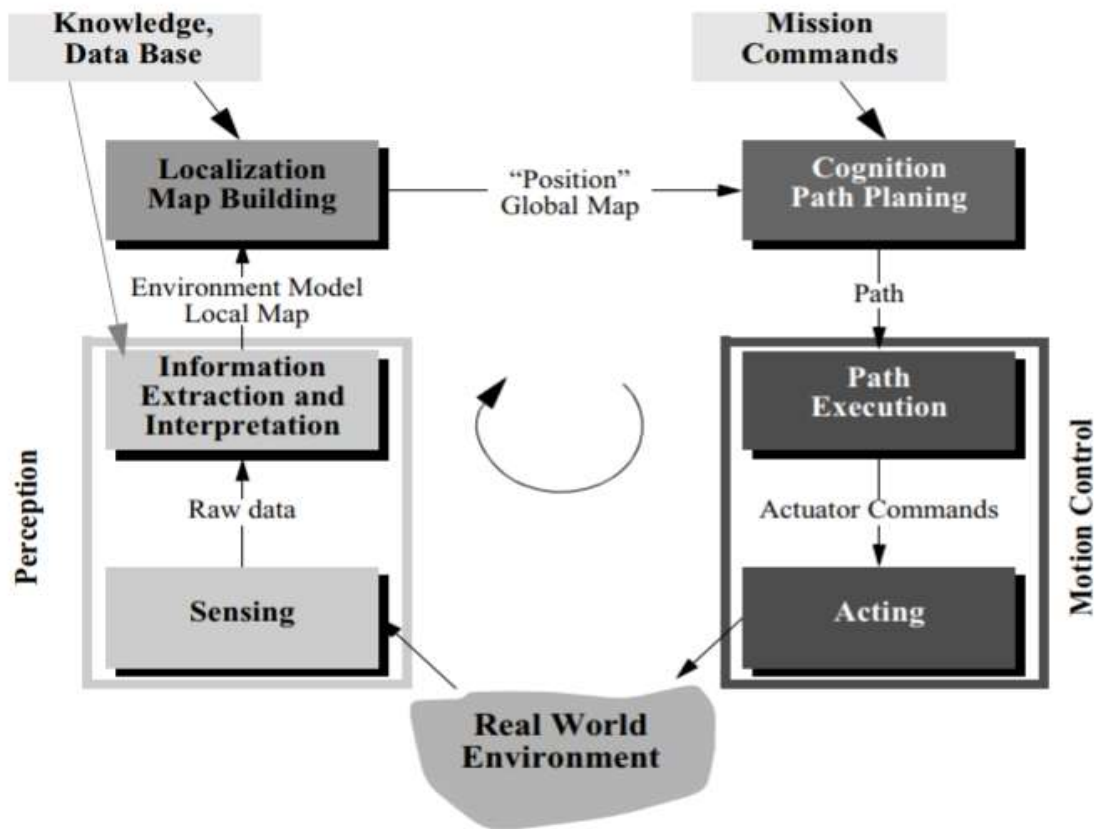
- Underwater Exploration
- Food Preparation
- Manufacturing
- Military
- Customer Service

Advantages of AI & Robotics

- Artificial intelligence can work 24*7 with no pay.
- This is the greatest benefit of artificial intelligence over humans.
- These machines can work without break & rest.
- Artificial intelligence is used in decision making & it can think faster.

Disadvantages of AI & Robotics

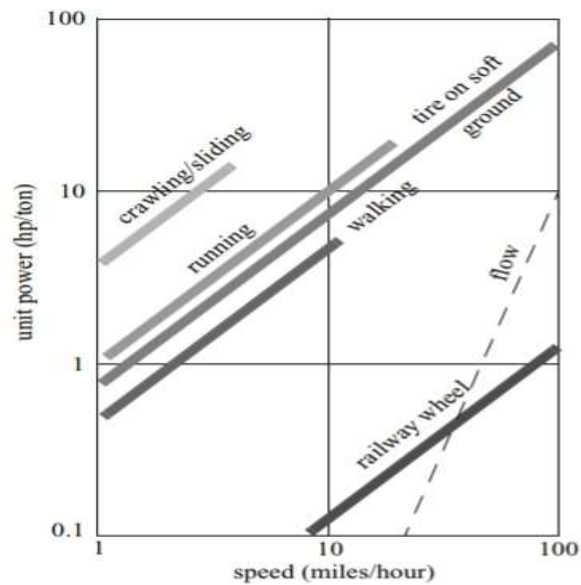
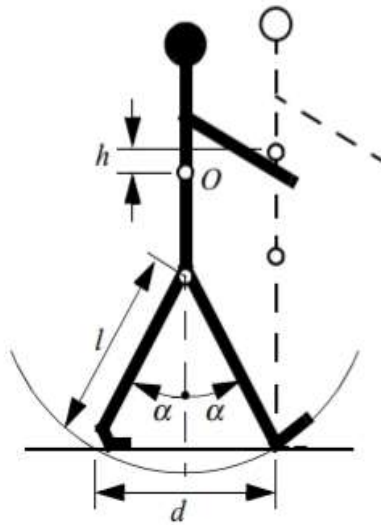
- Artificial Intelligence can be a potential threat to the very existence of human beings.
- Robots cost much money in maintenance & repair.
- Robots can store large amounts of data but the storage, access, and retrieval are not as effective as the human brain.
- Robots are not able to act any different from what they are programmed to do.



Robot Locomotion

- Locomotion is the method of moving from one place to another. The mechanism that makes a robot capable of moving in its environment is called as robot locomotion.
- That is to say, robot locomotion is the technique of aligning multiple tools and solutions for the movement of robots. It also is the mobilization of robots within a specific environment.
- As a result, the concept of robot locomotion enables robots to
 - Walk
 - Run
 - Jump
 - Fly
 - Roll
 - Crawl
 - Swim
 - Glide
- Moreover, it uses concepts of geometry to engage and interact with various contact points. Therefore, robot locomotion develops the ability of a static robot to dynamically reflect on its environment or terrain.

Type of motion	Resistance to motion	Basic kinematics of motion
Flow in a Channel 	Hydrodynamic forces	Eddies 
Crawl 	Friction forces	Longitudinal vibration 
Sliding 	Friction forces	Transverse vibration 
Running 	Loss of kinetic energy	Oscillatory movement of a multi-link pendulum 
Jumping 	Loss of kinetic energy	Oscillatory movement of a multi-link pendulum 
Walking 	Gravitational forces	Rolling of a polygon (see figure 2.2) 



Key Issues for Locomotion:

- Conventional locomotion systems are often unstable within complex and diverse environments. Moreover, their motions are scanty and insufficient which may lead to rigidity during various activities. Further, in the case of kinematics and dynamics, robot locomotion needs to overcome the following issues:
 - Stability:**
 - Quantity, Position, and Geometry of contact points.
 - Gravitational Centre Point.
 - Stability during static and dynamic status.
 - The propensity of the surface, environment, and terrain.
 - Characteristics of Contact:**
 - The point of contact is the path's size and shape to reach it.
 - The angular position of the Contact.

- c. The amount of friction, abrasions, and obstacles to reach the point of contact.
- iii. **Type of Environment:**
 - a. The structure, formation, and composition of the environment.
 - b. The medium and surface tension of the environment.

Types of robot locomotion

- Wheeled locomotion
- Legged locomotion
- Aerial locomotion

Legged Mobile Robots

- Legged robots are mobile robots that use mechanical limbs for movement, they are similar to wheeled robots, but their locomotion methods are more complicated compared to their wheeled counterparts, they perform much better than wheeled robots on uneven terrain, Legged robots are indispensable for most applications, such as
 - One-legged robots
 - Two-legged robots
 - Four-legged robots
 - Six-legged robots
- Legged locomotion consumes more power while demonstrating Jump, Hop, Walk, Trot and climb up or down.
- It requires more motors to accomplish a movement. It is suited for rough as well as smooth terrain where irregular or too smooth surface makes it consume more operational power. It is a little difficult to implement because of stability issues.
- The total number of possible gaits (a periodic sequence of release and lift events for each of the total legs) a robot can travel depending upon the number of robot legs.
- If a robot has K legs, then the number of possible events is,

$$N=(2K-1)!$$
- In the case of a two-legged robot (K=2), therefore the number of possible events is

$$N=(2K-1)!$$

$$N=(2(2)-1)!$$

$$N=(4-1)!$$

$$N=3!$$

$$N=3*2*1$$

$$N=6$$
- Hence, there are six possible different events:-
 - Lifting the right leg
 - Lifting the left leg
 - Releasing the right leg
 - Releasing the left leg
 - Releasing both the legs together
 - Lifting both the legs together

Leg configurations and stability

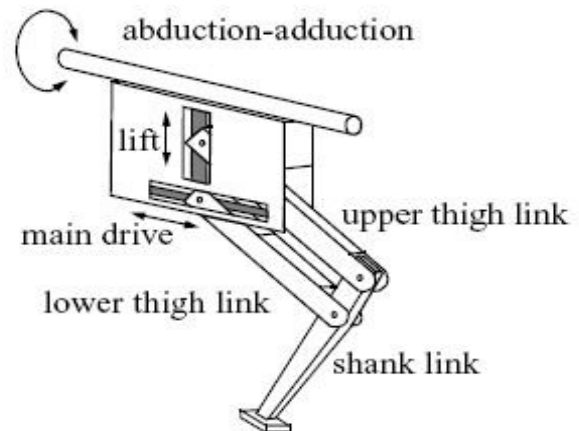
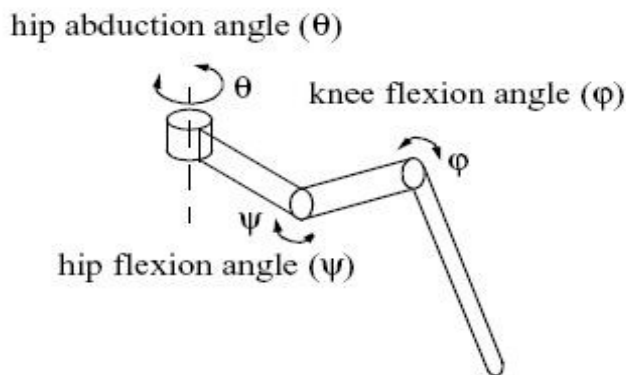
- Because legged robots are biologically inspired, it is instructive to examine biologically successful legged systems.

- Large animals, such as mammals and reptiles, have four legs, whereas insects have six or more legs. In some mammals, the ability to walk on only two legs has been perfected. Especially in the case of humans, balance has progressed to the point that we can even jump with one leg.
- This exceptional maneuverability comes at a price: much more complex active control to maintain balance. In contrast, a creature with three legs can exhibit a static, stable pose provided that it can ensure that its center of gravity is within the tripod of ground contact.
- Static stability, demonstrated by a three-legged stool, means that balance is maintained with no need for motion. A small deviation from stability (e.g., gently pushing the stool) is passively corrected toward the stable pose when the upsetting force stops.
- But a robot must be able to lift its legs in order to walk. In order to achieve static walking, a robot must have at least six legs. In such a configuration, it is possible to design a gait in which a statically stable tripod of legs is in contact with the ground at all times. Insects and spiders are immediately able to walk when born.
- For them, the problem of balance during walking is relatively simple. Mammals, with four legs, cannot achieve static walking, but are able to stand easily on four legs. Fauns, for example, spend several minutes attempting to stand before they are able to do so, then spend several more minutes learning to walk without falling. Humans, with two legs, cannot even stand in one place with static stability. Infants require months to stand and walk, and even longer to learn to jump, run, and stand on one leg.

Examples of Legged Mobile Robots:

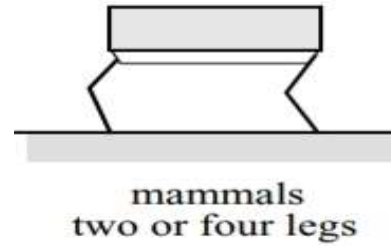
One-legged robots (pogo stick robots)

- One-legged robots (pogo stick robots) can use a hopping motion for navigation.
- They are stable, they can balance themselves.
- They are very efficient, but they come with complex design & control.
- The typical one-legged robot can be constructed by controlling robot posture.



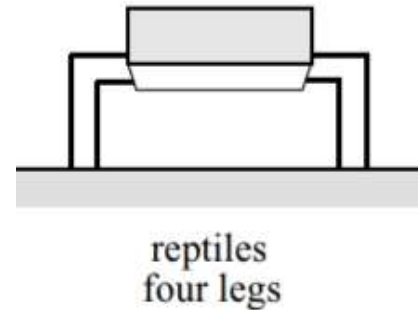
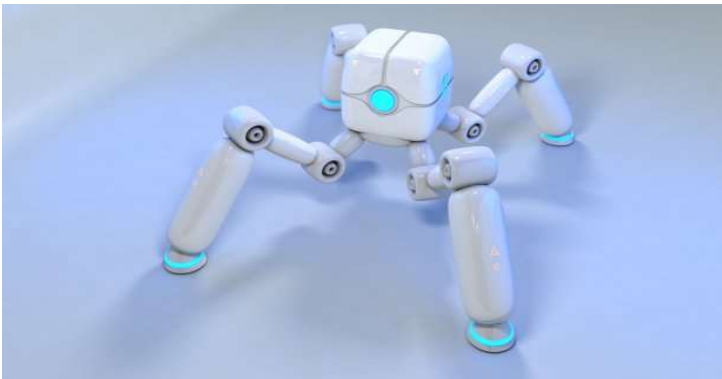
Two-legged robots (Bipedal robots)

- Two-legged robots (Bipedal robots) can exhibit bipedal motion.
- Stability is maintained by calculating each step and moving the weight in a well-defined way.
- They face two primary problems: stability control which refers to the robot's balance, and motion control refers to the robot's ability to move.



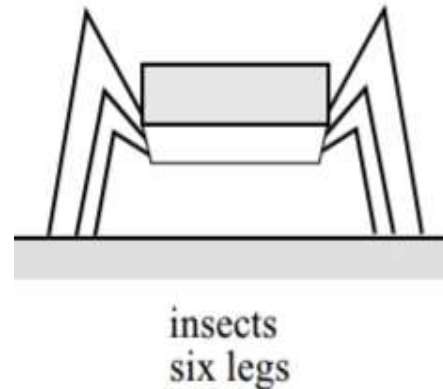
Four-legged robots (Quadrupedal robots)

- Four-legged robots are also called Quadruped robots “Big Dog”.
- They are developed to traverse difficult terrain.
- They can exhibit quadrupedal motion.
- They can benefit from increased stability over two-legged robots.



Six-legged robots (hexapods)

- Six-legged robots (hexapods) offer greater stability than bipedal or quadrupedal robots.
- They have 6 legs, their final designs mimic the mechanics of insects.
- These include the slowest wave gait, where pairs of legs move in a “wave” from the back to the front.



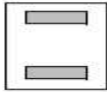
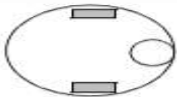
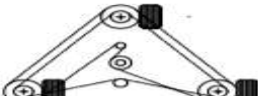
Wheeled Mobile Robots

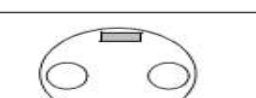
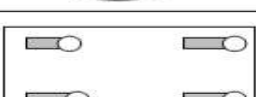
- Wheeled robots can change their positions with the help of their wheels, Wheeled robot motion can be achieved easily & its cost is pretty low, control of wheeled movement is easier, So, wheeled robots are one of the most frequently seen robots,
 - Single wheeled robots
 - Two-wheeled robots

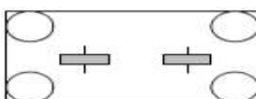
- Three-wheel robots
- Four-wheeled robots
- It requires fewer motors for accomplishing a movement. It is a little easy to implement as there are lesser stability issues in the case of more wheels. It is more power efficient as compared to legged locomotion.

Wheel Configuration:

Wheel configurations for rolling vehicles

# of wheels	Arrangement	Description	Typical examples
2		One steering wheel in the front, one traction wheel in the rear	Bicycle, motorcycle
		Two-wheel differential drive with the center of mass (COM) below the axle	Cye personal robot
3		Two-wheel centered differential drive with a third point of contact	Nomad Scout, smartRob EPFL
		Two independently driven wheels in the rear/front, 1 unpowered omnidirectional wheel in the front/rear	Many indoor robots, including the EPFL robots Pygmalion and Alice
		Two connected traction wheels (differential) in rear, 1 steered free wheel in front	Piaggio minitrucks
		Two free wheels in rear, 1 steered traction wheel in front	Neptune (Carnegie Mellon University), Hero-1
		Three motorized Swedish or spherical wheels arranged in a triangle; omnidirectional movement is possible	Stanford wheel Tribolo EPFL, Palm Pilot Robot Kit (CMU)
		Three synchronously motorized and steered wheels; the orientation is not controllable	"Synchro drive" Denning MRV-2, Georgia Institute of Technology, I-Robot B24, Nomad 200

# of wheels	Arrangement	Description	Typical examples
4		Two motorized wheels in the rear, 2 steered wheels in the front; steering has to be different for the 2 wheels to avoid slipping/skidding.	Car with rear-wheel drive
		Two motorized and steered wheels in the front, 2 free wheels in the rear; steering has to be different for the 2 wheels to avoid slipping/skidding.	Car with front-wheel drive
		Four steered and motorized wheels	Four-wheel drive, four-wheel steering Hyperion (CMU)
		Two traction wheels (differential) in rear/front, 2 omnidirectional wheels in the front/rear	Charlie (DMT-EPFL)
		Four omnidirectional wheels	Carnegie Mellon Uranus
		Two-wheel differential drive with 2 additional points of contact	EPFL Khepera, Hyperbot Chip
		Four motorized and steered castor wheels	Nomad XR4000

# of wheels	Arrangement	Description	Typical examples
6		Two motorized and steered wheels aligned in center, 1 omnidirectional wheel at each corner	First
		Two traction wheels (differential) in center, 1 omnidirectional wheel at each corner	Terregator (Carnegie Mellon University)

One Wheel robot

- One-wheeled robots are difficult to keep balanced due to the single point of contact with the ground.
- They have only one wheel.
- It is easier to use a spherical wheel rather than a typical disc wheel, as the **robot** can move in any direction along the sphere.
- Gyroscopes & counter-torque mechanisms can keep the **robot** upright.

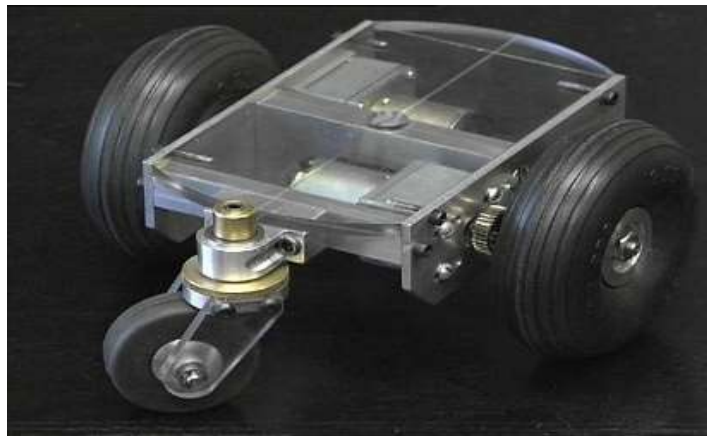
Two-wheeled robots

- Two-wheeled robots are simple, they have their wheels parallel to each other, and they are called dicycles, or one wheel in front of the other,
- For the robot that has the left and right wheels, it comes with at least two **sensors**, the tilt sensor is used to determine tilt angle & wheel encoders keep track of the position of the platform of the robot.



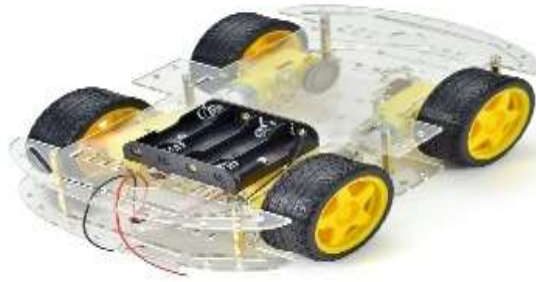
Three Wheel Robots

- Three-wheel robots are of two types based on the nature of steered wheels.
- The first type comes with wheels that are separately powered & the third is free rotating purely for balance
- The second type comes with two wheels that are powered by one source & a different source for the third wheel.
- The direction of the robot can be altered by changing the relative rate at which the two powered wheels are rotating, it will continue to go straight when two wheels have the same rate of rotation and the same direction.



4-wheel drive robots

- 4-wheel drive robot comes with 2 pairs of powered wheels.
- Each pair can turn in the same direction.
- If the pairs don't run at the same speed, the robot will slow & won't be able to drive straight
- A good design will have to incorporate some form of car-like steering.
- Differential Steering method allows the **robot** to turn in the same way the car does, when your **robot** is powered by a combustion engine, It only needs one motor.



Advantages of Wheeled robots

- ✓ Wheeled robots are robots that can move on the ground with the use of their motorized wheels to propel themselves.
- ✓ Wheeled robots are most popular among the consumer market.
- ✓ They offer low cost & simplicity.
- ✓ They come with any number of wheels, but three wheels are sufficient for static and dynamic balance.

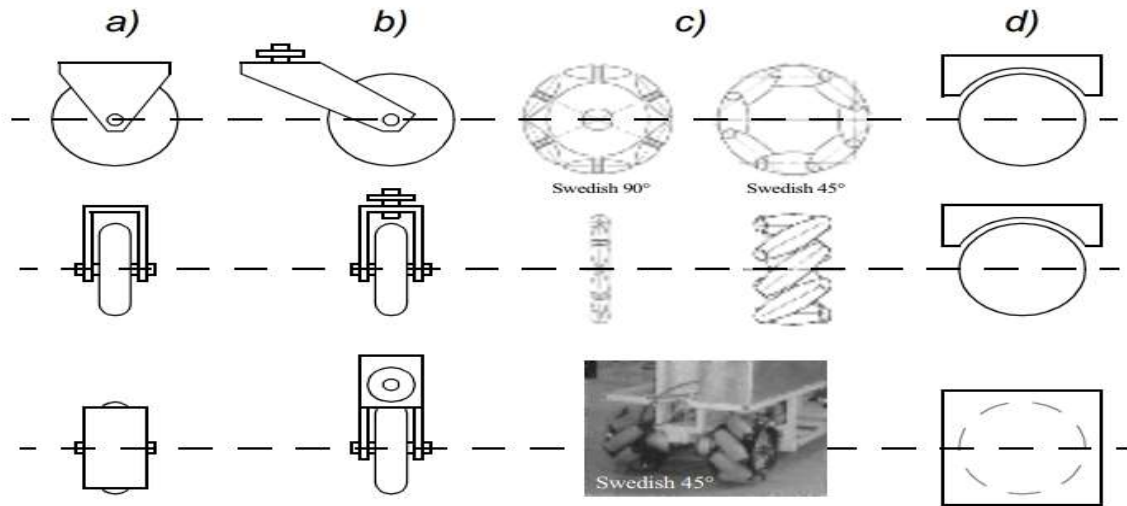
Disadvantages of wheeled robots

- ✓ Wheeled robots cannot navigate well over obstacles.
- ✓ Depending on the terrain, such as rocky terrain, sharp declines or areas with low friction.
- ✓ There are some situations where the wheels are not the best choice, The **robot** needs to pass small or large obstacles.

Examples of Wheel types:

- ✓ **Castor wheel** - It rotates around the offset steering joint and wheel axle.
- ✓ **Standard wheel** - It rotates around the contact and the wheel axle.
- ✓ **Ball or spherical wheel** - This wheel is technically difficult to implement due to architectural complexity. It is an Omni directional wheel with only one-directional movement is allowed.
- ✓ **Swedish 45 and Swedish 90 wheels** - It is an Omni-wheel, which rotates around the contact point, around the wheel axle, and the rollers.

	unpowered omnidirectional wheel (spherical, castor, Swedish);
	motorized Swedish wheel (Stanford wheel);
	unpowered standard wheel;
	motorized standard wheel;
	motorized and steered castor wheel;
	steered standard wheel;
	connected wheels.



Aerial Mobile Robots

- Aerial Robots also called as unmanned aerial vehicle (UAV). The unmanned aerial vehicle (UAV) is known as the drone. The unmanned aircraft system (UAS) is the aircraft without the human pilot aboard. The flight of UAVs can operate either under the remote control by the human operator or fully or intermittently autonomously by onboard computers.
- The unmanned aerial vehicles are preferred for the missions which are too dangerous or dull for the humans. They provide the attack capability for high-risk missions and they originated mostly in the military applications.
- The unmanned aerial vehicles can be used in other applications such as policing and the surveillance, the data collection, the aerial photography, the agriculture and the drone racing, They can be used in the commercial, the scientific and the recreational.
- The drones have many different names, They are referred to as the killer drones and the attack drones, They are called the spy drones or the surveillance drones, They can be both remote controlled by the pilot or controlled by the computer, and they are used to carry out certain actions without direct human piloting.



Application of Aerial Robots

- Remote-sensing
- Disaster response
- Image acquisition
- Surveillance
- Transportation
- Delivery of Goods

Importance of Aerial Robots

- Drone technology helps users improve their crop yield & reduce costs.
- Drones offer high-quality imaging that can help monitor crops while scanning and analyzing fields to collect necessary agricultural data.
- Drones that use AI help farmers to scan their fields and monitor every stage of the production cycle.
- Agricultural drones allow farmers see their fields from the sky, this birds-eye view would expose intending issues on the farm such as irrigation problems, soil variation as well as pest and fungal infestations.

UAVs advantages

- They have transformed the idea of the air power in modern war times.
- They are smaller than jet aircraft.
- They are less expensive.
- They don't put the pilots at risk when they crash.
- Unmanned Aerial Vehicle offers less stressful environment.
- It presents safer environment.

UAVs disadvantages

- Unmanned Aerial Vehicles contains overall budget problems.
- They are very expensive to produce and keep up (The costs may go down over time though).
- The human mistake in the remote controls can cause the plane to crash and they have limited abilities.

UNIT – 1

ROBOT LOCOMOTION – QUESTION BANK

PART - A

1. Define AI.

Artificial intelligence (AI) is the ability of machines to replicate or enhance human intellect, such as reasoning and learning from experience. Artificial intelligence has been used in computer programs for years, but it is now applied to many other products and services.

2. Define Robotics.

Robotics is a branch of engineering and computer science that involves the conception, design, manufacture and operation of robots. The objective of the robotics field is to create intelligent machines that can assist humans in a variety of ways.

3. Explain the usage of AI in Robotics.

One of the key ways in which AI is used in robotics is through machine learning. This technique enables robots to learn and perform specific tasks through observing and mimicking human actions. AI gives robots a computer vision that enables them to navigate, detect and determine their reactions accordingly.

4. List the application of AI.

- ✓ Astronomy
- ✓ Healthcare
- ✓ Gaming
- ✓ Social Media
- ✓ Robotics

5. List the application of Robotics?

- ✓ Underwater Exploration
- ✓ Food Preparation
- ✓ Manufacturing
- ✓ Military
- ✓ Customer Service

6. Define Robot locomotion.

Locomotion is the method of moving from one place to another. The mechanism that makes a robot capable of moving in its environment is called as robot locomotion. That is to say, robot locomotion is the technique of aligning multiple tools and solutions for the movement of robots. It also is the mobilization of robots within a specific environment.

7. What are the key issues in robot locomotion?

- ✓ **Stability:**
 - Quantity, Position, and Geometry of contact points.
 - Gravitational Centre Point.
 - Stability during static and dynamic status.
 - The propensity of the surface, environment, and terrain.
- ✓ **Characteristics of Contact:**
 - The point of contact is the path's size and shape to reach it.
 - The angular position of the Contact.
 - The amount of friction, abrasions, and obstacles to reach the point of contact.
- ✓ **Type of Environment:**
 - The structure, formation, and composition of the environment.
 - The medium and surface tension of the environment.

8. List the types of mobile robot locomotion.

- ✓ Wheeled locomotion
- ✓ Legged locomotion
- ✓ Aerial locomotion

9. Define legged mobile robot and List the types of legged mobile robots.

Legged robots are mobile robots that use mechanical limbs for movement, they are similar to wheeled robots, but their locomotion methods are more complicated compared to their wheeled counterparts, they perform much better than wheeled robots on uneven terrain, Legged robots are indispensable for most applications, such as

- ✓ One-legged robots
- ✓ Two-legged robots
- ✓ Four-legged robots
- ✓ Six-legged robots

10. Define leg configuration and stability in legged mobile robots.

The leg configuration of the robot is a series parallel hybrid mechanism with five active joints and six passive joints. The mechanism is a spring mass model that interacts organically with the environment and mimics the characteristics of human walking well.

To achieve statically stable walking a robot must have a minimum number of four legs, because during walking at least one leg is in the air. Statically stable walking means that all robots' motion can be stopped at every moment in the gait cycle without overturning.

11. Define wheeled mobile robots and list its types.

Wheeled robots can change their positions with the help of their wheels, Wheeled robot motion can be achieved easily & its cost is pretty low, control of wheeled movement is easier, So, wheeled robots are one of the most frequently seen robots,

- ✓ Single wheeled robots
- ✓ Two-wheeled robots
- ✓ Three-wheel robots
- ✓ Four-wheeled robots

12. Explain the types of wheels are used in wheeled mobile robots.

- ✓ **Castor wheel** - It rotates around the offset steering joint and wheel axle.
- ✓ **Standard wheel** - It rotates around the contact and the wheel axle.
- ✓ **Ball or spherical wheel** - This wheel is technically difficult to implement due to architectural complexity. It is an Omni directional wheel with only one-directional movement is allowed.
- ✓ **Swedish 45 and Swedish 90 wheels** - It is an Omni-wheel, which rotates around the contact point, around the wheel axle, and the rollers.

13. Define drone.

Aerial Robots also called as unmanned aerial vehicle (UAV). The unmanned aerial vehicle (UAV) is known as the drone. The unmanned aircraft system (UAS) is the aircraft without the human pilot aboard. The flight of UAVs can operate either under the remote control by the human operator or fully or intermittently autonomously by onboard computers.

14. List the applications of aerial mobile robots.

- ✓ Remote-sensing
- ✓ Disaster response
- ✓ Image acquisition
- ✓ Surveillance
- ✓ Transportation
- ✓ Delivery of Goods

15. Write down the importance of drones.

- ✓ Drone technology helps users improve their crop yield & reduce costs.
- ✓ Drones offer high-quality imaging that can help monitor crops while scanning and analyzing fields to collect necessary agricultural data.
- ✓ Drones that uses AI help farmers to scan their fields and monitor every stage of the production cycle.

- ✓ Agricultural drones allow farmers see their fields from the sky, this birds-eye view would expose intending issues on the farm such as irrigation problems, soil variation as well as pest and fungal infestations.

PART – B

- 1. Explain in the details about robot locomotion.**
- 2. Define legged mobile robots. Explain in the types of legged mobile robots.**
- 3. Define wheeled mobile robots. Explain in the types of wheeled mobile robots.**
- 4. Define Arial Mobile robots. Write the application and importance of drones.**

3

Mobile Robot Kinematics

3.1 Introduction

Kinematics is the most basic study of how mechanical systems behave. In mobile robotics, we need to understand the mechanical behavior of the robot both in order to design appropriate mobile robots for tasks and to understand how to create control software for an instance of mobile robot hardware.

Of course, mobile robots are not the first complex mechanical systems to require such analysis. Robot manipulators have been the subject of intensive study for more than thirty years. In some ways, manipulator robots are much more complex than early mobile robots: a standard welding robot may have five or more joints, whereas early mobile robots were simple differential-drive machines. In recent years, the robotics community has achieved a fairly complete understanding of the kinematics and even the dynamics (i.e., relating to force and mass) of robot manipulators [11, 32].

The mobile robotics community poses many of the same kinematic questions as the robot manipulator community. A manipulator robot's workspace is crucial because it defines the range of possible positions that can be achieved by its end effector relative to its fixture to the environment. A mobile robot's workspace is equally important because it defines the range of possible poses that the mobile robot can achieve in its environment. The robot arm's controllability defines the manner in which active engagement of motors can be used to move from pose to pose in the workspace. Similarly, a mobile robot's controllability defines possible paths and trajectories in its workspace. Robot dynamics places additional constraints on workspace and trajectory due to mass and force considerations. The mobile robot is also limited by dynamics; for instance, a high center of gravity limits the practical turning radius of a fast, car-like robot because of the danger of rolling.

But the chief difference between a mobile robot and a manipulator arm also introduces a significant challenge for *position estimation*. A manipulator has one end fixed to the environment. Measuring the position of an arm's end effector is simply a matter of understanding the kinematics of the robot and measuring the position of all intermediate joints. The manipulator's position is thus always computable by looking at current sensor data. But a

mobile robot is a self-contained automaton that can wholly move with respect to its environment. There is no direct way to measure a mobile robot's position instantaneously. Instead, one must integrate the motion of the robot over time. Add to this the inaccuracies of *motion estimation* due to slippage and it is clear that measuring a mobile robot's position precisely is an extremely challenging task.

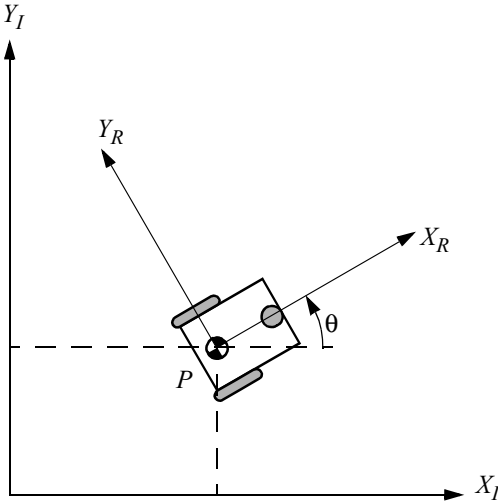
The process of understanding the motions of a robot begins with the process of describing the contribution each wheel provides for motion. Each wheel has a role in enabling the whole robot to move. By the same token, each wheel also imposes constraints on the robot's motion; for example, refusing to skid laterally. In the following section, we introduce notation that allows expression of robot motion in a global reference frame as well as the robot's local reference frame. Then, using this notation, we demonstrate the construction of simple forward kinematic models of motion, describing how the robot as a whole moves as a function of its geometry and individual wheel behavior. Next, we formally describe the kinematic constraints of individual wheels, and then combine these kinematic constraints to express the whole robot's kinematic constraints. With these tools, one can evaluate the paths and trajectories that define the robot's maneuverability.

3.2 Kinematic Models and Constraints

Deriving a model for the whole robot's motion is a bottom-up process. Each individual wheel contributes to the robot's motion and, at the same time, imposes constraints on robot motion. Wheels are tied together based on robot chassis geometry, and therefore their constraints combine to form constraints on the overall motion of the robot chassis. But the forces and constraints of each wheel must be expressed with respect to a clear and consistent reference frame. This is particularly important in mobile robotics because of its self-contained and mobile nature; a clear mapping between global and local frames of reference is required. We begin by defining these reference frames formally, then using the resulting formalism to annotate the kinematics of individual wheels and whole robots. Throughout this process we draw extensively on the notation and terminology presented in [52].

3.2.1 Representing robot position

Throughout this analysis we model the robot as a rigid body on wheels, operating on a horizontal plane. The total dimensionality of this robot chassis on the plane is three, two for position in the plane and one for orientation along the vertical axis, which is orthogonal to the plane. Of course, there are additional degrees of freedom and flexibility due to the wheel axles, wheel steering joints, and wheel castor joints. However by robot *chassis* we refer only to the rigid body of the robot, ignoring the joints and degrees of freedom internal to the robot and its wheels.

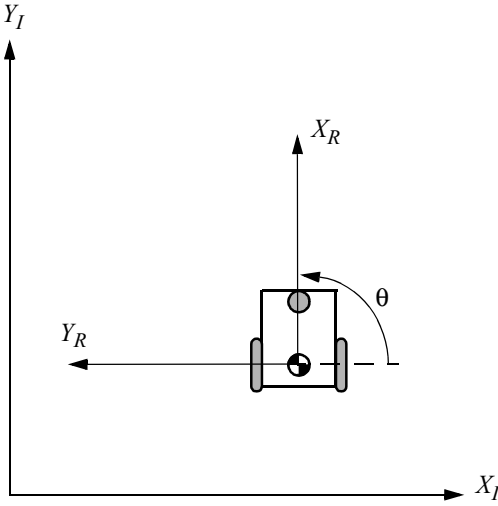
**Figure 3.1**

The global reference frame and the robot local reference frame.

In order to specify the position of the robot on the plane we establish a relationship between the global reference frame of the plane and the local reference frame of the robot, as in figure 3.1. The axes X_I and Y_I define an arbitrary inertial basis on the plane as the global reference frame from some origin O : $\{X_I, Y_I\}$. To specify the position of the robot, choose a point P on the robot chassis as its position reference point. The basis $\{X_R, Y_R\}$ defines two axes relative to P on the robot chassis and is thus the robot's local reference frame. The position of P in the global reference frame is specified by coordinates x and y , and the angular difference between the global and local reference frames is given by θ . We can describe the pose of the robot as a vector with these three elements. Note the use of the subscript I to clarify the basis of this pose as the global reference frame:

$$\xi_I = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \quad (3.1)$$

To describe robot motion in terms of component motions, it will be necessary to map motion along the axes of the global reference frame to motion along the axes of the robot's local reference frame. Of course, the mapping is a function of the current pose of the robot. This mapping is accomplished using the *orthogonal rotation matrix*:

**Figure 3.2**

The mobile robot aligned with a global axis.

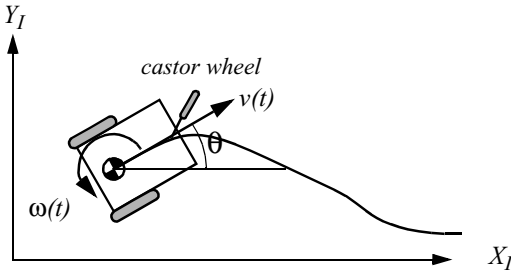
$$R(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3.2)$$

This matrix can be used to map motion in the global reference frame $\{X_I, Y_I\}$ to motion in terms of the local reference frame $\{X_R, Y_R\}$. This operation is denoted by $R(\theta)\xi_I$ because the computation of this operation depends on the value of θ :

$$\dot{\xi}_R = R\left(\frac{\pi}{2}\right)\dot{\xi}_I \quad (3.3)$$

For example, consider the robot in figure 3.2. For this robot, because $\theta = \frac{\pi}{2}$ we can easily compute the instantaneous rotation matrix R :

$$R\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3.4)$$

**Figure 3.3**

A differential-drive robot in its global reference frame.

Given some velocity $(\dot{x}, \dot{y}, \dot{\theta})$ in the global reference frame we can compute the components of motion along this robot's local axes X_R and Y_R . In this case, due to the specific angle of the robot, motion along X_R is equal to $\dot{y}$ and motion along Y_R is $-\dot{x}$:

$$\xi'_R = R\left(\frac{\pi}{2}\right)\xi_I = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \dot{y} \\ -\dot{x} \\ \dot{\theta} \end{bmatrix} \quad (3.5)$$

3.2.2 Forward kinematic models

In the simplest cases, the mapping described by equation (3.3) is sufficient to generate a formula that captures the forward kinematics of the mobile robot: how does the robot move, given its geometry and the speeds of its wheels? More formally, consider the example shown in figure 3.3.

This differential drive robot has two wheels, each with diameter r . Given a point P centered between the two drive wheels, each wheel is a distance l from P . Given r , l , θ , and the spinning speed of each wheel, ϕ_1 and ϕ_2 , a forward kinematic model would predict the robot's overall speed in the global reference frame:

$$\xi_I = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = f(l, r, \theta, \phi_1, \phi_2) \quad (3.6)$$

From equation (3.3) we know that we can compute the robot's motion in the global reference frame from motion in its local reference frame: $\xi_I = R(\theta)^{-1}\xi'_R$. Therefore, the strategy will be to first compute the contribution of each of the two wheels in the local reference

frame, $\dot{\xi}_R$. For this example of a differential-drive chassis, this problem is particularly straightforward.

Suppose that the robot's local reference frame is aligned such that the robot moves forward along $+X_R$, as shown in figure 3.1. First consider the contribution of each wheel's spinning speed to the translation speed at P in the direction of $+X_R$. If one wheel spins while the other wheel contributes nothing and is stationary, since P is halfway between the two wheels, it will move instantaneously with half the speed: $\dot{x}_{r1} = (1/2)r\dot{\phi}_1$ and $\dot{x}_{r2} = (1/2)r\dot{\phi}_2$. In a differential drive robot, these two contributions can simply be added to calculate the x_R component of $\dot{\xi}_R$. Consider, for example, a differential robot in which each wheel spins with equal speed but in opposite directions. The result is a stationary, spinning robot. As expected, $\dot{x}_R$ will be zero in this case. The value of $\dot{y}_R$ is even simpler to calculate. Neither wheel can contribute to sideways motion in the robot's reference frame, and so $\dot{y}_R$ is always zero. Finally, we must compute the rotational component $\dot{\theta}_R$ of $\dot{\xi}_R$. Once again, the contributions of each wheel can be computed independently and just added. Consider the right wheel (we will call this wheel 1). Forward spin of this wheel results in *counterclockwise* rotation at point P . Recall that if wheel 1 spins alone, the robot pivots around wheel 2. The rotation velocity ω_1 at P can be computed because the wheel is instantaneously moving along the arc of a circle of radius $2l$:

$$\omega_1 = \frac{r\dot{\phi}_1}{2l} \quad (3.7)$$

The same calculation applies to the left wheel, with the exception that forward spin results in *clockwise* rotation at point P :

$$\omega_2 = \frac{-r\dot{\phi}_2}{2l} \quad (3.8)$$

Combining these individual formulas yields a kinematic model for the differential-drive example robot:

$$\dot{\xi}_I = R(\theta)^{-1} \begin{bmatrix} \frac{r\dot{\phi}_1}{2} + \frac{r\dot{\phi}_2}{2} \\ 0 \\ \frac{r\dot{\phi}_1}{2l} + \frac{-r\dot{\phi}_2}{2l} \end{bmatrix} \quad (3.9)$$

We can now use this kinematic model in an example. However, we must first compute $R(\theta)^{-1}$. In general, calculating the inverse of a matrix may be challenging. In this case, however, it is easy because it is simply a transform from ξ_R to ξ_I rather than vice versa:

$$R(\theta)^{-1} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3.10)$$

Suppose that the robot is positioned such that $\theta = \pi/2$, $r = 1$, and $l = 1$. If the robot engages its wheels unevenly, with speeds $\dot{\phi}_1 = 4$ and $\dot{\phi}_2 = 2$, we can compute its velocity in the global reference frame:

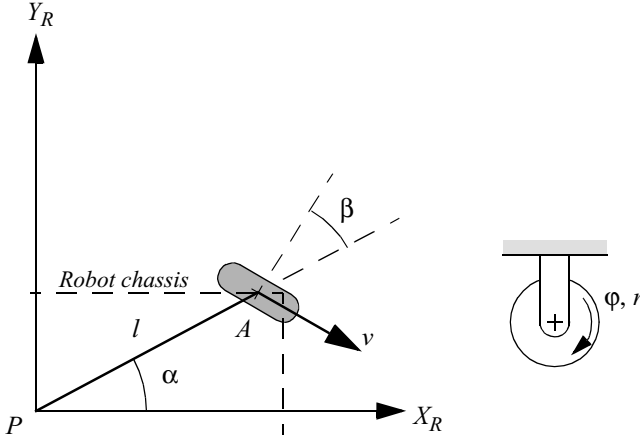
$$\xi_I = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \\ 1 \end{bmatrix} \quad (3.11)$$

So this robot will move instantaneously along the y -axis of the global reference frame with speed 3 while rotating with speed 1. This approach to kinematic modeling can provide information about the motion of a robot given its component wheel speeds in straightforward cases. However, we wish to determine the space of possible motions for each robot chassis design. To do this, we must go further, describing formally the constraints on robot motion imposed by each wheel. Section 3.2.3 begins this process by describing constraints for various wheel types; the rest of this chapter provides tools for analyzing the characteristics and workspace of a robot given these constraints.

3.2.3 Wheel kinematic constraints

The first step to a kinematic model of the robot is to express constraints on the motions of individual wheels. Just as shown in section 3.2.2, the motions of individual wheels can later be combined to compute the motion of the robot as a whole. As discussed in chapter 2, there are four basic wheel types with widely varying kinematic properties. Therefore, we begin by presenting sets of constraints specific to each wheel type.

However, several important assumptions will simplify this presentation. We assume that the plane of the wheel always remains vertical and that there is in all cases one single point of contact between the wheel and the ground plane. Furthermore, we assume that there is no sliding at this single point of contact. That is, the wheel undergoes motion only under conditions of pure rolling and rotation about the vertical axis through the contact point. For a more thorough treatment of kinematics, including sliding contact, refer to [25].

**Figure 3.4**

A fixed standard wheel and its parameters.

Under these assumptions, we present two constraints for every wheel type. The first constraint enforces the concept of rolling contact – that the wheel must roll when motion takes place in the appropriate direction. The second constraint enforces the concept of no lateral slippage – that the wheel must not slide orthogonal to the wheel plane.

3.2.3.1 Fixed standard wheel

The fixed standard wheel has no vertical axis of rotation for steering. Its angle to the chassis is thus fixed, and it is limited to motion back and forth along the wheel plane and rotation around its contact point with the ground plane. Figure 3.4 depicts a fixed standard wheel A and indicates its position pose relative to the robot's local reference frame $\{X_R, Y_R\}$. The position of A is expressed in polar coordinates by distance l and angle α . The angle of the wheel plane relative to the chassis is denoted by β , which is fixed since the fixed standard wheel is not steerable. The wheel, which has radius r , can spin over time, and so its rotational position around its horizontal axle is a function of time t : $\phi(t)$.

The rolling constraint for this wheel enforces that all motion along the direction of the wheel plane must be accompanied by the appropriate amount of wheel spin so that there is pure rolling at the contact point:

$$\left[\sin(\alpha + \beta) - \cos(\alpha + \beta) (-l) \cos \beta \right] R(\theta) \dot{\xi}_l - r \dot{\phi} = 0 \quad (3.12)$$

The first term of the sum denotes the total motion along the wheel plane. The three elements of the vector on the left represent mappings from each of $\dot{x}, \dot{y}, \dot{\theta}$ to their contributions for motion along the wheel plane. Note that the $R(\theta)\dot{\xi}_I$ term is used to transform the motion parameters $\dot{\xi}_I$ that are in the global reference frame $\{X_I, Y_I\}$ into motion parameters in the local reference frame $\{X_R, Y_R\}$ as shown in example equation (3.5). This is necessary because all other parameters in the equation, α, β, l , are in terms of the robot's local reference frame. This motion along the wheel plane must be equal, according to this constraint, to the motion accomplished by spinning the wheel, $r\dot{\phi}$.

The sliding constraint for this wheel enforces that the component of the wheel's motion orthogonal to the wheel plane must be zero:

$$\begin{bmatrix} \cos(\alpha + \beta) & \sin(\alpha + \beta) & l\sin\beta \end{bmatrix} R(\theta)\dot{\xi}_I = 0 \quad (3.13)$$

For example, suppose that wheel A is in a position such that $\{(\alpha = 0), (\beta = 0)\}$. This would place the contact point of the wheel on X_I with the plane of the wheel oriented parallel to Y_I . If $\theta = 0$, then the sliding constraint [equation (3.13)] reduces to

$$\begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = 0 \quad (3.14)$$

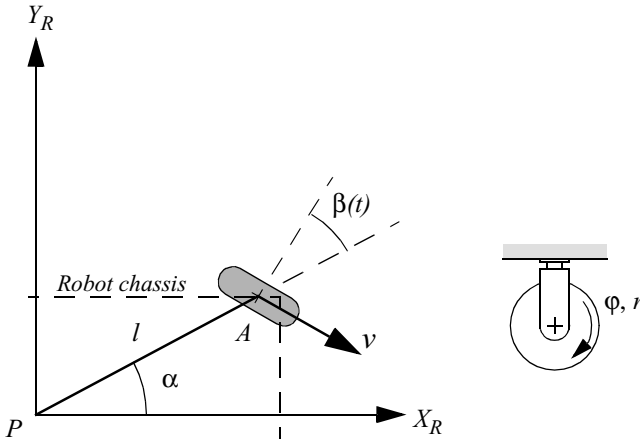
This constrains the component of motion along X_I to be zero and since X_I and X_R are parallel in this example, the wheel is constrained from sliding sideways, as expected.

3.2.3.2 Steered standard wheel

The steered standard wheel differs from the fixed standard wheel only in that there is an additional degree of freedom: the wheel may rotate around a vertical axis passing through the center of the wheel and the ground contact point. The equations of position for the steered standard wheel (figure 3.5) are identical to that of the fixed standard wheel shown in figure 3.4 with one exception. The orientation of the wheel to the robot chassis is no longer a single fixed value, β , but instead varies as a function of time: $\beta(t)$. The rolling and sliding constraints are

$$\begin{bmatrix} \sin(\alpha + \beta) & -\cos(\alpha + \beta) & (-l)\cos\beta \end{bmatrix} R(\theta)\dot{\xi}_I - r\dot{\phi} = 0 \quad (3.15)$$

$$\begin{bmatrix} \cos(\alpha + \beta) & \sin(\alpha + \beta) & l\sin\beta \end{bmatrix} R(\theta)\dot{\xi}_I = 0 \quad (3.16)$$

**Figure 3.5**

A steered standard wheel and its parameters.

These constraints are identical to those of the fixed standard wheel because, unlike $\dot{\phi}$, $\dot{\beta}$ does not have a direct impact on the instantaneous motion constraints of a robot. It is only by integrating over time that changes in steering angle can affect the mobility of a vehicle. This may seem subtle, but is a very important distinction between change in steering position, β , and change in wheel spin, ϕ .

3.2.3.3 Castor wheel

Castor wheels are able to steer around a vertical axis. However, unlike the steered standard wheel, the vertical axis of rotation in a castor wheel does not pass through the ground contact point. Figure 3.6 depicts a castor wheel, demonstrating that formal specification of the castor wheel's position requires an additional parameter.

The wheel contact point is now at position B , which is connected by a rigid rod AB of fixed length d to point A fixes the location of the vertical axis about which B steers, and this point A has a position specified in the robot's reference frame, as in figure 3.6. We assume that the plane of the wheel is aligned with AB at all times. Similar to the steered standard wheel, the castor wheel has two parameters that vary as a function of time. $\phi(t)$ represents the wheel spin over time as before. $\beta(t)$ denotes the steering angle and orientation of AB over time.

For the castor wheel, the rolling constraint is identical to equation (3.15) because the offset axis plays no role during motion that is aligned with the wheel plane:

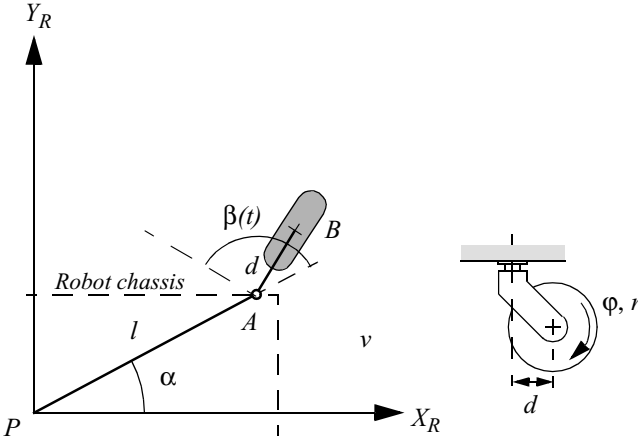


Figure 3.6
A castor wheel and its parameters.

$$\begin{bmatrix} \sin(\alpha + \beta) & -\cos(\alpha + \beta) & (-l) \cos \beta \end{bmatrix} R(\theta) \dot{\xi}_I - r \dot{\phi} = 0 \quad (3.17)$$

The castor geometry does, however, have significant impact on the sliding constraint. The critical issue is that the lateral force on the wheel occurs at point A because this is the attachment point of the wheel to the chassis. Because of the offset ground contact point relative to A , the constraint that there be zero lateral movement would be wrong. Instead, the constraint is much like a rolling constraint, in that appropriate rotation of the vertical axis must take place:

$$\begin{bmatrix} \cos(\alpha + \beta) & \sin(\alpha + \beta) & d + l \sin \beta \end{bmatrix} R(\theta) \dot{\xi}_I + d \dot{\beta} = 0 \quad (3.18)$$

In equation (3.18), any motion orthogonal to the wheel plane must be balanced by an equivalent and opposite amount of castor steering motion. This result is critical to the success of castor wheels because by setting the value of $\dot{\beta}$ any arbitrary lateral motion can be acceptable. In a steered standard wheel, the steering action does not by itself cause a movement of the robot chassis. But in a castor wheel the steering action itself moves the robot chassis because of the offset between the ground contact point and the vertical axis of rotation.



Figure 3.7
Office chair with five castor wheels.

More concisely, it can be surmised from equations (3.17) and (3.18) that, given *any* robot chassis motion ξ_I , there exists some value for spin speed $\dot{\phi}$ and steering speed $\dot{\beta}$ such that the constraints are met. Therefore, a robot with only castor wheels can move with any velocity in the space of possible robot motions. We term such systems *omnidirectional*.

A real-world example of such a system is the five-castor wheel office chair shown in figure 3.7. Assuming that all joints are able to move freely, you may select any motion vector on the plane for the chair and push it by hand. Its castor wheels will spin and steer as needed to achieve that motion without contact point sliding. By the same token, if each of the chair's castor wheels housed two motors, one for spinning and one for steering, then a control system would be able to move the chair along any trajectory in the plane. Thus, although the kinematics of castor wheels is somewhat complex, such wheels do not impose any real constraints on the kinematics of a robot chassis.

3.2.3.4 Swedish wheel

Swedish wheels have no vertical axis of rotation, yet are able to move *omnidirectionally* like the castor wheel. This is possible by adding a degree of freedom to the fixed standard wheel. Swedish wheels consist of a fixed standard wheel with rollers attached to the wheel perimeter with axes that are antiparallel to the main axis of the fixed wheel component. The exact angle γ between the roller axes and the main axis can vary, as shown in figure 3.8.

For example, given a Swedish 45-degree wheel, the motion vectors of the principal axis and the roller axes can be drawn as in figure 3.8. Since each axis can spin clockwise or counterclockwise, one can combine any vector along one axis with any vector along the other axis. These two axes are not necessarily independent (except in the case of the Swedish 90-degree wheel); however, it is visually clear that any desired direction of motion is achievable by choosing the appropriate two vectors.

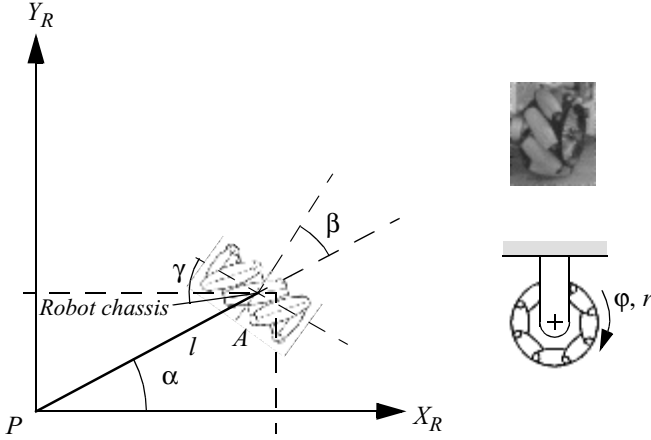


Figure 3.8
A Swedish wheel and its parameters.

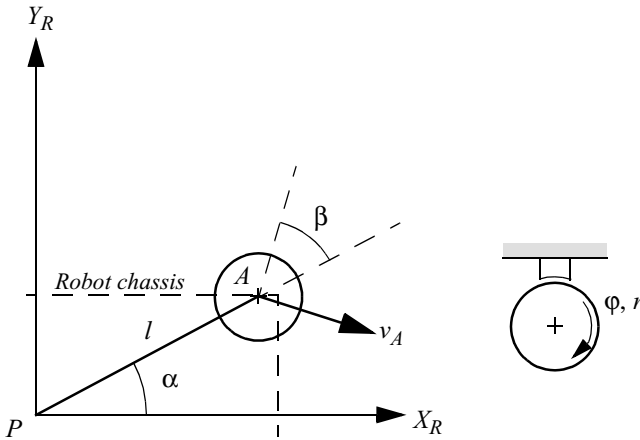
The pose of a Swedish wheel is expressed exactly as in a fixed standard wheel, with the addition of a term, γ , representing the angle between the main wheel plane and the axis of rotation of the small circumferential rollers. This is depicted in figure 3.8 within the robot's reference frame.

Formulating the constraint for a Swedish wheel requires some subtlety. The instantaneous constraint is due to the specific orientation of the small rollers. The axis around which these rollers spin is a zero component of velocity at the contact point. That is, moving in that direction without spinning the main axis is not possible without sliding. The motion constraint that is derived looks identical to the rolling constraint for the fixed standard wheel in equation (3.12) except that the formula is modified by adding γ such that the effective direction along which the rolling constraint holds is along this zero component rather than along the wheel plane:

$$\left[\sin(\alpha + \beta + \gamma) - \cos(\alpha + \beta + \gamma) (-l) \cos(\beta + \gamma) \right] R(\theta) \dot{\xi}_l - r \dot{\phi} \cos \gamma = 0 \quad (3.19)$$

Orthogonal to this direction the motion is not constrained because of the free rotation $\dot{\phi}_{sw}$ of the small rollers.

$$\left[\cos(\alpha + \beta + \gamma) \sin(\alpha + \beta + \gamma) l \sin(\beta + \gamma) \right] R(\theta) \dot{\xi}_l - r \dot{\phi} \sin \gamma - r_{sw} \dot{\phi}_{sw} = 0 \quad (3.20)$$

**Figure 3.9**

A spherical wheel and its parameters.

The behavior of this constraint and thereby the Swedish wheel changes dramatically as the value γ varies. Consider $\gamma = 0$. This represents the Swedish 90-degree wheel. In this case, the zero component of velocity is in line with the wheel plane and so equation (3.19) reduces exactly to equation (3.12), the fixed standard wheel rolling constraint. But because of the rollers, there is no sliding constraint orthogonal to the wheel plane [see equation (3.20)]. By varying the value of ϕ , any desired motion vector can be made to satisfy equation (3.19) and therefore the wheel is omnidirectional. In fact, this special case of the Swedish design results in fully decoupled motion, in that the rollers and the main wheel provide orthogonal directions of motion.

At the other extreme, consider $\gamma = \pi/2$. In this case, the rollers have axes of rotation that are parallel to the main wheel axis of rotation. Interestingly, if this value is substituted for γ in equation (3.19) the result is the fixed standard wheel sliding constraint, equation (3.13). In other words, the rollers provide no benefit in terms of lateral freedom of motion since they are simply aligned with the main wheel. However, in this case the main wheel never needs to spin and therefore the rolling constraint disappears. This is a degenerate form of the Swedish wheel and therefore we assume in the remainder of this chapter that $\gamma \neq \pi/2$.

3.2.3.5 Spherical wheel

The final wheel type, a ball or spherical wheel, places no direct constraints on motion (figure 3.9). Such a mechanism has no principal axis of rotation, and therefore no appropriate rolling or sliding constraints exist. As with castor wheels and Swedish wheels, the spherical

wheel is clearly omnidirectional and places no constraints on the robot chassis kinematics. Therefore equation (3.21) simply describes the roll rate of the ball in the direction of motion v_A of point A of the robot.

$$\left[\sin(\alpha + \beta) \quad -\cos(\alpha + \beta) \quad (-l) \cos \beta \right] R(\theta) \dot{\xi}_I - r \dot{\phi} = 0 \quad (3.21)$$

By definition the wheel rotation orthogonal to this direction is zero.

$$\left[\cos(\alpha + \beta) \quad \sin(\alpha + \beta) \quad l \sin \beta \right] R(\theta) \dot{\xi}_I = 0 \quad (3.22)$$

As can be seen, the equations for the spherical wheel are exactly the same as for the fixed standard wheel. However, the interpretation of equation (3.22) is different. The omnidirectional spherical wheel can have any arbitrary direction of movement, where the motion direction given by β is a free variable deduced from equation (3.22). Consider the case that the robot is in pure translation in the direction of Y_R . Then equation (3.22) reduces to $\sin(\alpha + \beta) = 0$, thus $\beta = -\alpha$, which makes sense for this special case.

3.2.4 Robot kinematic constraints

Given a mobile robot with M wheels we can now compute the kinematic constraints of the robot chassis. The key idea is that each wheel imposes zero or more constraints on robot motion, and so the process is simply one of appropriately combining all of the kinematic constraints arising from all of the wheels based on the placement of those wheels on the robot chassis.

We have categorized all wheels into five categories: (1) fixed and (2) steerable standard wheels, (3) castor wheels, (4) Swedish wheels, and (5) spherical wheels. But note from the wheel kinematic constraints in equations (3.17), (3.18), and (3.19) that the castor wheel, Swedish wheel, and spherical wheel impose *no* kinematic constraints on the robot chassis, since $\dot{\xi}_I$ can range freely in all of these cases owing to the internal wheel degrees of freedom.

Therefore only fixed standard wheels and steerable standard wheels have impact on robot chassis kinematics and therefore require consideration when computing the robot's kinematic constraints. Suppose that the robot has a total of N standard wheels, comprising N_f fixed standard wheels and N_s steerable standard wheels. We use $\beta_s(t)$ to denote the variable steering angles of the N_s steerable standard wheels. In contrast, β_f refers to the orientation of the N_f fixed standard wheels as depicted in figure 3.4. In the case of wheel spin, both the fixed and steerable wheels have rotational positions around the horizontal axle that vary as a function of time. We denote the fixed and steerable cases separately as $\phi_f(t)$ and $\phi_s(t)$, and use $\phi(t)$ as an aggregate matrix that combines both values:

$$\varphi(t) = \begin{bmatrix} \varphi_f(t) \\ \varphi_s(t) \end{bmatrix} \quad (3.23)$$

The rolling constraints of all wheels can now be collected in a single expression:

$$J_1(\beta_s)R(\theta)\dot{\xi}_I - J_2\dot{\phi} = 0 \quad (3.24)$$

This expression bears a strong resemblance to the rolling constraint of a single wheel, but substitutes matrices in lieu of single values, thus taking into account all wheels. J_2 is a constant diagonal $N \times N$ matrix whose entries are radii r of all standard wheels. $J_1(\beta_s)$ denotes a matrix with projections for all wheels to their motions along their individual wheel planes:

$$J_1(\beta_s) = \begin{bmatrix} J_{1f} \\ J_{1s}(\beta_s) \end{bmatrix} \quad (3.25)$$

Note that $J_1(\beta_s)$ is only a function of β_s and not β_f . This is because the orientations of steerable standard wheels vary as a function of time, whereas the orientations of fixed standard wheels are constant. J_{1f} is therefore a constant matrix of projections for all fixed standard wheels. It has size $(N_f \times 3)$, with each row consisting of the three terms in the three-matrix from equation (3.12) for each fixed standard wheel. $J_{1s}(\beta_s)$ is a matrix of size $(N_s \times 3)$, with each row consisting of the three terms in the three-matrix from equation (3.15) for each steerable standard wheel.

In summary, equation (3.24) represents the constraint that all standard wheels must spin around their horizontal axis an appropriate amount based on their motions along the wheel plane so that rolling occurs at the ground contact point.

We use the same technique to collect the sliding constraints of all standard wheels into a single expression with the same structure as equations (3.13) and (3.16):

$$C_1(\beta_s)R(\theta)\dot{\xi}_I = 0 \quad (3.26)$$

$$C_1(\beta_s) = \begin{bmatrix} C_{1f} \\ C_{1s}(\beta_s) \end{bmatrix} \quad (3.27)$$

C_{1f} and C_{1s} are $(N_f \times 3)$ and $(N_s \times 3)$ matrices whose rows are the three terms in the three-matrix of equations (3.13) and (3.16) for all fixed and steerable standard wheels. Thus

equation (3.26) is a constraint over all standard wheels that their components of motion orthogonal to their wheel planes must be zero. This sliding constraint over all standard wheels has the most significant impact on defining the overall maneuverability of the robot chassis, as explained in the next section.

3.2.5 Examples: robot kinematic models and constraints

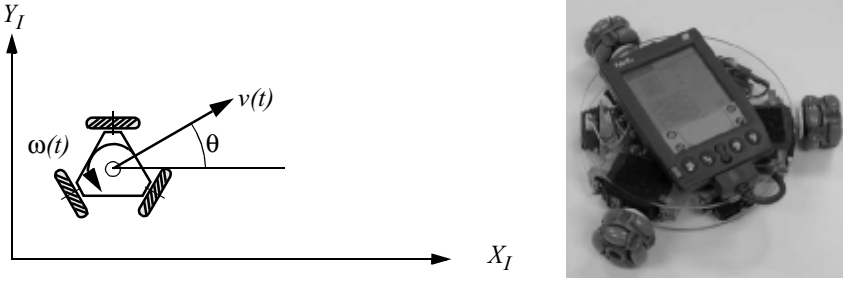
In section 3.2.2 we presented a forward kinematic solution for $\dot{\xi}_I$ in the case of a simple differential-drive robot by combining each wheel's contribution to robot motion. We can now use the tools presented above to construct the same kinematic expression by direct application of the rolling constraints for every wheel type. We proceed with this technique applied again to the differential drive robot, enabling verification of the method as compared to the results of section 3.2.2. Then we proceed to the case of the three-wheeled omnidirectional robot.

3.2.5.1 A differential-drive robot example

First, refer to equations (3.24) and (3.26). These equations relate robot motion to the rolling and sliding constraints $J_1(\beta_s)$ and $C_1(\beta_s)$, and the wheel spin speed of the robot's wheels, $\dot{\phi}$. Fusing these two equations yields the following expression:

$$\begin{bmatrix} J_1(\beta_s) \\ C_1(\beta_s) \end{bmatrix} R(\theta) \dot{\xi}_I = \begin{bmatrix} J_2 \dot{\phi} \\ 0 \end{bmatrix} \quad (3.28)$$

Once again, consider the differential drive robot in figure 3.3. We will construct $J_1(\beta_s)$ and $C_1(\beta_s)$ directly from the rolling constraints of each wheel. The castor is unpowered and is free to move in any direction, so we ignore this third point of contact altogether. The two remaining drive wheels are not steerable, and therefore $J_1(\beta_s)$ and $C_1(\beta_s)$ simplify to J_{1f} and C_{1f} respectively. To employ the fixed standard wheel's rolling constraint formula, equation (3.12), we must first identify each wheel's values for α and β . Suppose that the robot's local reference frame is aligned such that the robot moves forward along $+X_R$, as shown in figure 3.1. In this case, for the right wheel $\alpha = -\pi/2$, $\beta = \pi$, and for the left wheel, $\alpha = \pi/2$, $\beta = 0$. Note the value of β for the right wheel is necessary to ensure that positive spin causes motion in the $+X_R$ direction (figure 3.4). Now we can compute the J_{1f} and C_{1f} matrix using the matrix terms from equations (3.12) and (3.13). Because the two fixed standard wheels are parallel, equation (3.13) results in only one independent equation, and equation (3.28) gives

**Figure 3.10**

A three-wheel omnidrive robot developed by Carnegie Mellon University (www.cs.cmu.edu/~pprk).

$$\begin{bmatrix} \begin{bmatrix} 1 & 0 & l \\ 1 & 0 & -l \\ 0 & 1 & 0 \end{bmatrix} \end{bmatrix} R(\theta) \dot{\xi}_I = \begin{bmatrix} J_2 \Phi \\ 0 \end{bmatrix} \quad (3.29)$$

Inverting equation (3.29) yields the kinematic equation specific to our differential drive robot:

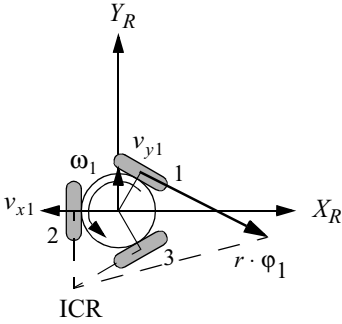
$$\dot{\xi}_I = R(\theta)^{-1} \begin{bmatrix} 1 & 0 & l \\ 1 & 0 & -l \\ 0 & 1 & 0 \end{bmatrix}^{-1} \begin{bmatrix} J_2 \Phi \\ 0 \end{bmatrix} = R(\theta)^{-1} \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 \\ \frac{1}{2l} & -\frac{1}{2l} & 0 \end{bmatrix} \begin{bmatrix} J_2 \Phi \\ 0 \end{bmatrix} \quad (3.30)$$

This demonstrates that, for the simple differential-drive case, the combination of wheel rolling and sliding constraints describes the kinematic behavior, based on our manual calculation in section 3.2.2.

3.2.5.2 An omnidirectional robot example

Consider the omniwheel robot shown in figure 3.10. This robot has three Swedish 90-degree wheels, arranged radially symmetrically, with the rollers perpendicular to each main wheel.

First we must impose a specific local reference frame upon the robot. We do so by choosing point P at the center of the robot, then aligning the robot with the local reference

**Figure 3.11**

The local reference frame plus detailed parameters for wheel 1.

frame such that X_R is colinear with the axis of wheel 2. Figure 3.11 shows the robot and its local reference frame arranged in this manner.

We assume that the distance between each wheel and P is l , and that all three wheels have the same radius, r . Once again, the value of $\dot{\xi}_I$ can be computed as a combination of the rolling constraints of the robot's three omnidirectional wheels, as in equation (3.28). As with the differential-drive robot, since this robot has no steerable wheels, $J_1(\beta_s)$ simplifies to J_{1f} :

$$\dot{\xi}_I = R(\theta)^{-1} J_{1f}^{-1} J_2 \dot{\phi} \quad (3.31)$$

We calculate J_{1f} using the matrix elements of the rolling constraints for the Swedish wheel, given by equation (3.19). But to use these values, we must establish the values α, β, γ for each wheel. Referring to figure (3.8), we can see that $\gamma = 0$ for the Swedish 90-degree wheel. Note that this immediately simplifies equation (3.19) to equation (3.12), the rolling constraints of a fixed standard wheel. Given our particular placement of the local reference frame, the value of α for each wheel is easily computed: $(\alpha_1 = \pi/3), (\alpha_2 = \pi), (\alpha_3 = -\pi/3)$. Furthermore, $\beta = 0$ for all wheels because the wheels are tangent to the robot's circular body. Constructing and simplifying J_{1f} using equation (3.12) yields

$$J_{1f} = \begin{bmatrix} \sin \frac{\pi}{3} & -\cos \frac{\pi}{3} & -l \\ 0 & -\cos \pi & -l \\ \sin -\frac{\pi}{3} & -\cos -\frac{\pi}{3} & -l \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} & -l \\ 0 & 1 & -l \\ -\frac{\sqrt{3}}{2} & -\frac{1}{2} & -l \end{bmatrix} \quad (3.32)$$

Once again, computing the value of ξ_I requires calculating the inverse, J_{1f}^{-1} , as needed in equation (3.31). One approach would be to apply rote methods for calculating the inverse of a 3 x 3 square matrix. A second approach would be to compute the contribution of each Swedish wheel to chassis motion, as shown in section 3.2.2. We leave this process as an exercise for the enthusiast. Once the inverse is obtained, ξ_I can be isolated:

$$\xi_I = R(\theta)^{-1} \begin{bmatrix} \frac{1}{\sqrt{3}} & 0 & -\frac{1}{\sqrt{3}} \\ -\frac{1}{3} & \frac{2}{3} & -\frac{1}{3} \\ -\frac{1}{3l} & -\frac{1}{3l} & -\frac{1}{3l} \end{bmatrix} J_2 \dot{\phi} \quad (3.33)$$

Consider a specific omnidrive chassis with $l = 1$ and $r = 1$ for all wheels. The robot's local reference frame and global reference frame are aligned, so that $\theta = 0$. If wheels 1, 2, and 3 spin at speeds $(\phi_1 = 4)$, $(\phi_2 = 1)$, $(\phi_3 = 2)$, what is the resulting motion of the whole robot? Using the equation above, the answer can be calculated readily:

$$\xi_I = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{3}} & 0 & -\frac{1}{\sqrt{3}} \\ -\frac{1}{3} & \frac{2}{3} & -\frac{1}{3} \\ -\frac{1}{3} & -\frac{1}{3} & -\frac{1}{3} \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 4 \\ 1 \\ 2 \end{bmatrix} = \begin{bmatrix} \frac{2}{\sqrt{3}} \\ -\frac{4}{3} \\ \frac{7}{3} \end{bmatrix} \quad (3.34)$$

So this robot will move instantaneously along the x -axis with positive speed and along the y axis with negative speed while rotating clockwise. We can see from the above examples that robot motion can be predicted by combining the rolling constraints of individual wheels.

The sliding constraints comprising $C_1(\beta_s)$ can be used to go even further, enabling us to evaluate the maneuverability and workspace of the robot rather than just its predicted motion. Next, we examine methods for using the sliding constraints, sometimes in conjunction with rolling constraints, to generate powerful analyses of the maneuverability of a robot chassis.

3.3 Mobile Robot Maneuverability

The kinematic mobility of a robot chassis is its ability to directly move in the environment. The basic constraint limiting mobility is the rule that every wheel must satisfy its sliding constraint. Therefore, we can formally derive robot mobility by starting from equation (3.26).

In addition to instantaneous kinematic motion, a mobile robot is able to further manipulate its position, over time, by steering steerable wheels. As we will see in section 3.3.3, the overall maneuverability of a robot is thus a combination of the mobility available based on the kinematic sliding constraints of the standard wheels, plus the additional freedom contributed by steering and spinning the steerable standard wheels.

3.3.1 Degree of mobility

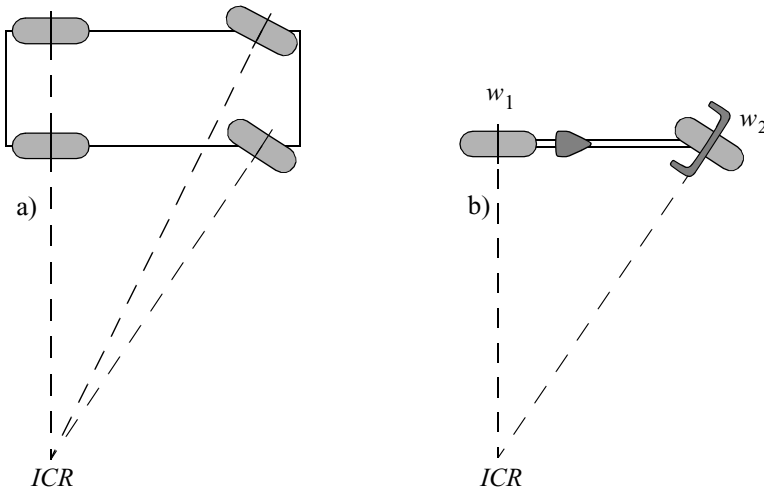
Equation (3.26) imposes the constraint that every wheel must avoid any lateral slip. Of course, this holds separately for each and every wheel, and so it is possible to specify this constraint separately for fixed and for steerable standard wheels:

$$C_{1f}R(\theta)\dot{\xi}_I = 0 \quad (3.35)$$

$$C_{1s}(\beta_s)R(\theta)\dot{\xi}_I = 0 \quad (3.36)$$

For both of these constraints to be satisfied, the motion vector $R(\theta)\dot{\xi}_I$ must belong to the *null space* of the projection matrix $C_1(\beta_s)$, which is simply a combination of C_{1f} and C_{1s} . Mathematically, the null space of $C_1(\beta_s)$ is the space N such that for any vector n in N , $C_1(\beta_s)n = 0$. If the kinematic constraints are to be honored, then the motion of the robot must always be within this space N . The kinematic constraints [equations (3.35) and (3.36)] can also be demonstrated geometrically using the concept of a robot's *instantaneous center of rotation (ICR)*.

Consider a single standard wheel. It is forced by the sliding constraint to have zero lateral motion. This can be shown geometrically by drawing a *zero motion line* through its horizontal axis, perpendicular to the wheel plane (figure 3.12). At any given instant, wheel motion along the zero motion line must be zero. In other words, the wheel must be moving

**Figure 3.12**

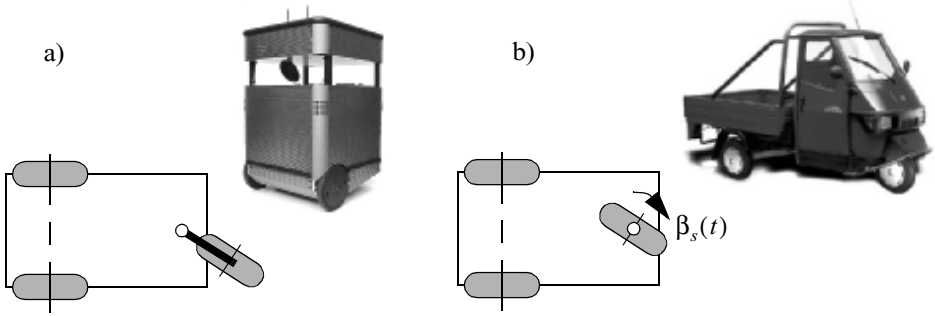
(a) Four-wheel with car-like Ackerman steering. (b) bicycle.

instantaneously along some circle of radius R such that the center of that circle is located on the zero motion line. This center point, called the instantaneous center of rotation, may lie anywhere along the zero motion line. When R is at infinity, the wheel moves in a straight line.

A robot such as the Ackerman vehicle in figure 3.12a can have several wheels, but must always have a single ICR. Because all of its zero motion lines meet at a single point, there is a single solution for robot motion, placing the ICR at this meet point.

This ICR geometric construction demonstrates how robot mobility is a function of the number of constraints on the robot's motion, not the number of wheels. In figure 3.12b, the bicycle shown has two wheels, w_1 and w_2 . Each wheel contributes a constraint, or a zero motion line. Taken together the two constraints result in a single point as the only remaining solution for the ICR. This is because the two constraints are independent, and thus each further constrains overall robot motion.

But in the case of the differential drive robot in figure 3.13a, the two wheels are aligned along the same horizontal axis. Therefore, the ICR is constrained to lie along a line, not at a specific point. In fact, the second wheel imposes no additional kinematic constraints on robot motion since its zero motion line is identical to that of the first wheel. Thus, although the bicycle and differential-drive chassis have the same number of nonomnidirectional wheels, the former has two independent kinematic constraints while the latter has only one.

**Figure 3.13**

(a) Differential drive robot with two individually motorized wheels and a castor wheel, e.g., the Pygmalion robot at EPFL. (b) Tricycle with two fixed standard wheels and one steered standard wheel, e.g. Piaggio minitransporter.

The Ackerman vehicle of figure 3.12a demonstrates another way in which a wheel may be unable to contribute an independent constraint to the robot kinematics. This vehicle has two steerable standard wheels. Given the instantaneous position of just one of these steerable wheels and the position of the fixed rear wheels, there is only a single solution for the *ICR*. The position of the second steerable wheel is absolutely constrained by the *ICR*. Therefore, it offers no independent constraints to robot motion.

Robot chassis kinematics is therefore a function of the set of *independent* constraints arising from all standard wheels. The mathematical interpretation of independence is related to the *rank* of a matrix. Recall that the rank of a matrix is the smallest number of independent rows or columns. Equation (3.26) represents all sliding constraints imposed by the wheels of the mobile robot. Therefore $\text{rank}[C_1(\beta_s)]$ is the number of independent constraints.

The greater the number of independent constraints, and therefore the greater the rank of $C_1(\beta_s)$, the more constrained is the mobility of the robot. For example, consider a robot with a single fixed standard wheel. Remember that we consider only standard wheels. This robot may be a unicycle or it may have several Swedish wheels; however, it has exactly one fixed standard wheel. The wheel is at a position specified by parameters α, β, l relative to the robot's local reference frame. $C_1(\beta_s)$ is comprised of C_{1f} and C_{1s} . However, since there are no steerable standard wheels C_{1s} is empty and therefore $C_1(\beta_s)$ contains only C_{1f} . Because there is one fixed standard wheel, this matrix has a rank of one and therefore this robot has a single independent constrain on mobility:

$$C_1(\beta_s) = C_{1f} = \begin{bmatrix} \cos(\alpha + \beta) & \sin(\alpha + \beta) & l \sin \beta \end{bmatrix} \quad (3.37)$$

Now let us add an additional fixed standard wheel to create a differential-drive robot by constraining the second wheel to be aligned with the same horizontal axis as the original wheel. Without loss of generality, we can place point P at the midpoint between the centers of the two wheels. Given α_1, β_1, l_1 for wheel w_1 and α_2, β_2, l_2 for wheel w_2 , it holds geometrically that $\{(l_1 = l_2), (\beta_1 = \beta_2 = 0), (\alpha_1 + \pi = \alpha_2)\}$. Therefore, in this case, the matrix $C_1(\beta_s)$ has two constraints but a rank of one:

$$C_1(\beta_s) = C_{1f} = \begin{bmatrix} \cos(\alpha_1) & \sin(\alpha_1) & 0 \\ \cos(\alpha_1 + \pi) & \sin(\alpha_1 + \pi) & 0 \end{bmatrix} \quad (3.38)$$

Alternatively, consider the case when w_2 is placed in the wheel plane of w_1 but with the same orientation, as in a bicycle with the steering locked in the forward position. We again place point P between the two wheel centers, and orient the wheels such that they lie on axis x_1 . This geometry implies that $\{(l_1 = l_2), (\beta_1 = \beta_2 = \pi/2), (\alpha_1 = 0), (\alpha_2 = \pi)\}$ and, therefore, the matrix $C_1(\beta_s)$ retains two independent constraints and has a rank of two:

$$C_1(\beta_s) = C_{1f} = \begin{bmatrix} \cos(\pi/2) & \sin(\pi/2) & l_1 \sin(\pi/2) \\ \cos(3\pi/2) & \sin(3\pi/2) & l_1 \sin(\pi/2) \end{bmatrix} = \begin{bmatrix} 0 & 1 & l_1 \\ 0 & -1 & l_1 \end{bmatrix} \quad (3.39)$$

In general, if $\text{rank}[C_{1f}] > 1$ then the vehicle can, at best, only travel along a circle or along a straight line. This configuration means that the robot has two or more independent constraints due to fixed standard wheels that do not share the same horizontal axis of rotation. Because such configurations have only a degenerate form of mobility in the plane, we do not consider them in the remainder of this chapter. Note, however, that some degenerate configurations such as the four-wheeled slip/skid steering system are useful in certain environments, such as on loose soil and sand, even though they fail to satisfy sliding constraints. Not surprisingly, the price that must be paid for such violations of the sliding constraints is that dead reckoning based on odometry becomes less accurate and power efficiency is reduced dramatically.

In general, a robot will have zero or more fixed standard wheels and zero or more steerable standard wheels. We can therefore identify the possible range of rank values for any robot: $0 \leq \text{rank}[C_1(\beta_s)] \leq 3$. Consider the case $\text{rank}[C_1(\beta_s)] = 0$. This is only possible if there are zero independent kinematic constraints in $C_1(\beta_s)$. In this case there are neither fixed nor steerable standard wheels attached to the robot frame: $N_f = N_s = 0$.

Consider the other extreme, $\text{rank}[C_1(\beta_s)] = 3$. This is the maximum possible rank since the kinematic constraints are specified along three degrees of freedom (i.e., the constraint matrix is three columns wide). Therefore, there cannot be more than three indepen-

dent constraints. In fact, when $\text{rank}[C_1(\beta_s)] = 3$, then the robot is completely constrained in all directions and is, therefore, degenerate since motion in the plane is totally impossible.

Now we are ready to formally define a robot's *degree of mobility* δ_m :

$$\delta_m = \dim N[C_1(\beta_s)] = 3 - \text{rank}[C_1(\beta_s)] \quad (3.40)$$

The dimensionality of the null space ($\dim N$) of matrix $C_1(\beta_s)$ is a measure of the number of degrees of freedom of the robot chassis that can be immediately manipulated through changes in wheel velocity. It is logical therefore that δ_m must range between 0 and 3.

Consider an ordinary differential-drive chassis. On such a robot there are two fixed standard wheels sharing a common horizontal axis. As discussed above, the second wheel adds no independent kinematic constraints to the system. Therefore, $\text{rank}[C_1(\beta_s)] = 1$ and $\delta_m = 2$. This fits with intuition: a differential drive robot can control both the rate of its change in orientation and its forward/reverse speed, *simply by manipulating wheel velocities*. In other words, its *ICR* is constrained to lie on the infinite line extending from its wheels' horizontal axes.

In contrast, consider a bicycle chassis. This configuration consists of one fixed standard wheel and one steerable standard wheel. In this case, each wheel contributes an independent sliding constraint to $C_1(\beta_s)$. Therefore, $\delta_m = 1$. Note that the bicycle has the same total number of nonomidirectional wheels as the differential-drive chassis, and indeed one of its wheels is steerable. Yet it has one less degree of mobility. Upon reflection this is appropriate. A bicycle only has control over its forward/reverse speed by direct manipulation of wheel velocities. Only by steering can the bicycle change its *ICR*.

As expected, based on equation (3.40) any robot consisting only of omnidirectional wheels such as Swedish or spherical wheels will have the maximum mobility, $\delta_m = 3$. Such a robot can directly manipulate all three degrees of freedom.

3.3.2 Degree of steerability

The degree of mobility defined above quantifies the degrees of controllable freedom based on changes to wheel velocity. Steering can also have an eventual impact on a robot chassis pose ξ , although the impact is indirect because after changing the angle of a steerable standard wheel, the robot must move for the change in steering angle to have impact on pose.

As with mobility, we care about the number of independently controllable steering parameters when defining the *degree of steerability* δ_s :

$$\delta_s = \text{rank}[C_{1s}(\beta_s)] \quad (3.41)$$

Recall that in the case of mobility, an increase in the rank of $C_1(\beta_s)$ implied more kinematic constraints and thus a less mobile system. In the case of steerability, an increase in the rank of $C_{1s}(\beta_s)$ implies more degrees of steering freedom and thus greater eventual maneuverability. Since $C_1(\beta_s)$ includes $C_{1s}(\beta_s)$, this means that a steered standard wheel can both decrease mobility and increase steerability: its particular orientation at any instant imposes a kinematic constraint, but its ability to change that orientation can lead to additional trajectories.

The range of δ_s can be specified: $0 \leq \delta_s \leq 2$. The case $\delta_s = 0$ implies that the robot has no steerable standard wheels, $N_s = 0$. The case $\delta_s = 1$ is most common when a robot configuration includes one or more steerable standard wheels.

For example, consider an ordinary automobile. In this case $N_f = 2$ and $N_s = 2$. But the fixed wheels share a common axle and so $\text{rank}[C_{1f}] = 1$. The fixed wheels and any one of the steerable wheels constrain the *ICR* to be a point along the line extending from the rear axle. Therefore, the second steerable wheel cannot impose any independent kinematic constraint and so $\text{rank}[C_{1s}(\beta_s)] = 1$. In this case $\delta_m = 1$ and $\delta_s = 1$.

The case $\delta_s = 2$ is only possible in robots with no fixed standard wheels: $N_f = 0$. Under these circumstances, it is possible to create a chassis with two separate steerable standard wheels, like a pseudobicycle (or the two-steer) in which both wheels are steerable. Then, orienting one wheel constrains the *ICR* to a line while the second wheel can constrain the *ICR* to any point along that line. Interestingly, this means that the $\delta_s = 2$ implies that the robot can place its *ICR* anywhere on the ground plane.

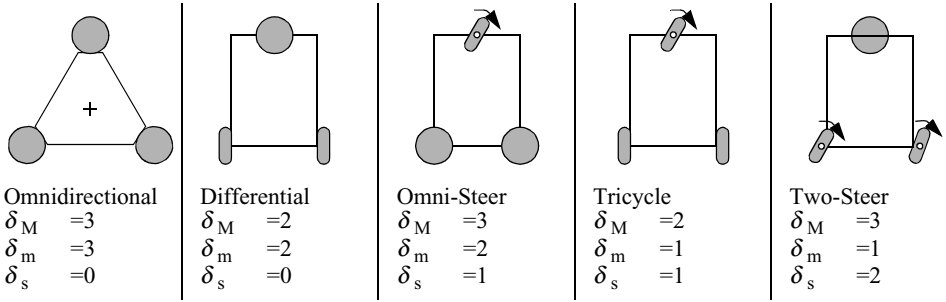
3.3.3 Robot maneuverability

The overall degrees of freedom that a robot can manipulate, called the *degree of maneuverability* δ_M , can be readily defined in terms of mobility and steerability:

$$\delta_M = \delta_m + \delta_s \quad (3.42)$$

Therefore maneuverability includes both the degrees of freedom that the robot manipulates directly through wheel velocity and the degrees of freedom that it indirectly manipulates by changing the steering configuration and moving. Based on the investigations of the previous sections, one can draw the basic types of wheel configurations. They are depicted in figure 3.14

Note that two robots with the same δ_M are not necessarily equivalent. For example, differential drive and tricycle geometries (figure 3.13) have equal maneuverability $\delta_M = 2$. In differential drive all maneuverability is the result of direct mobility because $\delta_m = 2$ and $\delta_s = 0$. In the case of a tricycle the maneuverability results from steering also: $\delta_m = 1$ and $\delta_s = 1$. Neither of these configurations allows the *ICR* to range anywhere on the plane. In both cases, the *ICR* must lie on a predefined line with respect to the robot refer-

**Figure 3.14**

The five basic types of three-wheel configurations. The spherical wheels can be replaced by castor or Swedish wheels without influencing maneuverability. More configurations with various numbers of wheels are found in chapter 2.

ence frame. In the case of differential drive, this line extends from the common axle of the two fixed standard wheels, with the differential wheel velocities setting the *ICR* point on this line. In a tricycle, this line extends from the shared common axle of the fixed wheels, with the steerable wheel setting the *ICR* point along this line.

More generally, for any robot with $\delta_M = 2$ the *ICR* is always constrained to lie on a line and for any robot with $\delta_M = 3$ the *ICR* can be set to any point on the plane.

One final example will demonstrate the use of the tools we have developed above. One common robot configuration for indoor mobile robotics research is the *synchro drive* configuration (figure 2.22). Such a robot has two motors and three wheels that are locked together. One motor provides power for spinning all three wheels while the second motor provides power for steering all three wheels. In a three-wheeled synchro drive robot $N_f = 0$ and $N_s = 3$. Therefore $\text{rank}[C_{1s}(\beta_s)]$ can be used to determine both δ_m and δ_s . The three wheels do not share a common axle, therefore two of the three contribute independent sliding constraints. The third must be dependent on these two constraints for motion to be possible. Therefore $\text{rank}[C_{1s}(\beta_s)] = 2$ and $\delta_m = 1$. This is intuitively correct. A synchro drive robot with the steering frozen manipulates only one degree of freedom, consisting of traveling back and forth on a straight line.

However an interesting complication occurs when considering δ_s . Based on equation (3.41) the robot should have $\delta_s = 2$. Indeed, for a three-wheel-steering robot with the geometric configuration of a synchro drive robot this would be correct. However, we have additional information: in a synchro drive configuration a single motor steers all three wheels using a belt drive. Therefore, although ideally, if the wheels were independently steerable, then the system would achieve $\delta_s = 2$, in the case of synchro drive the drive

system further constrains the kinematics such that in reality $\delta_s = 1$. Finally, we can compute maneuverability based on these values: $\delta_M = 2$ for a synchro drive robot.

This result implies that a synchro drive robot can only manipulate, in total, two degrees of freedom. In fact, if the reader reflects on the wheel configuration of a synchro drive robot it will become apparent that there is no way for the chassis orientation to change. Only the $x - y$ position of the chassis can be manipulated and so, indeed, a synchro drive robot has only two degrees of freedom, in agreement with our mathematical conclusion.

3.4 Mobile Robot Workspace

For a robot, maneuverability is equivalent to its control degrees of freedom. But the robot is situated in some environment, and the next question is to situate our analysis in the environment. We care about the ways in which the robot can use its control degrees of freedom to position itself in the environment. For instance, consider the Ackerman vehicle, or automobile. The total number of control degrees of freedom for such a vehicle is $\delta_M = 2$, one for steering and the second for actuation of the drive wheels. But what is the total degrees of freedom of the vehicle in its environment? In fact it is three: the car can position itself on the plane at any x, y point and with any angle θ .

Thus identifying a robot's space of possible configurations is important because surprisingly it can exceed δ_M . In addition to *workspace*, we care about how the robot is able to move between various configurations: what are the types of paths that it can follow and, furthermore, what are its possible trajectories through this configuration space? In the remainder of this discussion, we move away from inner kinematic details such as wheels and focus instead on the robot chassis pose and the chassis degrees of freedom. With this in mind, let us place the robot in the context of its workspace now.

3.4.1 Degrees of freedom

In defining the workspace of a robot, it is useful to first examine its *admissible velocity space*. Given the kinematic constraints of the robot, its velocity space describes the independent components of robot motion that the robot can control. For example, the velocity space of a unicycle can be represented with two axes, one representing the instantaneous forward speed of the unicycle and the second representing the instantaneous change in orientation, $\dot{\theta}$, of the unicycle.

The number of dimensions in the velocity space of a robot is the number of independently achievable velocities. This is also called the *differentiable degrees of freedom (DDOF)*. A robot's *DDOF* is *always* equal to its degree of mobility δ_m . For example, a bicycle has the following degree of maneuverability: $\delta_M = \delta_m + \delta_s = 1 + 1 = 2$. The *DDOF* of a bicycle is indeed 1.

In contrast to a bicycle, consider an *omnibot*, a robot with three Swedish wheels. We know that in this case there are zero standard wheels and therefore $\delta_M = \delta_m + \delta_s = 3 + 0 = 3$. So, the omnibot has three differential degrees of freedom. This is appropriate, given that because such a robot has no kinematic motion constraints, it is able to independently set all three pose variables: $\dot{x}, \dot{y}, \dot{\theta}$.

Given the difference in *DDOF* between a bicycle and an omnibot, consider the overall degrees of freedom in the workspace of each configuration. The omnibot can achieve any pose (x, y, θ) in its environment and can do so by directly achieving the goal positions of all three axes simultaneously because $DDOF = 3$. Clearly, it has a workspace with $DOF = 3$.

Can a bicycle achieve any pose (x, y, θ) in its environment? It can do so, but achieving some goal points may require more time and energy than an equivalent omnibot. For example, if a bicycle configuration must move laterally 1 m, the simplest successful maneuver would involve either a spiral or a back-and-forth motion similar to *parallel parking* of automobiles. Nevertheless, a bicycle can achieve any (x, y, θ) and therefore the workspace of a bicycle has $DOF=3$ as well.

Clearly, there is an inequality relation at work: $DDOF \leq \delta_M \leq DOF$. Although the dimensionality of a robot's workspace is an important attribute, it is clear from the example above that the particular paths available to a robot matter as well. Just as workspace *DOF* governs the robot's ability to achieve various poses, so the robot's *DDOF* governs its ability to achieve various paths.

3.4.2 Holonomic robots

In the robotics community, when describing the path space of a mobile robot, often the concept of holonomy is used. The term *holonomic* has broad applicability to several mathematical areas, including differential equations, functions and constraint expressions. In mobile robotics, the term refers specifically to the kinematic constraints of the robot chassis. A *holonomic robot* is a robot that has zero nonholonomic kinematic constraints. Conversely, a *nonholonomic robot* is a robot with one or more nonholonomic kinematic constraints.

A *holonomic kinematic constraint* can be expressed as an explicit function of position variables only. For example, in the case of a mobile robot with a single fixed standard wheel, a holonomic kinematic constraint would be expressible using $\alpha_1, \beta_1, l_1, r_1, \varphi_1, x, y, \theta$ only. Such a constraint may not use derivatives of these values, such as $\dot{\phi}$ or $\dot{\xi}$. A *nonholonomic kinematic constraint* requires a differential relationship, such as the derivative of a position variable. Furthermore, it cannot be integrated to provide a constraint in terms of the position variables only. Because of this latter point of view, nonholonomic systems are often called *nonintegrable* systems.

Consider the fixed standard wheel sliding constraint:

$$\begin{bmatrix} \cos(\alpha + \beta) & \sin(\alpha + \beta) & l \sin \beta \end{bmatrix} R(\theta) \dot{\xi}_I = 0 \quad (3.43)$$

This constraint must use robot motion $\dot{\xi}$ rather than pose ξ because the point is to constrain robot motion perpendicular to the wheel plane to be zero. The constraint is nonintegrable, depending explicitly on robot motion. Therefore, the sliding constraint is a nonholonomic constraint. Consider a bicycle configuration, with one fixed standard wheel and one steerable standard wheel. Because the fixed wheel sliding constraint will be in force for such a robot, we can conclude that the bicycle is a nonholonomic robot.

But suppose that one locks the bicycle steering system, so that it becomes two fixed standard wheels with separate but parallel axes. We know that $\delta_M = 1$ for such a configuration. Is it nonholonomic? Although it may not appear so because of the sliding and rolling constraints, the locked bicycle is actually holonomic. Consider the workspace of this locked bicycle. It consists of a single infinite line along which the bicycle can move (assuming the steering was frozen straight ahead). For formulaic simplicity, assume that this infinite line is aligned with X_I in the global reference frame and that $\{\beta_{1,2} = \pi/2, \alpha_1 = 0, \alpha_2 = \pi\}$. In this case the sliding constraints of both wheels can be replaced with an equally complete set of constraints on the robot pose: $\{y = 0, \theta = 0\}$. This eliminates two nonholonomic constraints, corresponding to the sliding constraints of the two wheels.

The only remaining nonholonomic kinematic constraints are the rolling constraints for each wheel:

$$\begin{bmatrix} -\sin(\alpha + \beta) & \cos(\alpha + \beta) & l \cos \beta \end{bmatrix} R(\theta) \dot{\xi}_I + r \dot{\phi} = 0 \quad (3.44)$$

This constraint is required for each wheel to relate the speed of wheel spin to the speed of motion projected along the wheel plane. But in the case of our locked bicycle, given the initial rotational position of a wheel at the origin, ϕ_o , we can replace this constraint with one that directly relates position on the line, x , with wheel rotation angle, ϕ : $\phi = (x/r) + \phi_o$.

The locked bicycle is an example of the first type of holonomic robot – where constraints do exist but are all holonomic kinematic constraints. This is the case for all holonomic robots with $\delta_M < 3$. The second type of holonomic robot exists when there are no kinematic constraints, that is, $N_f = 0$ and $N_s = 0$. Since there are no kinematic constraints, there are also no nonholonomic kinematic constraints and so such a robot is always holonomic. This is the case for all holonomic robots with $\delta_M = 3$.

An alternative way to describe a holonomic robot is based on the relationship between the differential degrees of freedom of a robot and the degrees of freedom of its workspace: *a robot is holonomic if and only if $DDOF = DOF$* . Intuitively, this is because it is only through nonholonomic constraints (imposed by steerable or fixed standard wheels) that a robot can achieve a workspace with degrees of freedom exceeding its differential degrees of freedom, $DOF > DDOF$. Examples include differential drive and bicycle/tricycle configurations.

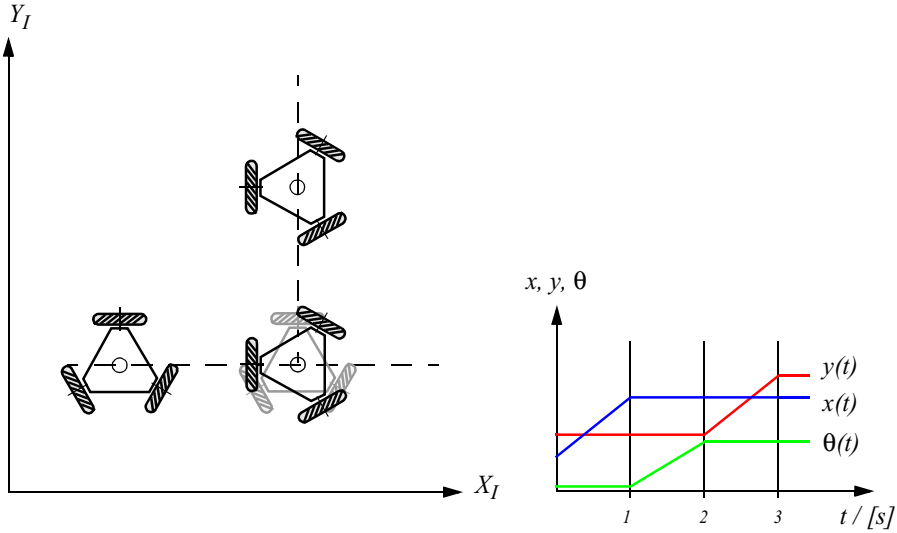
In mobile robotics, useful chassis generally must achieve poses in a workspace with dimensionality 3, so in general we require $DOF = 3$ for the chassis. But the “holonomic” abilities to maneuver around obstacles without affecting orientation and to track at a target while following an arbitrary path are important additional considerations. For these reasons, the particular form of holonomy most relevant to mobile robotics is that of $DDOF = DOF = 3$. We define this class of robot configurations as omnidirectional: an *omnidirectional robot* is a holonomic robot with $DDOF = 3$.

3.4.3 Path and trajectory considerations

In mobile robotics, we care not only about the robot’s ability to reach the required final configurations but also about *how* it gets there. Consider the issue of a robot’s ability to follow paths: in the best case, a robot should be able to trace any path through its workspace of poses. Clearly, any omnidirectional robot can do this because it is holonomic in a three-dimensional workspace. Unfortunately, omnidirectional robots must use unconstrained wheels, limiting the choice of wheels to Swedish wheels, castor wheels, and spherical wheels. These wheels have not yet been incorporated into designs allowing far larger amounts of ground clearance and suspensions. Although powerful from a path space point of view, they are thus much less common than fixed and steerable standard wheels, mainly because their design and fabrication are somewhat complex and expensive.

Additionally, nonholonomic constraints might drastically improve stability of movements. Consider an omnidirectional vehicle driving at high speed on a curve with constant diameter. During such a movement the vehicle will be exposed to a non-negligible centripetal force. This lateral force pushing the vehicle out of the curve has to be counteracted by the motor torque of the omnidirectional wheels. In case of motor or control failure, the vehicle will be thrown out of the curve. However, for a car-like robot with kinematic constraints, the lateral forces are passively counteracted through the sliding constraints, mitigating the demands on motor torque.

But recall an earlier example of high maneuverability using standard wheels: the bicycle on which both wheels are steerable, often called the *two-steer*. This vehicle achieves a degree of steerability of 2, resulting in a high degree of maneuverability: $\delta_M = \delta_m + \delta_s = 1 + 2 = 3$. Interestingly, this configuration is not holonomic, yet has a high degree of maneuverability in a workspace with $DOF = 3$.

**Figure 3.15**

Example of robot trajectory with omnidirectional robot: move for 1 second with constant speed of 1 m/s along axis X_I ; change orientation counterclockwise 90 degree, in 1 second; move for 1 second with constant speed of 1 m/s along axis Y_I .

The maneuverability result, $\delta_M = 3$, means that the two-steer can select any *ICR* by appropriately steering its two wheels. So, how does this compare to an omnidirectional robot? The ability to manipulate its *ICR* in the plane means that the two-steer can follow *any* path in its workspace. More generally, any robot with $\delta_M = 3$ can follow any path in its workspace from its initial pose to its final pose. An omnidirectional robot can also follow any path in its workspace and, not surprisingly, since $\delta_m = 3$ in an omnidirectional robot, then it must follow that $\delta_M = 3$.

But there is still a difference between a degree of freedom granted by steering versus by direct control of wheel velocity. This difference is clear in the context of *trajectories* rather than paths. A trajectory is like a path, except that it occupies an additional dimension: time. Therefore, for an omnidirectional robot on the ground plane a path generally denotes a trace through a 3D space of pose; for the same robot a trajectory denotes a trace through the 4D space of pose plus time.

For example, consider a goal trajectory in which the robot moves along axis X_I at a constant speed of 1 m/s for 1 second, then changes orientation counterclockwise 90 degrees also in 1 second, then moves parallel to axis Y_I for 1 final second. The desired 3-second trajectory is shown in figure 3.15, using plots of x , y and θ in relation to time.

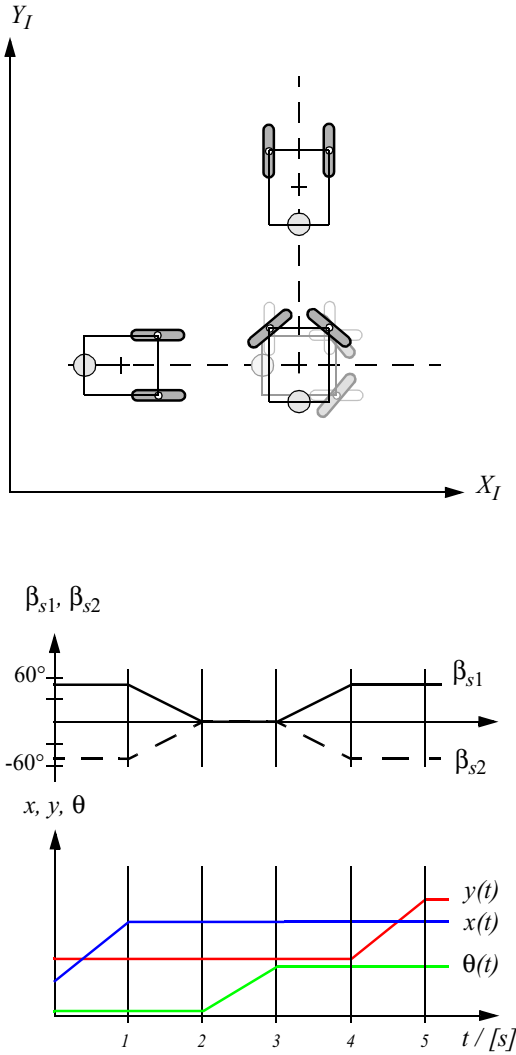


Figure 3.16

Example of robot trajectory similar to figure 3.15 with two steered wheels: move for 1 second with constant speed of 1 m/s along axis X_I ; rotate steered wheels $-50 / 50$ degree respectively; change orientation counterclockwise 90 degree in 1 second; rotate steered wheels $50 / -50$ degree respectively; move for 1 second with constant speed of 1 m/s along axis Y_I .

Can the omnidirectional robot accomplish this trajectory? We assume that the robot can achieve some arbitrary, finite velocity at each wheel. For simplicity, we further assume that acceleration is infinite; that is, it takes zero time to reach any desired velocity. Under these assumptions, the omnidirectional robot can indeed follow the trajectory of figure 3.15. The transition between the motion of second 1 and second 2, for example, involves only changes to the wheel velocities.

Because the two-steer has $\delta_M = 3$, it must be able to follow the path that would result from projecting this trajectory into timeless workspace. However, it cannot follow this 4D trajectory. Even if steering velocity is finite and arbitrary, although the two-steer would be able to change steering speed instantly, it would have to wait for the angle of the steerable wheels to change to the desired position before initiating a change in the robot chassis orientation. In short, the two-steer requires changes to internal degrees of freedom and because these changes take time, arbitrary trajectories are not attainable. Figure 3.16 depicts the most similar trajectory that a two-steer can achieve. In contrast to the desired three phases of motion, this trajectory has five phases.

3.5 Beyond Basic Kinematics

The above discussion of mobile robot kinematics is only an introduction to a far richer topic. When speed and force are also considered, as is particularly necessary in the case of high-speed mobile robots, dynamic constraints must be expressed in addition to kinematic constraints. Furthermore, many mobile robots such as tank-type chassis and four-wheel slip/skid systems violate the kinematic models above. When analyzing such systems, it is often necessary to explicitly model the dynamics of viscous friction between the robot and the ground plane.

More significantly, the kinematic analysis of a mobile robot system provides results concerning the theoretical workspace of that mobile robot. However to effectively move in this workspace a mobile robot must have appropriate actuation of its degrees of freedom. This problem, called motorization, requires further analysis of the forces that must be actively supplied to realize the kinematic range of motion available to the robot.

In addition to motorization, there is the question of controllability: under what conditions can a mobile robot travel from the initial pose to the goal pose in bounded time? Answering this question requires knowledge – both knowledge of the robot kinematics and knowledge of the control systems that can be used to actuate the mobile robot. Mobile robot control is therefore a return to the practical question of designing a real-world control algorithm that can drive the robot from pose to pose using the trajectories demanded for the application.

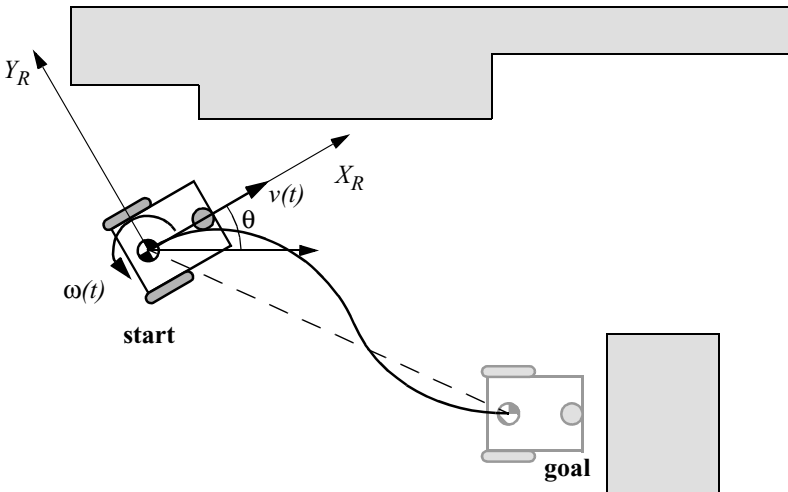


Figure 3.17
Typical situation for feedback control of a mobile robot

3.6 Motion Control (Kinematic Control)

As seen above, motion control might not be an easy task for nonholonomic systems. However, it has been studied by various research groups, for example, [8, 39, 52, 53, 137] and some adequate solutions for motion control of a mobile robot system are available.

3.6.1 Open loop control (trajectory-following)

The objective of a kinematic controller is to follow a trajectory described by its position or velocity profile as a function of time. This is often done by dividing the trajectory (path) in motion segments of clearly defined shape, for example, straight *lines* and segments of a *circle*. The control problem is thus to precompute a smooth trajectory based on line and circle segments which drives the robot from the initial position to the final position (figure 3.18). This approach can be regarded as open-loop motion control, because the measured robot position is not fed back for velocity or position control. It has several disadvantages:

- It is not at all an easy task to precompute a feasible trajectory if all limitations and constraints of the robot's velocities and accelerations have to be considered.
- The robot will not automatically adapt or correct the trajectory if dynamic changes of the environment occur.

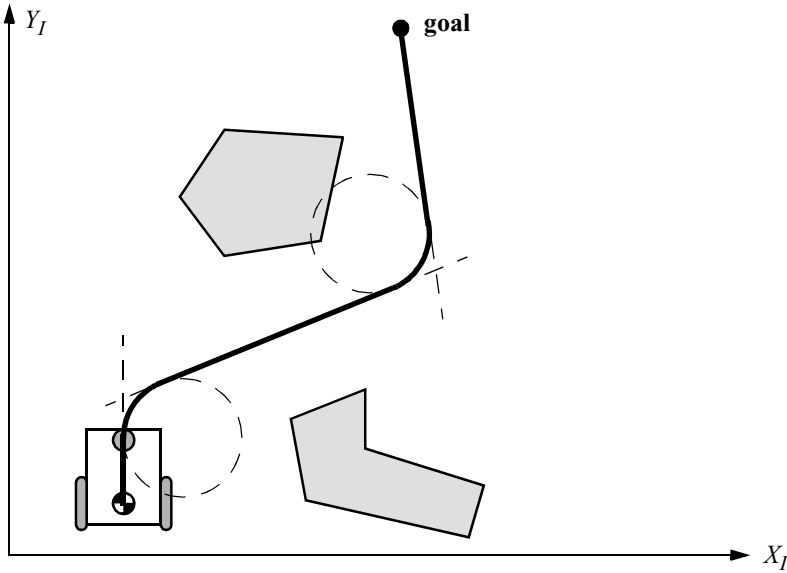


Figure 3.18

Open-loop control of a mobile robot based on straight lines and circular trajectory segments.

- The resulting trajectories are usually not smooth, because the transitions from one trajectory segment to another are, for most of the commonly used segments (e.g., lines and part of circles), not smooth. This means there is a discontinuity in the robot's acceleration.

3.6.2 Feedback control

A more appropriate approach in motion control of a mobile robot is to use a real-state feedback controller. With such a controller the robot's path-planning task is reduced to setting intermediate positions (subgoals) lying on the requested path. One useful solution for a stabilizing feedback control of differential-drive mobile robots is presented in section 3.6.2.1. It is very similar to the controllers presented in [39, 100]. Others can be found in [8, 52, 53, 137].

3.6.2.1 Problem statement

Consider the situation shown in figure 3.17, with an arbitrary position and orientation of the robot and a predefined goal position and orientation. The actual pose error vector given in the robot reference frame $\{X_R, Y_R, \theta\}$ is $e = {}^R[x, y, \theta]^T$ with x, y , and θ being the goal coordinates of the robot.

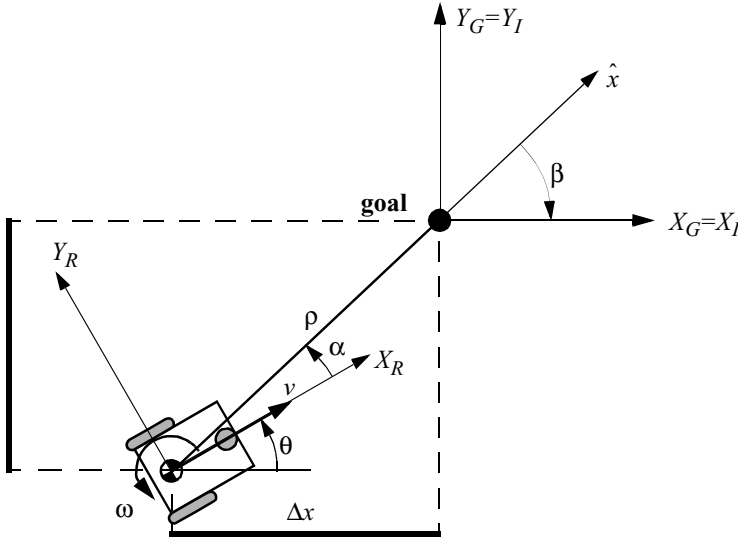


Figure 3.19
Robot kinematics and its frames of interests.

The task of the controller layout is to find a control matrix K , if it exists

$$K = \begin{bmatrix} k_{11} & k_{12} & k_{13} \\ k_{21} & k_{22} & k_{23} \end{bmatrix} \quad \text{with } k_{ij} = k(t, e) \quad (3.45)$$

such that the control of $v(t)$ and $\omega(t)$

$$\begin{bmatrix} v(t) \\ \omega(t) \end{bmatrix} = K \cdot e = K \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}^R \quad (3.46)$$

drives the error e toward zero.²

$$\lim_{t \rightarrow \infty} e(t) = 0 \quad (3.47)$$

2. Remember that $v(t)$ is always heading in the X_R direction of the robot's reference frame due to the nonholonomic constraint.

3.6.2.2 Kinematic model

We assume, without loss of generality, that the goal is at the origin of the inertial frame (figure 3.19). In the following the position vector $[x, y, \theta]^T$ is always represented in the inertial frame.

The kinematics of a differential-drive mobile robot described in the inertial frame $\{X_I, Y_I, \theta\}$ is given by

$${}^I \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (3.48)$$

where $\dot{x}$ and $\dot{y}$ are the linear velocities in the direction of the X_I and Y_I of the inertial frame.

Let α denote the angle between the x_R axis of the robot's reference frame and the vector $\hat{x}$ connecting the center of the axle of the wheels with the final position. If $\alpha \in I_1$, where

$$I_1 = \left(-\frac{\pi}{2}, \frac{\pi}{2} \right] \quad (3.49)$$

then consider the coordinate transformation into polar coordinates with its origin at the goal position.

$$\rho = \sqrt{\Delta x^2 + \Delta y^2} \quad (3.50)$$

$$\alpha = -\theta + \text{atan2}(\Delta y, \Delta x) \quad (3.51)$$

$$\beta = -\theta - \alpha \quad (3.52)$$

This yields a system description, in the new polar coordinates, using a matrix equation

$$\begin{bmatrix} \dot{\rho} \\ \dot{\alpha} \\ \dot{\beta} \end{bmatrix} = \begin{bmatrix} -\cos \alpha & 0 \\ \frac{\sin \alpha}{\rho} & -1 \\ -\frac{\sin \alpha}{\rho} & 0 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (3.53)$$

where ρ is the distance between the center of the robot's wheel axle and the goal position,

θ denotes the angle between the X_R axis of the robot reference frame, and the X_I axis associated with the final position v and ω are the tangent and the angular velocity respectively.

On the other hand, if $\alpha \in I_2$, where

$$I_2 = (-\pi, -\pi/2] \cup (\pi/2, \pi] \quad (3.54)$$

redefining the forward direction of the robot by setting $v = -v$, we obtain a system described by a matrix equation of the form

$$\begin{bmatrix} \dot{\rho} \\ \dot{\alpha} \\ \dot{\beta} \end{bmatrix} = \begin{bmatrix} \cos \alpha & 0 \\ -\frac{\sin \alpha}{\rho} & 1 \\ \frac{\sin \alpha}{\rho} & 0 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (3.55)$$

3.6.2.3 Remarks on the kinematic model in polar coordinates [eq. (3.53) and (3.55)]

- The coordinate transformation is not defined at $x = y = 0$; as in such a point the determinant of the Jacobian matrix of the transformation is not defined, that is unbounded.
- For $\alpha \in I_1$ the forward direction of the robot points toward the goal, for $\alpha \in I_2$ it is the reverse direction.
- By properly defining the forward direction of the robot at its initial configuration, it is always possible to have $\alpha \in I_1$ at $t = 0$. However, this does not mean that α remains in I_1 for all time t . Hence, to avoid that the robot changes direction during approaching the goal, it is necessary to determine, if possible, the controller in such a way that $\alpha \in I_1$ for all t , whenever $\alpha(0) \in I_1$. The same applies for the reverse direction (see stability issues below).

3.6.2.4 The control law

The control signals v and ω must now be designed to drive the robot from its actual configuration, say $(\rho_0, \alpha_0, \beta_0)$, to the goal position. It is obvious that equation (3.53) presents a discontinuity at $\rho = 0$; thus the theorem of Brockett does not obstruct smooth stabilizability.

If we consider now the linear control law

$$v = k_\rho \rho \quad (3.56)$$

$$\omega = k_\alpha \alpha + k_\beta \beta \quad (3.57)$$

we get with equation (3.53) a closed-loop system described by

$$\begin{bmatrix} \dot{\rho} \\ \dot{\alpha} \\ \dot{\beta} \end{bmatrix} = \begin{bmatrix} -k_\rho \rho \cos \alpha \\ k_\rho \sin \alpha - k_\alpha \alpha - k_\beta \beta \\ -k_\rho \sin \alpha \end{bmatrix} \quad (3.58)$$

The system does not have any singularity at $\rho = 0$ and has a unique equilibrium point at $(\rho, \alpha, \beta) = (0, 0, 0)$. Thus it will drive the robot to this point, which is the goal position.

- In the Cartesian coordinate system the control law [equation (3.57)] leads to equations which are not defined at $x = y = 0$.
- Be aware of the fact that the angles α and β have always to be expressed in the range $(-\pi, \pi)$.
- Observe that the control signal v has always a constant sign, that is, it is positive whenever $\alpha(0) \in I_1$ and it is always negative otherwise. This implies that the robot performs its parking maneuver always in a single direction and without reversing its motion.

In figure 3.20 you find the resulting paths when the robot is initially on a circle in the xy plane. All movements have smooth trajectories toward the goal in the center. The control parameters for this simulation were set to

$$k = (k_\rho, k_\alpha, k_\beta) = (3, 8, -1.5). \quad (3.59)$$

3.6.2.5 Local stability issue

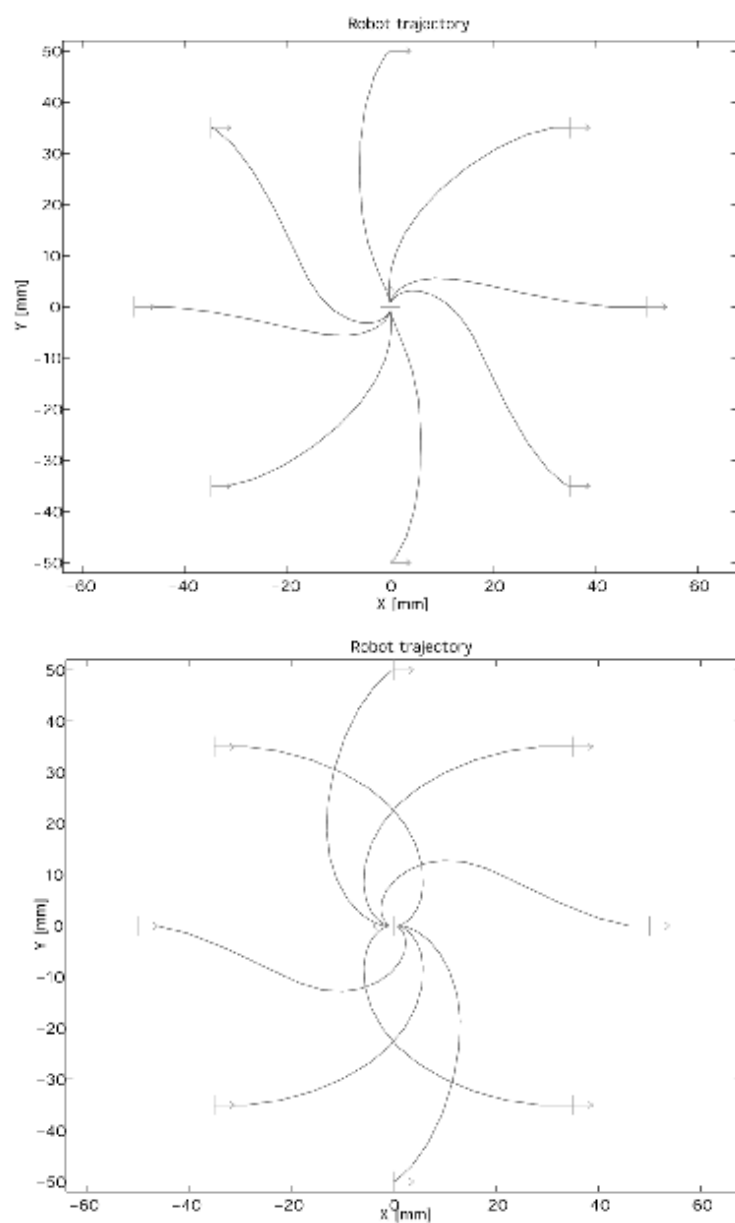
It can further be shown, that the closed-loop control system [equation (3.58)] is locally exponentially stable if

$$k_\rho > 0 ; \quad k_\beta < 0 ; \quad k_\alpha - k_\rho > 0 \quad (3.60)$$

Proof:

Linearized around the equilibrium $(\cos x = 1, \sin x = x)$ position, equation (3.58) can be written as

$$\begin{bmatrix} \dot{\rho} \\ \dot{\alpha} \\ \dot{\beta} \end{bmatrix} = \begin{bmatrix} -k_\rho & 0 & 0 \\ 0 & -(k_\alpha - k_\rho) & -k_\beta \\ 0 & -k_\rho & 0 \end{bmatrix} \begin{bmatrix} \rho \\ \alpha \\ \beta \end{bmatrix}, \quad (3.61)$$

**Figure 3.20**

Resulting paths when the robot is initially on the unit circle in the x,y plane.

hence it is locally exponentially stable if the eigenvalues of the matrix

$$A = \begin{bmatrix} -k_p & 0 & 0 \\ 0 & -(k_\alpha - k_p) & -k_\beta \\ 0 & -k_p & 0 \end{bmatrix} \quad (3.62)$$

all have a negative real part. The characteristic polynomial of the matrix A is

$$(\lambda + k_p)(\lambda^2 + \lambda(k_\alpha - k_p) - k_p k_\beta) \quad (3.63)$$

and all roots have negative real part if

$$k_p > 0 ; \quad -k_\beta > 0 ; \quad k_\alpha - k_p > 0 \quad (3.64)$$

which proves the claim.

For robust position control, it might be advisable to apply the strong stability condition, which ensures that the robot does not change direction during its approach to the goal:

$$k_p > 0 ; \quad k_\beta < 0 ; \quad k_\alpha + \frac{5}{3}k_\beta - \frac{2}{\pi}k_p > 0 \quad (3.65)$$

This implies that $\alpha \in I_1$ for all t , whenever $\alpha(0) \in I_1$ and $\alpha \in I_2$ for all t , whenever $\alpha(0) \in I_2$ respectively. This strong stability condition has also been verified in applications.

UNIT – 3

ROBOT PERCEPTION

Sensors for mobile robots – computer vision for robots – image processing for robotics – place recognition – range data.

Sensors for Mobile Robots

- A sensor is a device that detects and responds to some type of input from the physical environment. Sophisticated sensors can be used to acquire information about the robot's environment or even to directly measure a robot's global position.

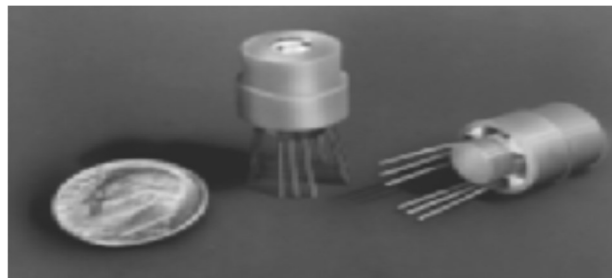
Sensor Classification

- We classify sensors using two important functional axes: proprioceptive/exteroceptive and passive/active.
- ✓ **Proprioceptive sensors** measure values internal to the system.
 - Examples: Motor Speed, Wheel Load, Robot Arm Joint Angles, Battery Voltage.
- ✓ **Exteroceptive sensors** acquire information from the robot's environment.
 - Examples: Distance Measurements, Light Intensity, Sound Amplitude.
- ✓ **Passive sensors** measure ambient environmental energy entering the sensor.
 - Examples: Microphones, And CCD Or CMOS Cameras.
- ✓ **Active sensors** emit energy into the environment, then measure the environmental reaction.
 - Examples: Wheel Quadrature Encoders, Ultrasonic Sensors, And Laser Rangefinders.

Sensors used for mobility

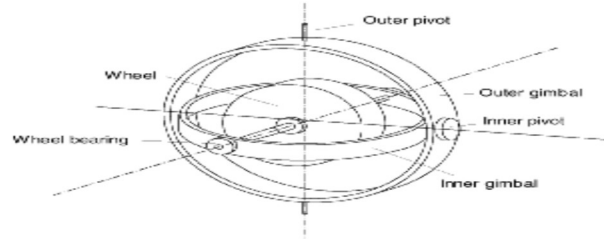
1. Inertial measurement units (IMUs)

- It typically combines multiple accelerometers and gyroscopes. They can also include magnetometers and barometers.
- Instantaneous pose (position and orientation) of the robot, velocity (linear, angular) acceleration (linear, angular) and other parameters are obtained through the IMU in 3D space.



2. GPS

- It provides latitude, longitude and altitude information. Over the years, GPS accuracy has increased significantly and highly accurate modes, such as RTK, also exist.
- GPS-denied areas, such as indoor areas, tunnels and so forth, and slow update rates remain a GPS's top limitations.
- But they are important sensors for outdoor mobile robots and provide an accurate periodic reference.



3. Laser Sensors

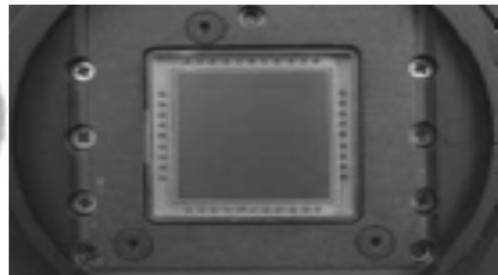
- Depending on whether they are indoor or outdoor robots and the speed at which the robot moves, laser sensors can vary significantly in price, performance, robustness, range and weight.

4. Encoders

- It counts the precise number of rotations of the robot wheels, thereby estimating how far the robot has travelled.
- The terms odometry or dead-reckoning are used for distance calculation with wheel encoders.
- They suffer from long-term drifts and hence need to be combined with other sensors.

5. Vision sensors

- It such as cameras, both 2D and 3D, as well as depth cameras, play a very critical role in AMRs.
- Computer vision and deep learning on the sensor data can aid object detection and avoidance, obstacle recognition and obstacle tracking.
- 3D cameras, depth and stereo vision cameras provide pose, i.e., position and orientation, of an object in 3D space.

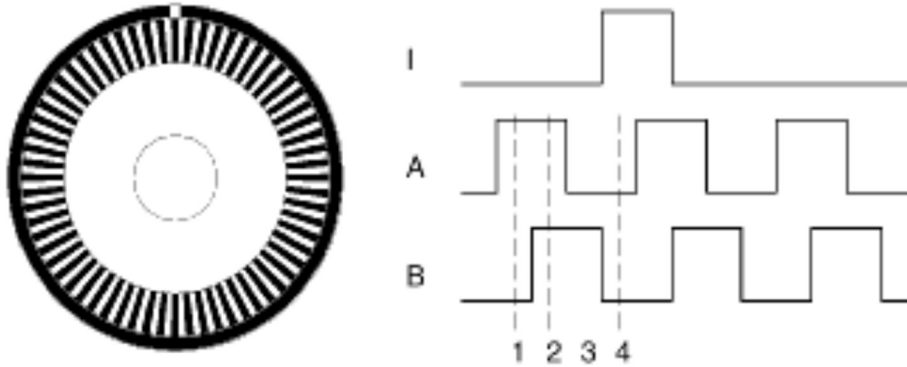


6. Ultrasonic Sensors

- If there is an object in the range of an ultrasonic sensor pulse, part or all of the pulse will be reflected back to the transmitter as an echo and can be detected through the receiver.
- By measuring the difference in time between the transmitted pulse and the received echo, it is possible to determine object's range. Sonars are impacted by multipath reflections.

7. Pulsed and millimeter wave radars

- It detects objects at long range and provide velocity, angle and bearing parameters typically measured to the centroid of the object.
- They work in all weather conditions while most other sensors fail in complex environments, such as rain, fog and lighting variations.



- Robust and accurate localization schemes combine data received from IMUs, wheel encoders, GPS, laser, radar, ultrasonic and vision software algorithms to implement SLAM techniques. Depending on the application and specification of navigation and object avoidance, the fusion can be limited to few sensors or all sensors.

Selection and placement of sensors

- The placement of sensors inside the robot also requires a very careful design exercise. Sensors in IMUs are extremely sensitive to external forces, such as vibrations, stray magnetic fields, lighting conditions and more. Accurate static translation and rotation offsets need to be calculated using physical methods and calibration techniques. Some of the sensors require antennas and, once again, selection of the right antenna and its placement are critical.

Computer Vision for Robots

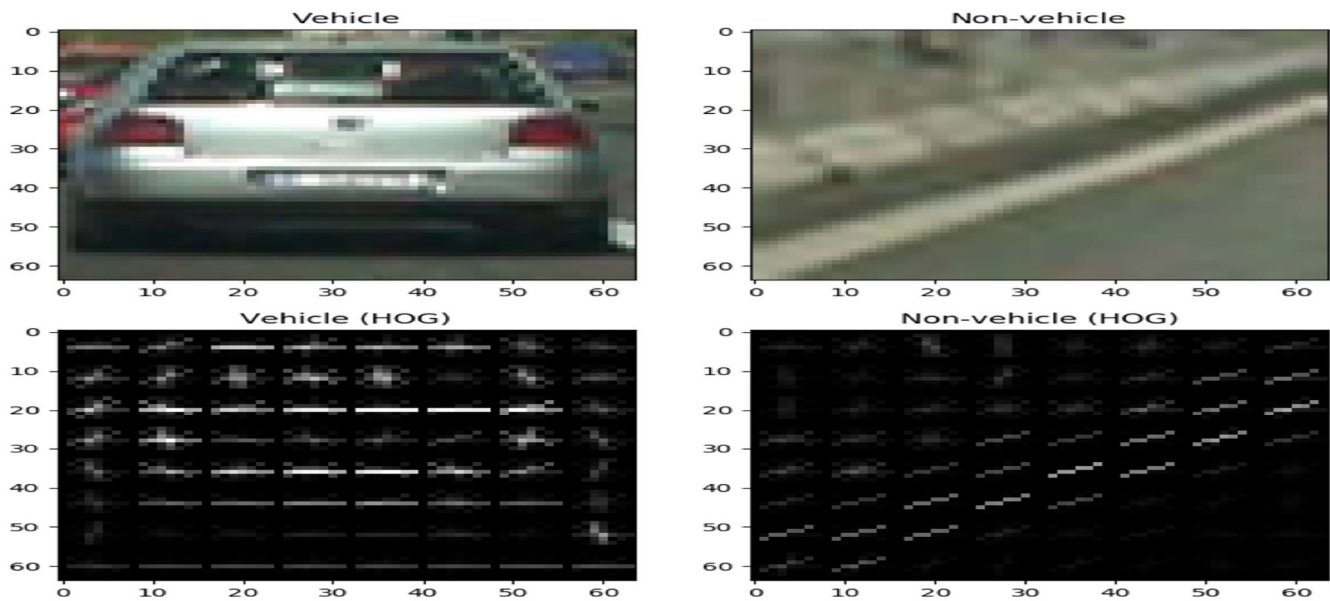
- A Computer vision system uses a sensor in the robot for viewing and recognizing an object with the help of a computer. Computer vision is used in a variety of industrial processes, such as material inspection, object recognition, pattern recognition, electronic component analysis, along with the recognition of signatures, optical characters, and currency.

1. Vehicle detection

- Vehicle detection processes on roads are used for vehicle tracking, counts, the average speed of each individual vehicle, traffic analysis, and vehicle categorizing objectives and may be implemented under different environmental changes.

HOG features extraction for learning:

- HOG or Histogram of Oriented Gradients is a technique that maps all the locations of the gradients (change in color or luminosity) in a picture, as well as their orientation.



2. Spatial binning:

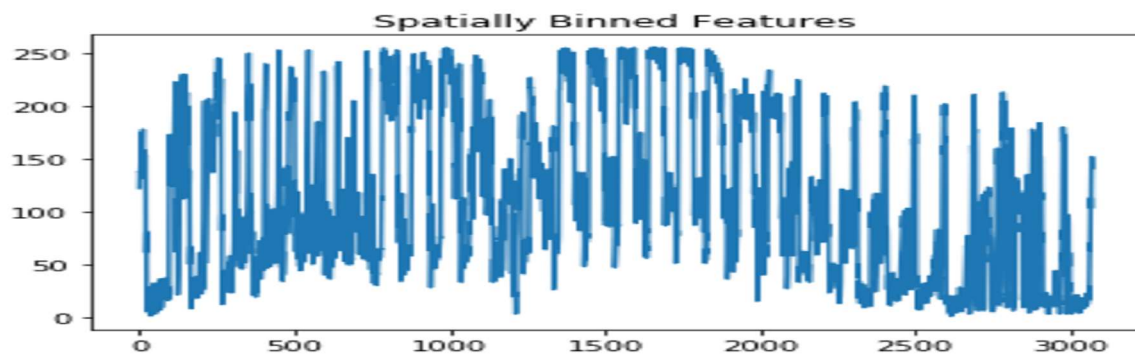
- Spatial binning discretizes the location values into a small number of groups associated with geographical areas. This allows us to filter out some of the noise and only retain the most important features of the car while making the algorithm run faster as the resulting images are smaller.



Original

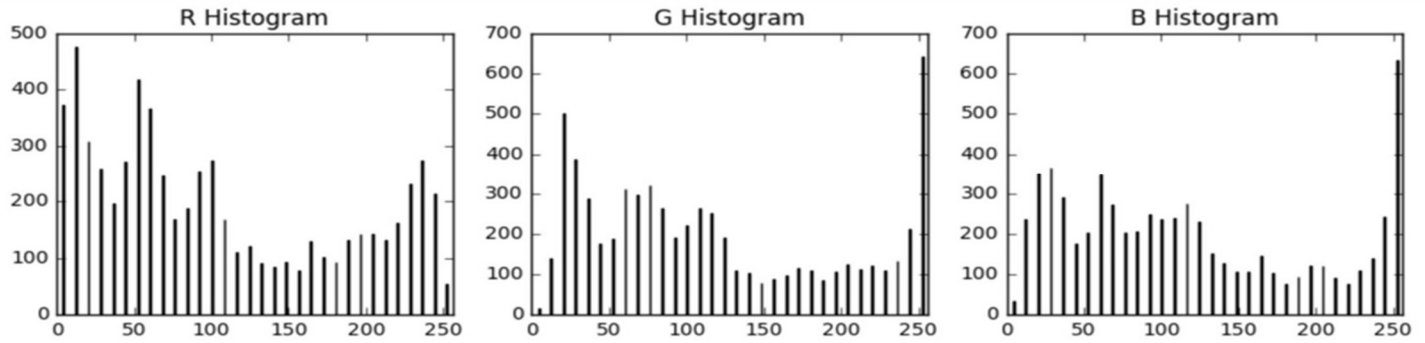
64 x 64 pix

32 x 32 pix



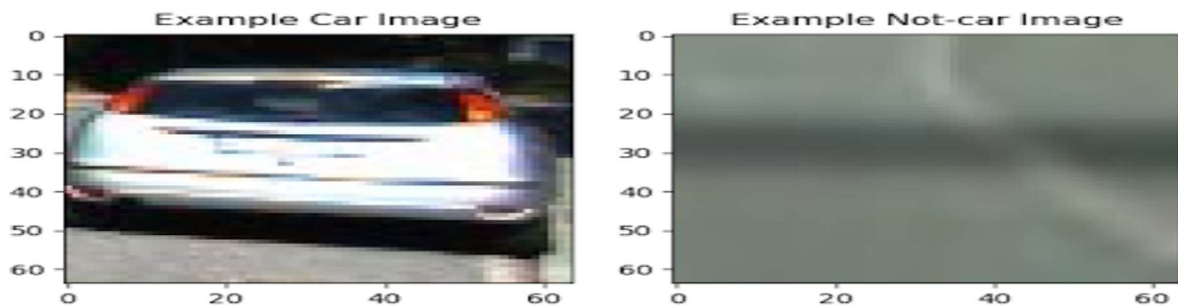
3. Color channel histograms:

- This will give us pixel intensity for each image, allowing us to use the common car colors as a feature for recognition.



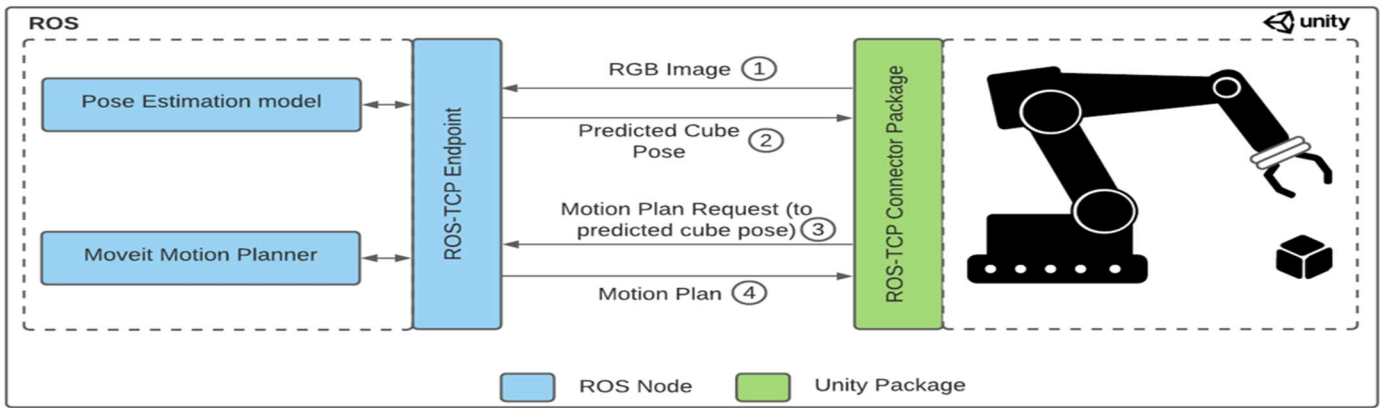
4. Training a classifier:

- We start by reading in all the vehicle and non-vehicle images. Here is an example of one of each of the vehicle and non-vehicle classes:



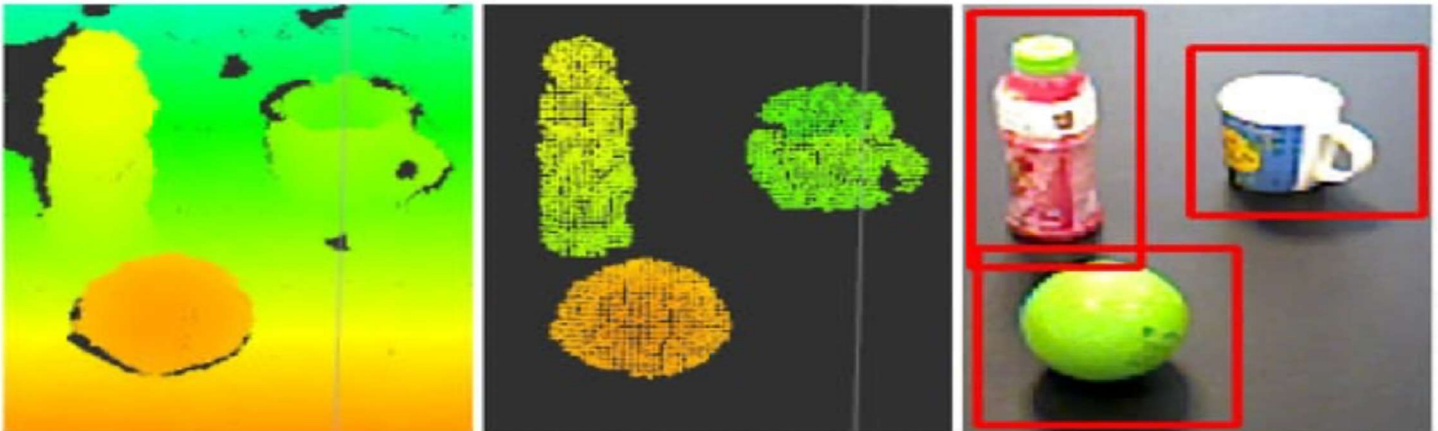
5. Robotic arms pick and place of objects:

- An intelligent robot is expected to not only be able to perceive the environment, but also interact with the environment.
- Among all these abilities, object grasping is fundamental and significant in that it will bring enormous productivity to society.
- For example, an industrial robot can accomplish the pick-and-bin task which is laborious for human laborers, and a domestic robot is able to provide assistance to disabled people in their daily grasping tasks.



6. Object Segmentation:

- In digital image processing and computer vision, image segmentation is the process of partitioning a digital image into multiple segments.
- The goal of segmentation is to simplify and/or change the representation of an image into something that is more meaningful and easier to analyze.
- Image segmentation is typically used to locate objects and boundaries (lines, curves, etc.) in images.



7. Object Classification:

- Object detection is a computer technology related to computer vision and image processing that deals with detecting instances of semantic objects of a certain class (such as humans, buildings, or cars) in digital images and videos.
- Object classification is performed using CNN. CNN's are able to learn how to extract features from images effectively for image classification.

Computer Vision is Playing the Following Role in Various Fields:

- ✓ Face Recognition
- ✓ Video Surveillance
- ✓ Object Detection
- ✓ Object Recognition

- ✓ Medical Imaging Analysis
- ✓ Localization and Mapping
- ✓ Augmented Reality/Virtual Reality
- ✓ Human Expressions & Emotional Analysis
- ✓ Transforming the paperwork into digital data

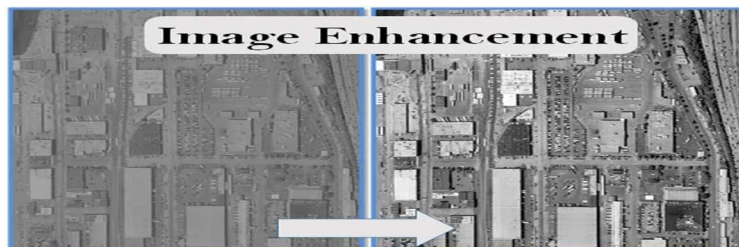
Image Processing for Robotics

- Image processing is a fundamental aspect of robotics, enabling machines to understand and interpret visual information from the world around them. By using techniques such as image enhancement, restoration, and segmentation, robots can extract useful information from images and use it to make decisions and perform tasks.
- Advanced perception and control are essential for robots to be able to operate effectively in complex environments. With advanced perception, robots can recognize objects, faces, and scenes, allowing them to understand and navigate their surroundings.

Image Processing Techniques

1. Image enhancement:

- Image enhancement techniques are used to improve the visual quality of an image, making it easier for a robot to interpret. These techniques include adjusting the brightness, contrast, and color balance of an image, as well as removing noise and blur.
- Image enhancement techniques are used to improve the quality of an image so that a robot can more easily interpret it. For example, increasing the contrast of an image can make it easier for a robot to identify objects within the image.



2. Image restoration:

- Image restoration techniques are used to repair or improve the quality of a degraded image. These techniques include removing noise, blur, and distortion, as well as restoring missing or corrupted parts of an image.
- Image restoration techniques are used to improve the quality of a degraded image, allowing a robot to more accurately interpret it. For example, removing noise from an image can make it easier for a robot to recognize objects within the image.



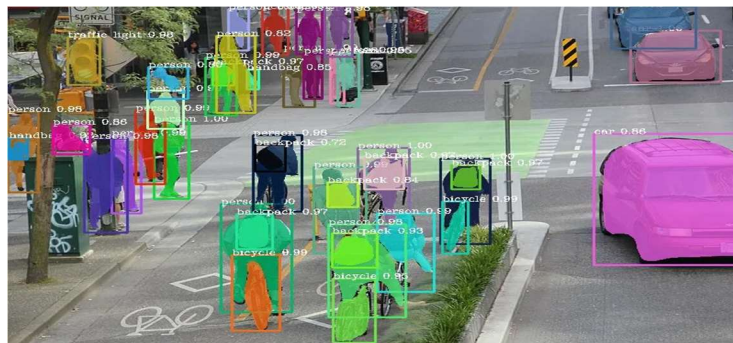
Before



After

3. Image segmentation:

- Image segmentation is the process of dividing an image into multiple segments or regions, each of which corresponds to a different object or part of the scene. This allows a robot to identify and separate different objects or regions of interest within an image.
- Image segmentation is used in robotics to identify and separate different objects or regions of interest within an image. This is particularly useful for object recognition and navigation tasks. For example, a robot can use image segmentation to identify a specific object within an image and navigate towards it. Additionally, image processing techniques can be used for 3D reconstruction and localization, which are fundamental for robots to understand the spatial context and navigate in the environment.



Advanced Perception Techniques

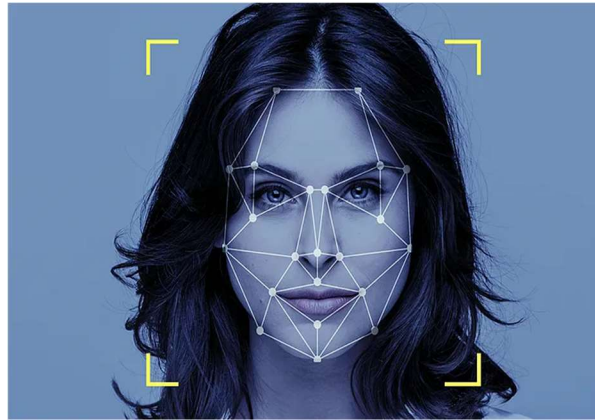
1. Object recognition:

- Object recognition is the ability of a robot to identify and classify objects within an image or video stream. This is typically done using machine learning techniques, such as deep learning, which allow the robot to learn to recognize objects from a training dataset.
- Object recognition is used in robotics for a wide range of tasks, such as grasping, manipulation, and navigation. For example, a robot equipped with object recognition can be programmed to pick up a specific object within a cluttered environment.



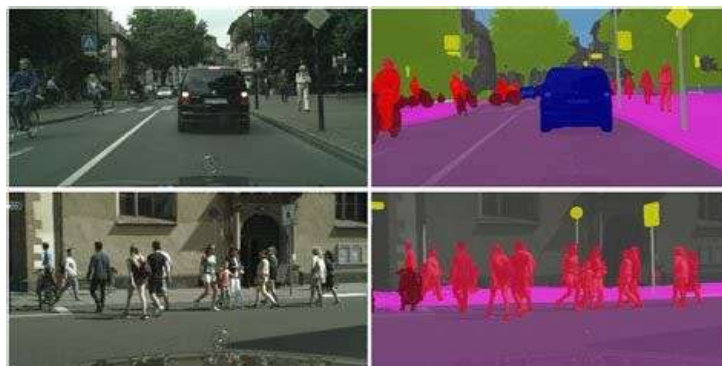
2. Face recognition:

- Face recognition is the ability of a robot to identify and verify individuals within an image or video stream. This is typically done using machine learning techniques, such as deep learning, which allow the robot to learn to recognize faces from a training dataset.
- Face recognition is used in robotics for tasks such as security and personal assistance. For example, a robot equipped with face recognition can be used to verify the identity of an individual before granting them access to a secure area.



3. Scene understanding:

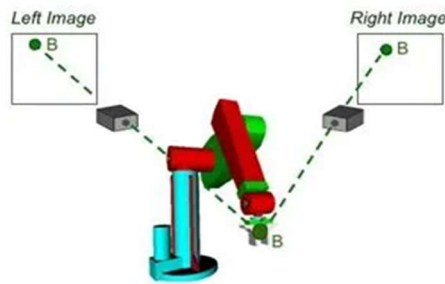
- Scene understanding is the ability of a robot to interpret and understand the context of an image or video stream. This can include identifying the scene type (e.g. indoor or outdoor), understanding the layout of the scene, and recognizing the relationships between objects within the scene.
- Scene understanding is used in robotics for tasks such as navigation and localization. For example, a robot equipped with scene understanding can be programmed to navigate through an unfamiliar environment by recognizing the layout of the scene and identifying landmarks.



Control

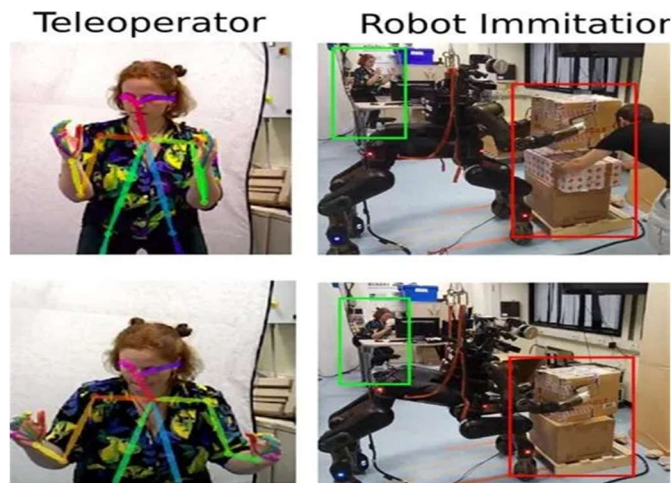
1. Image-based control:

- Image-based control is a technique where the robot's control inputs are based on the visual information it receives from its cameras. This allows the robot to control its movement based on the visual features of the environment, such as the position and orientation of objects.
- Image-based control is used in robotics for tasks such as grasping and manipulation. For example, a robot equipped with image-based control can be programmed to pick up an object by aligning its gripper with the object's position and orientation in the image.



2. Visual servoing:

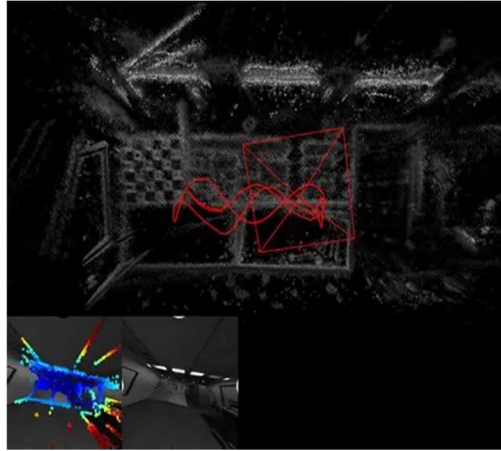
- Visual servoing is a technique where the robot's control inputs are based on the visual error between the current image and a desired image. This allows the robot to control its movement based on the visual features of the environment and the desired position and orientation of the end-effector.
- Visual servoing is used in robotics for tasks such as manipulation and navigation. For example, a robot equipped with visual servoing can be programmed to move its end-effector to a specific position and orientation relative to an object in the image.



3. Visual odometry:

- Visual odometry is a technique where the robot uses visual information to estimate its own motion and position, without the need for external sensors such as wheel encoders.

- Visual odometry is used in robotics for tasks such as localization and navigation. For example, a robot equipped with visual odometry can use visual information to estimate its own position and motion and navigate through an environment without the need for external sensors.



Applications

1. Industrial automation

- In industrial automation, image processing has been successfully implemented in automated warehouses and factories, where robots equipped with image processing can perform tasks such as identifying, picking and placing products with high accuracy and efficiency.

2. Surveillance and security

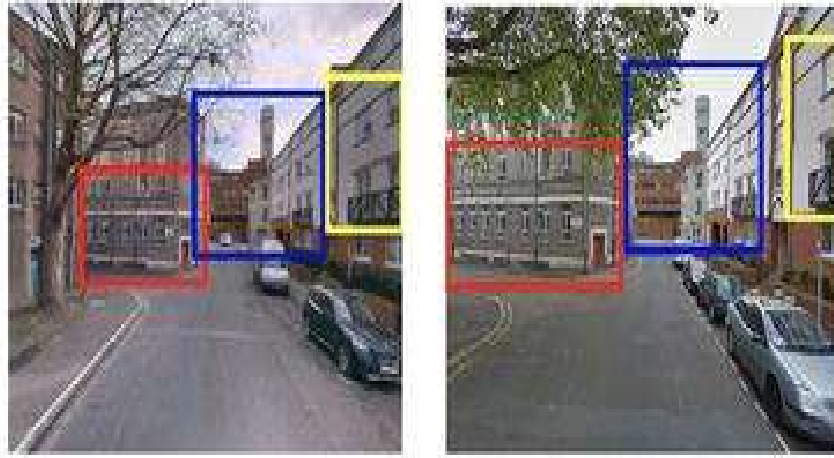
- In surveillance and security, image processing has been used for monitoring and securing large areas, tracking individuals, and identifying suspicious behavior. For example, image processing is used in security cameras to recognize individuals and vehicles, and in drones for surveillance tasks.

3. Robotics in medicine

- In robotics in medicine, image processing has been successfully implemented in image-guided surgery, where robots equipped with image processing can assist surgeons in performing surgeries with greater precision and accuracy. Additionally, image processing is used in therapy robots, such as exoskeletons, to assist patients with mobility issues.

Place Recognition

- Visual Place Recognition is the task of matching a view of a place with a different view of the same place taken at a different time.



- Mobile robots often use place recognition techniques to navigate and orient themselves within an environment. These techniques involve identifying and remembering specific locations based on sensor data such as images, lidar scans, or GPS coordinates.
- Common methods include feature matching, SLAM (Simultaneous Localization and Mapping), and deep learning approaches. These techniques help robots build maps, localize themselves, and make informed decisions about their surroundings while navigating.
- Location recognition, in the context of mobile robots, refers to the ability of a robot to identify and determine its current location within a known map or environment. This process involves comparing the sensor data collected by the robot (such as images, lidar scans, or other sensory information) with previously stored data to find a match.

Various techniques are used for location recognition:

1. Feature Matching:

- Identifying distinctive features in the sensor data and matching them with features in the map. This could involve key points, edges, or other visual or geometric landmarks.

2. SLAM (Simultaneous Localization and Mapping):

- A technique where a robot simultaneously builds a map of its environment while localizing itself within that map. SLAM can be done with a variety of sensors and algorithms.

3. Visual Odometry:

- Estimating the robot's motion by tracking features in consecutive camera images to determine its position and orientation changes.

4. Deep Learning:

- Utilizing neural networks to directly recognize places from sensor data, which can be trained to associate specific locations with patterns in the data.

5. Fingerprinting:

- Creating a database of sensor readings at different locations and using them as a reference to recognize the current location based on similarity.

6. Graph-based Approaches:

- Representing the environment as a graph where nodes correspond to locations and edges represent connections. The robot's current location is estimated by analyzing the graph structure.

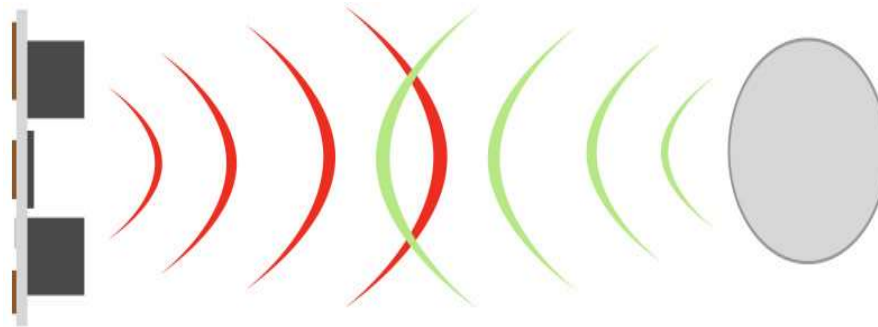
7. Radio Signal Strength:

- Using WiFi or Bluetooth signal strength measurements to identify known locations based on signal patterns.

Range Data

Range:

- Range is a distance, from minimum to maximum, that a sensor is capable of returning accurate readings.



Working Method of Distance Sensors:

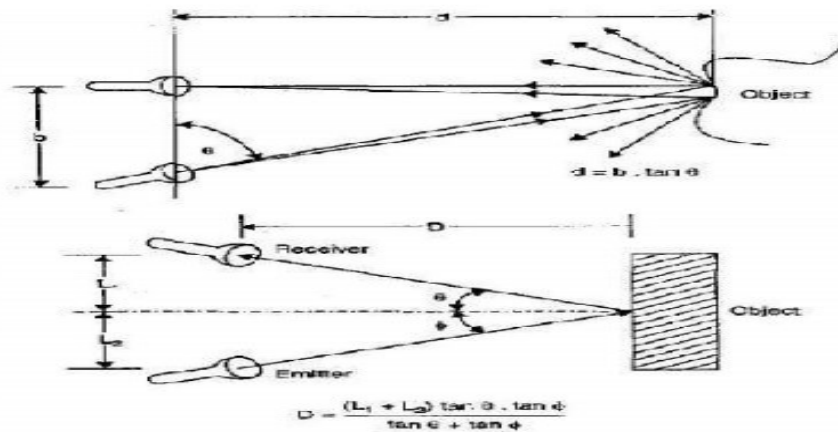
- Distance sensors (or proximity sensors) generally work by outputting a signal of some kind, (eg laser, IR LED, ultrasonic waves) and then reading how it has changed on its return.
- That change may be in the intensity of the returned signal or the time it takes the signal to return.

Working principle of Range sensors

- The distance between the object and the robot hand is measured using the range sensors Within it is range of operation.
- The calculation of the distance is by visual processing. Range sensors find use in robot navigation and avoidance of the obstacles in the path.
- The - location and the general shape characteristics of the part in the work envelope of the robot done by special applications for the range sensors.
- There are several approaches like, triangulation method, structured lighting approach and time-of flight range finders etc. In these cases the source of illumination can be light-source, laser beam or based on ultrasonic.

1. Triangulation Method:

- The object is swept over by a narrow beam of sharp light. The sensor focused on a small spot of the object surface detects the reflected beam of light. If θ is the angle made by the light source and the sensor, the distance



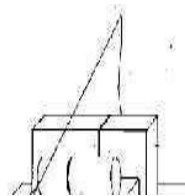
Triangulation Method of Range Sensing.

$$d = b \cdot \tan \theta$$

The distance 'd' can be easily transformed into 3D-co-ordinates

2. Structured Lighting Approach:

- This approach consists of projecting a light pattern the distortion of the pattern to calculate the range. A pattern in use today is a sheet of light generated narrow slit.
- As illustrated in. Figure, the intersection yields a light Stripe which is viewed through a television camera displaced a distance B from the light source.



Resolution:

- Resolution refers to the smallest change in distance that a sensor is able to detect. For example, an IR LED may have a resolution of about 5mm, while a VCSEL unit may have a resolution of about 1mm.

Update rate:

- Usually measured in Hz, update rate comes into play with moving objects. the faster the refresh rate, the more reading per second the sensor will receive, important information if your sensor is traveling towards a fixed object at a high rate of speed.

Distance Sensing Technologies

- There are many unique options to choose from when you want to incorporate a distance/proximity sensor into your project.
- We focus primarily on four varieties of distance sensors: affordable and easy LEDs; crowd-favorite LIDAR; multi-use, multi-purpose ultrasonic; and the compact yet high-resolution VCSEL.

- Each option has its pros and cons, and we have guides, projects and tools to make sure you know which is right for you!

Application:

- An industrial robot's range of motion determines the movements it is capable of. Range of motion for a robot depends upon the degrees of freedom (DOF) it has. Each degree of freedom is represented by an axis. Each axis contains a motor that is responsible for producing a specific motion.

UNIT – III

ROBOT PERCEPTION – QUESTION BANK

PART – A

1. What is sensor?

A sensor is a device that detects and responds to some type of input from the physical environment. Sophisticated sensors can be used to acquire information about the robot's environment or even to directly measure a robot's global position.

2. What are the types of sensors used in mobile robot construction?

- ✓ **Proprioceptive sensors** measure values internal to the system.
 - Examples: Motor Speed, Wheel Load, Robot Arm Joint Angles, Battery Voltage.
- ✓ **Exteroceptive sensors** acquire information from the robot's environment.
 - Examples: Distance Measurements, Light Intensity, Sound Amplitude.
- ✓ **Passive sensors** measure ambient environmental energy entering the sensor.
 - Examples: Microphones, And CCD Or CMOS Cameras.
- ✓ **Active sensors** emit energy into the environment, then measure the environmental reaction.
 - Examples: Wheel Quadrature Encoders, Ultrasonic Sensors, And Laser Rangefinders.

3. Examine the sensor classification.

- ✓ **Proprioceptive sensors** measure values internal to the system.
 - Examples: Motor Speed, Wheel Load, Robot Arm Joint Angles, Battery Voltage.
- ✓ **Exteroceptive sensors** acquire information from the robot's environment.
 - Examples: Distance Measurements, Light Intensity, Sound Amplitude.
- ✓ **Passive sensors** measure ambient environmental energy entering the sensor.
 - Examples: Microphones, And CCD Or CMOS Cameras.
- ✓ **Active sensors** emit energy into the environment, then measure the environmental reaction.
 - Examples: Wheel Quadrature Encoders, Ultrasonic Sensors, And Laser Rangefinders.

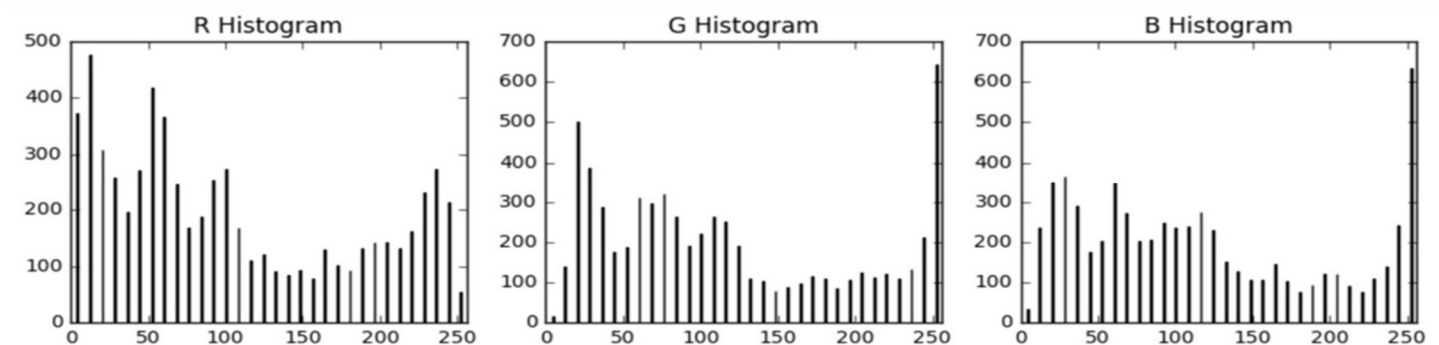
4. Define computer vision for robots.

A Computer vision system uses a sensor in the robot for viewing and recognizing an object with the help of a computer. Computer vision is used in a variety of industrial processes, such as material inspection, object

recognition, pattern recognition, electronic component analysis, along with the recognition of signatures, optical characters, and currency.

5. What is color channel histogram?

- This will give us pixel intensity for each image, allowing us to use the common car colors as a feature for recognition.



6. What are the techniques included in robot arm pic and place of object concepts?

- An intelligent robot is expected to not only be able to perceive the environment, but also interact with the environment.
- Among all these abilities, object grasping is fundamental and significant in that it will bring enormous productivity to society.
- For example, an industrial robot can accomplish the pick-and-bin task which is laborious for human laborers, and a domestic robot is able to provide assistance to disabled people in their daily grasping tasks.

7. What is the technique included in object segmentation process?

- In digital image processing and computer vision, image segmentation is the process of partitioning a digital image into multiple segments.
- The goal of segmentation is to simplify and/or change the representation of an image into something that is more meaningful and easier to analyze.
- Image segmentation is typically used to locate objects and boundaries (lines, curves, etc.) in images.

8. List the application of computer vision for robots.

- Industrial automation
- Surveillance and security
- Robotics in medicine

9. Define image processing technique.

Image processing is a fundamental aspect of robotics, enabling machines to understand and interpret visual information from the world around them. By using techniques such as image enhancement, restoration, and segmentation, robots can extract useful information from images and use it to make decisions and perform tasks.

10. Explain the common image processing techniques in robotics.

✓ **Image Processing Techniques**

- Image enhancement

- Image restoration
- Image segmentation

✓ **Advanced Perception Techniques**

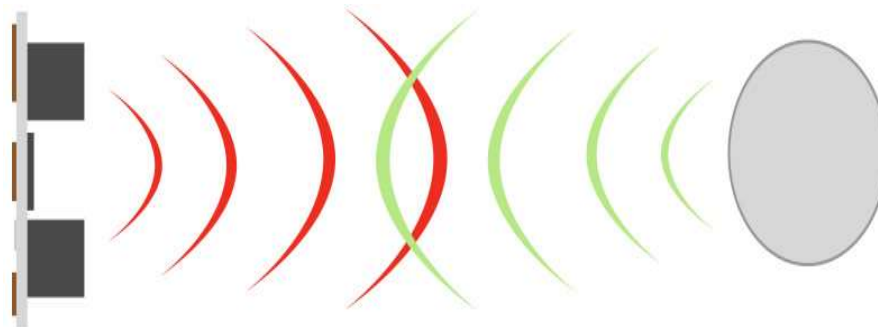
- Object recognition
- Face recognition
- Scene understanding

11. What is place recognition?

- Visual Place Recognition is the task of matching a view of a place with a different view of the same place taken at a different time.
- Mobile robots often use place recognition techniques to navigate and orient themselves within an environment. These techniques involve identifying and remembering specific locations based on sensor data such as images, lidar scans, or GPS coordinates.

12. Define range data.

- Range is a distance, from minimum to maximum, that a sensor is capable of returning accurate readings.



PART – B

1. **Explain the sensors usage in robotics.**
2. **Examine the technique of computer vision for robotics.**
3. **Examine the concepts of image processing in robotics with applications.**
4. **Explain the concepts of place recognition in robot's technology.**
5. **Define range data. Examine the range data algorithm with example.**

5 Mobile Robot Localization

5.1 Introduction

Navigation is one of the most challenging competences required of a mobile robot. Success in navigation requires success at the four building blocks of navigation: *perception*, the robot must interpret its sensors to extract meaningful data; *localization*, the robot must determine its position in the environment (figure 5.1); *cognition*, the robot must decide how to act to achieve its goals; and *motion control*, the robot must modulate its motor outputs to achieve the desired trajectory.

Of these four components (figure 5.2), localization has received the greatest research attention in the past decade and, as a result, significant advances have been made on this front. In this chapter, we explore the successful localization methodologies of recent years. First, section 5.2 describes how sensor and effector uncertainty is responsible for the difficulties of localization. Then, section 5.3 describes two extreme approaches to dealing with the challenge of robot localization: avoiding localization altogether, and performing explicit map-based localization. The remainder of the chapter discusses the question of rep-

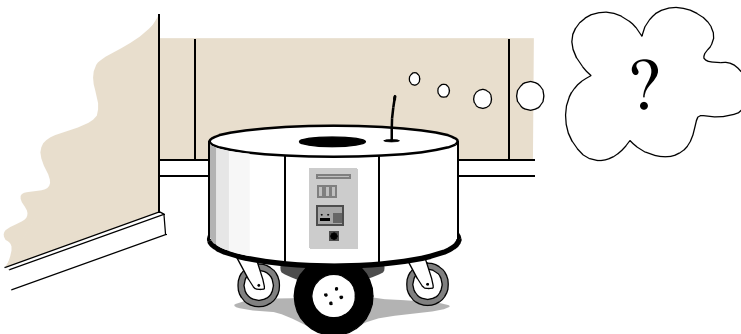


Figure 5.1
Where am I?

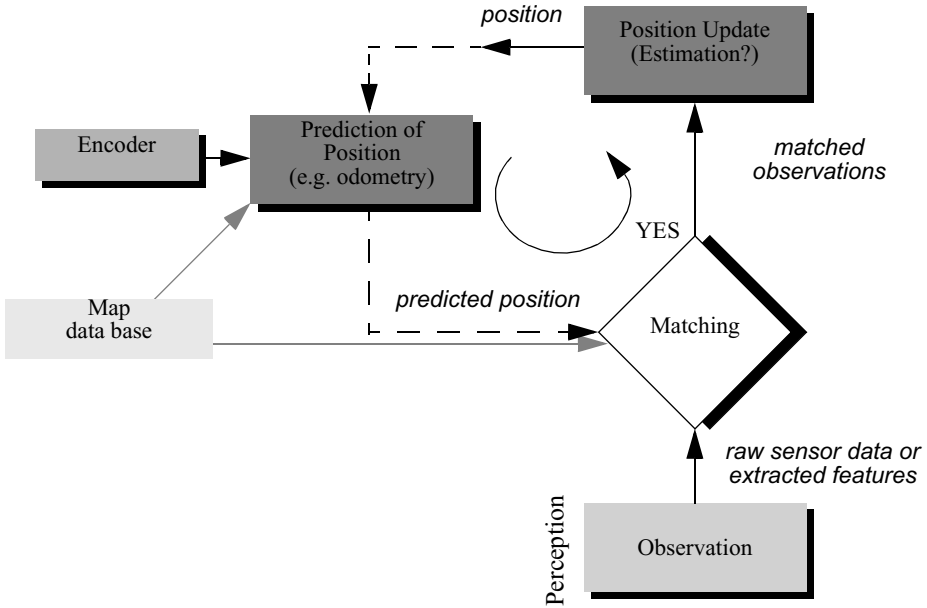


Figure 5.2
General schematic for mobile robot localization.

resentation, then presents case studies of successful localization systems using a variety of representations and techniques to achieve mobile robot localization competence.

5.2 The Challenge of Localization: Noise and Aliasing

If one could attach an accurate GPS (global positioning system) sensor to a mobile robot, much of the localization problem would be obviated. The GPS would inform the robot of its exact position, indoors and outdoors, so that the answer to the question, “Where am I?”, would always be immediately available. Unfortunately, such a sensor is not currently practical. The existing GPS network provides accuracy to within several meters, which is unacceptable for localizing human-scale mobile robots as well as miniature mobile robots such as desk robots and the body-navigating nanorobots of the future. Furthermore, GPS technologies cannot function indoors or in obstructed areas and are thus limited in their workspace.

But, looking beyond the limitations of GPS, localization implies more than knowing one’s absolute position in the Earth’s reference frame. Consider a robot that is interacting with humans. This robot may need to identify its absolute position, but its relative position

with respect to target humans is equally important. Its localization task can include identifying humans using its sensor array, then computing its relative position to the humans. Furthermore, during the *cognition* step a robot will select a strategy for achieving its goals. If it intends to reach a particular location, then localization may not be enough. The robot may need to acquire or build an environmental model, a *map*, that aids it in planning a path to the goal. Once again, localization means more than simply determining an absolute pose in space; it means building a map, then identifying the robot's position relative to that map.

Clearly, the robot's sensors and effectors play an integral role in all the above forms of localization. It is because of the inaccuracy and incompleteness of these sensors and effectors that localization poses difficult challenges. This section identifies important aspects of this sensor and effector suboptimality.

5.2.1 Sensor noise

Sensors are the fundamental robot input for the process of *perception*, and therefore the degree to which sensors can discriminate the world state is critical. *Sensor noise* induces a limitation on the consistency of sensor readings in the same environmental state and, therefore, on the number of useful bits available from each sensor reading. Often, the source of sensor noise problems is that some environmental features are not captured by the robot's representation and are thus overlooked.

For example, a vision system used for indoor navigation in an office building may use the color values detected by its color CCD camera. When the sun is hidden by clouds, the illumination of the building's interior changes because of the windows throughout the building. As a result, hue values are not constant. The color CCD appears noisy from the robot's perspective as if subject to random error, and the hue values obtained from the CCD camera will be unusable, unless the robot is able to note the position of the sun and clouds in its representation.

Illumination dependence is only one example of the apparent noise in a vision-based sensor system. Picture jitter, signal gain, blooming, and blurring are all additional sources of noise, potentially reducing the useful content of a color video image.

Consider the noise level (i.e., apparent random error) of ultrasonic range-measuring sensors (e.g., sonars) as discussed in section 4.1.2.3. When a sonar transducer emits sound toward a relatively smooth and angled surface, much of the signal will coherently reflect away, failing to generate a return echo. Depending on the material characteristics, a small amount of energy may return nonetheless. When this level is close to the gain threshold of the sonar sensor, then the sonar will, at times, succeed and, at other times, fail to detect the object. From the robot's perspective, a virtually unchanged environmental state will result in two different possible sonar readings: one short and one long.

The poor signal-to-noise ratio of a sonar sensor is further confounded by interference between multiple sonar emitters. Often, research robots have between twelve and forty-

eight sonars on a single platform. In acoustically reflective environments, multipath interference is possible between the sonar emissions of one transducer and the echo detection circuitry of another transducer. The result can be dramatically large errors (i.e., underestimation) in ranging values due to a set of coincidental angles. Such errors occur rarely, less than 1% of the time, and are virtually random from the robot's perspective.

In conclusion, sensor noise reduces the useful information content of sensor readings. Clearly, the solution is to take multiple readings into account, employing temporal fusion or multisensor fusion to increase the overall information content of the robot's inputs.

5.2.2 Sensor aliasing

A second shortcoming of mobile robot sensors causes them to yield little information content, further exacerbating the problem of perception and, thus, localization. The problem, known as *sensor aliasing*, is a phenomenon that humans rarely encounter. The human sensory system, particularly the visual system, tends to receive unique inputs in each unique local state. In other words, every different place looks different. The power of this unique mapping is only apparent when one considers situations where this fails to hold. Consider moving through an unfamiliar building that is completely dark. When the visual system sees only black, one's localization system quickly degrades. Another useful example is that of a human-sized maze made from tall hedges. Such mazes have been created for centuries, and humans find them extremely difficult to solve without landmarks or clues because, without visual uniqueness, human localization competence degrades rapidly.

In robots, the nonuniqueness of sensor readings, or *sensor aliasing*, is the norm and not the exception. Consider a narrow-beam rangefinder such as an ultrasonic or infrared rangefinder. This sensor provides range information in a single direction without any additional data regarding material composition such as color, texture, and hardness. Even for a robot with several such sensors in an array, there are a variety of environmental states that would trigger the same sensor values across the array. Formally, there is a many-to-one mapping from environmental states to the robot's perceptual inputs. Thus, the robot's percepts cannot distinguish from among these many states. A classic problem with sonar-based robots involves distinguishing between humans and inanimate objects in an indoor setting. When facing an apparent obstacle in front of itself, should the robot say "Excuse me" because the obstacle may be a moving human, or should the robot plan a path around the object because it may be a cardboard box? With sonar alone, these states are aliased and differentiation is impossible.

The problem posed to navigation because of sensor aliasing is that, even with noise-free sensors, the amount of information is generally insufficient to identify the robot's position from a single-percept reading. Thus techniques must be employed by the robot programmer that base the robot's localization on a series of readings and, thus, sufficient information to recover the robot's position over time.

5.2.3 Effector noise

The challenges of localization do not lie with sensor technologies alone. Just as robot sensors are noisy, limiting the information content of the signal, so robot effectors are also noisy. In particular, a single action taken by a mobile robot may have several different possible results, even though from the robot's point of view the initial state before the action was taken is well known.

In short, mobile robot effectors introduce uncertainty about future state. Therefore the simple act of moving tends to increase the uncertainty of a mobile robot. There are, of course, exceptions. Using *cognition*, the motion can be carefully planned so as to minimize this effect, and indeed sometimes to actually result in more certainty. Furthermore, when the robot's actions are taken in concert with careful interpretation of sensory feedback, it can compensate for the uncertainty introduced by noisy actions using the information provided by the sensors.

First, however, it is important to understand the precise nature of the effector noise that impacts mobile robots. It is important to note that, from the robot's point of view, this error in motion is viewed as an error in odometry, or the robot's inability to estimate its own position over time using knowledge of its kinematics and dynamics. The true source of error generally lies in an incomplete model of the environment. For instance, the robot does not model the fact that the floor may be sloped, the wheels may slip, and a human may push the robot. All of these unmodeled sources of error result in inaccuracy between the physical motion of the robot, the intended motion of the robot, and the proprioceptive sensor estimates of motion.

In odometry (wheel sensors only) and dead reckoning (also heading sensors) the position update is based on *proprioceptive* sensors. The movement of the robot, sensed with wheel encoders or heading sensors or both, is integrated to compute position. Because the sensor measurement errors are integrated, the position error accumulates over time. Thus the position has to be updated from time to time by other localization mechanisms. Otherwise the robot is not able to maintain a meaningful position estimate in the long run.

In the following we concentrate on odometry based on the wheel sensor readings of a differential-drive robot only (see also [4, 57, 58]). Using additional heading sensors (e.g., gyroscope) can help to reduce the cumulative errors, but the main problems remain the same.

There are many sources of odometric error, from environmental factors to resolution:

- Limited resolution during integration (time increments, measurement resolution, etc.);
- Misalignment of the wheels (deterministic);
- Uncertainty in the wheel diameter and in particular unequal wheel diameter (deterministic);
- Variation in the contact point of the wheel;

- Unequal floor contact (slipping, nonplanar surface, etc.).

Some of the errors might be *deterministic (systematic)*, thus they can be eliminated by proper calibration of the system. However, there are still a number of *nondeterministic (random)* errors which remain, leading to uncertainties in position estimation over time. From a geometric point of view one can classify the errors into three types:

1. Range error: integrated path length (distance) of the robot's movement
→ sum of the wheel movements
2. Turn error: similar to range error, but for turns
→ difference of the wheel motions
3. Drift error: difference in the error of the wheels leads to an error in the robot's angular orientation

Over long periods of time, turn and drift errors far outweigh range errors, since their contribution to the overall position error is nonlinear. Consider a robot whose position is initially perfectly well-known, moving forward in a straight line along the x -axis. The error in the y -position introduced by a move of d meters will have a component of $d \sin \Delta\theta$, which can be quite large as the angular error $\Delta\theta$ grows. Over time, as a mobile robot moves about the environment, the rotational error between its internal reference frame and its original reference frame grows quickly. As the robot moves away from the origin of these reference frames, the resulting linear error in position grows quite large. It is instructive to establish an error model for odometric accuracy and see how the errors propagate over time.

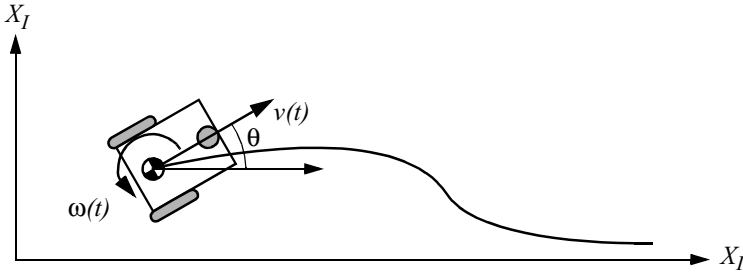
5.2.4 An error model for odometric position estimation

Generally the pose (position) of a robot is represented by the vector

$$p = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \quad (5.1)$$

For a differential-drive robot the position can be estimated starting from a known position by integrating the movement (summing the incremental travel distances). For a discrete system with a fixed sampling interval Δt the incremental travel distances $(\Delta x; \Delta y; \Delta\theta)$ are

$$\Delta x = \Delta s \cos(\theta + \Delta\theta/2) \quad (5.2)$$

**Figure 5.3**

Movement of a differential-drive robot.

$$\Delta y = \Delta s \sin(\theta + \Delta\theta/2) \quad (5.3)$$

$$\Delta\theta = \frac{\Delta s_r - \Delta s_l}{b} \quad (5.4)$$

$$\Delta s = \frac{\Delta s_r + \Delta s_l}{2} \quad (5.5)$$

where

$(\Delta x; \Delta y; \Delta\theta)$ = path traveled in the last sampling interval;

$\Delta s_r; \Delta s_l$ = traveled distances for the right and left wheel respectively;

b = distance between the two wheels of differential-drive robot.

Thus we get the updated position p' :

$$p' = \begin{bmatrix} x' \\ y' \\ \theta' \end{bmatrix} = p + \begin{bmatrix} \Delta s \cos(\theta + \Delta\theta/2) \\ \Delta s \sin(\theta + \Delta\theta/2) \\ \Delta\theta \end{bmatrix} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} + \begin{bmatrix} \Delta s \cos(\theta + \Delta\theta/2) \\ \Delta s \sin(\theta + \Delta\theta/2) \\ \Delta\theta \end{bmatrix} \quad (5.6)$$

By using the relation for $(\Delta s; \Delta\theta)$ of equations (5.4) and (5.5) we further obtain the basic equation for odometric position update (for differential drive robots):

$$p' = f(x, y, \theta, \Delta s_r, \Delta s_l) = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} + \begin{bmatrix} \frac{\Delta s_r + \Delta s_l}{2} \cos\left(\theta + \frac{\Delta s_r - \Delta s_l}{2b}\right) \\ \frac{\Delta s_r + \Delta s_l}{2} \sin\left(\theta + \frac{\Delta s_r - \Delta s_l}{2b}\right) \\ \frac{\Delta s_r - \Delta s_l}{b} \end{bmatrix} \quad (5.7)$$

As we discussed earlier, odometric position updates can give only a very rough estimate of the actual position. Owing to integration errors of the uncertainties of p and the motion errors during the incremental motion $(\Delta s_r; \Delta s_l)$ the position error based on odometry integration grows with time.

In the next step we will establish an error model for the integrated position p' to obtain the covariance matrix $\Sigma_{p'}$ of the odometric position estimate. To do so, we assume that at the starting point the initial covariance matrix Σ_p is known. For the motion increment $(\Delta s_r; \Delta s_l)$ we assume the following covariance matrix Σ_Δ :

$$\Sigma_\Delta = \text{covar}(\Delta s_r, \Delta s_l) = \begin{bmatrix} k_r |\Delta s_r| & 0 \\ 0 & k_l |\Delta s_l| \end{bmatrix} \quad (5.8)$$

where Δs_r and Δs_l are the distances traveled by each wheel, and k_r , k_l are error constants representing the nondeterministic parameters of the motor drive and the wheel-floor interaction. As you can see, in equation (5.8) we made the following assumptions:

- The two errors of the individually driven wheels are independent⁵;
- The variance of the errors (left and right wheels) are proportional to the absolute value of the traveled distances $(\Delta s_r; \Delta s_l)$.

These assumptions, while not perfect, are suitable and will thus be used for the further development of the error model. The *motion errors* are due to imprecise movement because of deformation of wheel, slippage, unequal floor, errors in encoders, and so on. The values for the error constants k_r and k_l depend on the robot and the environment and should be experimentally established by performing and analyzing representative movements.

If we assume that p and $\Delta_{r,l} = (\Delta s_r; \Delta s_l)$ are uncorrelated and the derivation of f [equation (5.7)] is reasonably approximated by the first-order Taylor expansion (linearization), we conclude, using the error propagation law (see section 4.2.2),

5. If there is more knowledge regarding the actual robot kinematics, the correlation terms of the covariance matrix could also be used.

$$\Sigma_{p'} = \nabla_p f \cdot \Sigma_p \cdot \nabla_p f^T + \nabla_{\Delta_{rl}} f \cdot \Sigma_{\Delta} \cdot \nabla_{\Delta_{rl}} f^T \quad (5.9)$$

The covariance matrix Σ_p is, of course, always given by the $\Sigma_{p'}$ of the previous step, and can thus be calculated after specifying an initial value (e.g., 0).

Using equation (5.7) we can develop the two *Jacobians*, $F_p = \nabla_p f$ and $F_{\Delta_{rl}} = \nabla_{\Delta_{rl}} f$:

$$F_p = \nabla_p f = \nabla_p (f^T) = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} & \frac{\partial f}{\partial \theta} \end{bmatrix} = \begin{bmatrix} 1 & 0 & -\Delta s \sin(\theta + \Delta\theta/2) \\ 0 & 1 & \Delta s \cos(\theta + \Delta\theta/2) \\ 0 & 0 & 1 \end{bmatrix} \quad (5.10)$$

$$F_{\Delta_{rl}} = \begin{bmatrix} \frac{1}{2} \cos\left(\theta + \frac{\Delta\theta}{2}\right) - \frac{\Delta s}{2b} \sin\left(\theta + \frac{\Delta\theta}{2}\right) & \frac{1}{2} \cos\left(\theta + \frac{\Delta\theta}{2}\right) + \frac{\Delta s}{2b} \sin\left(\theta + \frac{\Delta\theta}{2}\right) \\ \frac{1}{2} \sin\left(\theta + \frac{\Delta\theta}{2}\right) + \frac{\Delta s}{2b} \cos\left(\theta + \frac{\Delta\theta}{2}\right) & \frac{1}{2} \sin\left(\theta + \frac{\Delta\theta}{2}\right) - \frac{\Delta s}{2b} \cos\left(\theta + \frac{\Delta\theta}{2}\right) \\ \frac{1}{b} & -\frac{1}{b} \end{bmatrix} \quad (5.11)$$

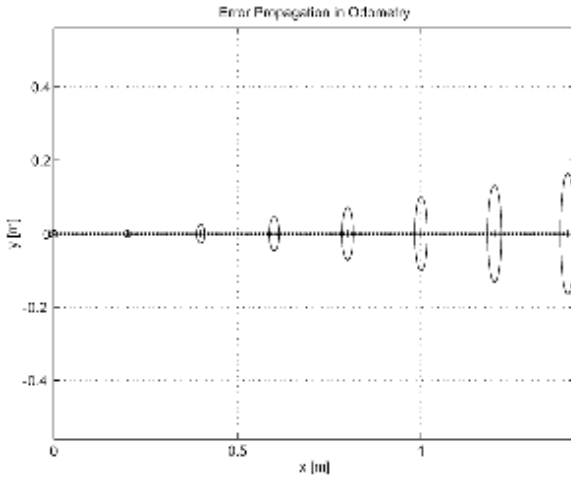
The details for arriving at equation (5.11) are

$$F_{\Delta_{rl}} = \nabla_{\Delta_{rl}} f = \begin{bmatrix} \frac{\partial f}{\partial \Delta s_r} & \frac{\partial f}{\partial \Delta s_l} \end{bmatrix} = \dots \quad (5.12)$$

$$\begin{bmatrix} \frac{\partial \Delta s}{\partial \Delta s_r} \cos\left(\theta + \frac{\Delta\theta}{2}\right) + \frac{\Delta s}{2} \sin\left(\theta + \frac{\Delta\theta}{2}\right) \frac{\partial \Delta\theta}{\partial \Delta s_r} & \frac{\partial \Delta s}{\partial \Delta s_l} \cos\left(\theta + \frac{\Delta\theta}{2}\right) + \frac{\Delta s}{2} \sin\left(\theta + \frac{\Delta\theta}{2}\right) \frac{\partial \Delta\theta}{\partial \Delta s_l} \\ \frac{\partial \Delta s}{\partial \Delta s_r} \sin\left(\theta + \frac{\Delta\theta}{2}\right) + \frac{\Delta s}{2} \cos\left(\theta + \frac{\Delta\theta}{2}\right) \frac{\partial \Delta\theta}{\partial \Delta s_r} & \frac{\partial \Delta s}{\partial \Delta s_l} \sin\left(\theta + \frac{\Delta\theta}{2}\right) + \frac{\Delta s}{2} \cos\left(\theta + \frac{\Delta\theta}{2}\right) \frac{\partial \Delta\theta}{\partial \Delta s_l} \\ \frac{\partial \Delta\theta}{\partial \Delta s_r} & \frac{\partial \Delta\theta}{\partial \Delta s_l} \end{bmatrix} \quad (5.13)$$

and with

$$\Delta s = \frac{\Delta s_r + \Delta s_l}{2} ; \quad \Delta\theta = \frac{\Delta s_r - \Delta s_l}{b} \quad (5.14)$$

**Figure 5.4**

Growth of the pose uncertainty for straight-line movement: Note that the uncertainty in y grows much faster than in the direction of movement. This results from the integration of the uncertainty about the robot's orientation. The ellipses drawn around the robot positions represent the uncertainties in the x,y direction (e.g. 3σ). The uncertainty of the orientation θ is not represented in the picture although its effect can be indirectly observed.

$$\frac{\partial \Delta s}{\partial \Delta s_r} = \frac{1}{2} \quad ; \quad \frac{\partial \Delta s}{\partial \Delta s_l} = \frac{1}{2} \quad ; \quad \frac{\partial \Delta \theta}{\partial \Delta s_r} = \frac{1}{b} \quad ; \quad \frac{\partial \Delta \theta}{\partial \Delta s_l} = -\frac{1}{b} \quad (5.15)$$

we obtain equation (5.11).

Figures 5.4 and 5.5 show typical examples of how the position errors grow with time. The results have been computed using the error model presented above.

Once the error model has been established, the error parameters must be specified. One can compensate for deterministic errors properly calibrating the robot. However the error parameters specifying the nondeterministic errors can only be quantified by statistical (repetitive) measurements. A detailed discussion of odometric errors and a method for calibration and quantification of deterministic and nondeterministic errors can be found in [5]. A method for on-the-fly odometry error estimation is presented in [105].

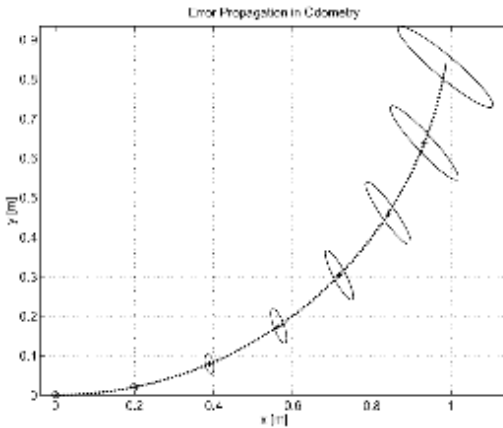


Figure 5.5

Growth of the pose uncertainty for circular movement ($r = \text{const}$): Again, the uncertainty perpendicular to the movement grows much faster than that in the direction of movement. Note that the main axis of the uncertainty ellipse does not remain perpendicular to the direction of movement.

5.3 To Localize or Not to Localize: Localization-Based Navigation versus Programmed Solutions

Figure 5.6 depicts a standard indoor environment that a mobile robot navigates. Suppose that the mobile robot in question must deliver messages between two specific rooms in this environment: rooms *A* and *B*. In creating a navigation system, it is clear that the mobile robot will need sensors and a motion control system. Sensors are absolutely required to avoid hitting moving obstacles such as humans, and some motion control system is required so that the robot can deliberately move.

It is less evident, however, whether or not this mobile robot will require a *localization system*. Localization may seem mandatory in order to successfully navigate between the two rooms. It is through localizing on a map, after all, that the robot can hope to recover its position and detect when it has arrived at the goal location. It is true that, at the least, the robot must have a way of detecting the goal location. However, explicit localization with reference to a map is not the only strategy that qualifies as a goal detector.

An alternative, espoused by the behavior-based community, suggests that, since sensors and effectors are noisy and information-limited, one should avoid creating a geometric map for localization. Instead, this community suggests designing sets of behaviors that together result in the desired robot motion. Fundamentally, this approach avoids explicit reasoning about localization and position, and thus generally avoids explicit path planning as well.

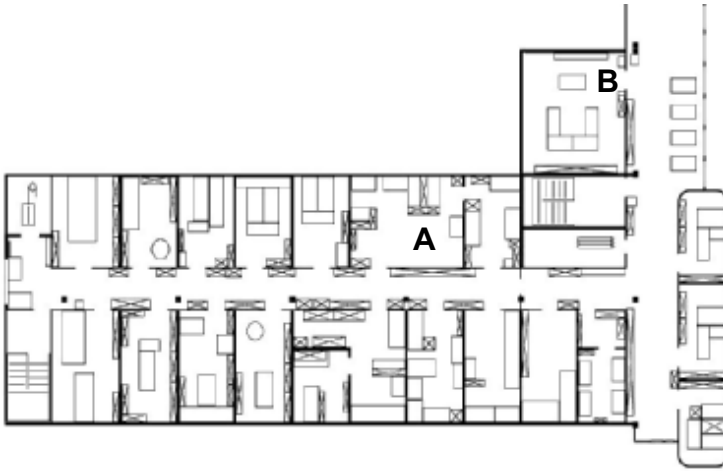


Figure 5.6
A sample environment.

This technique is based on a belief that there exists a procedural solution to the particular navigation problem at hand. For example, in figure 5.6, the behavioralist approach to navigating from room *A* to room *B* might be to design a left-wall following behavior and a detector for room *B* that is triggered by some unique queue in room *B*, such as the color of the carpet. Then the robot can reach room *B* by engaging the left-wall follower with the room *B* detector as the termination condition for the program.

The architecture of this solution to a specific navigation problem is shown in figure 5.7. The key advantage of this method is that, when possible, it may be implemented very quickly for a single environment with a small number of goal positions. It suffers from some disadvantages, however. First, the method does not directly scale to other environments or to larger environments. Often, the navigation code is location-specific, and the same degree of coding and debugging is required to move the robot to a new environment.

Second, the underlying procedures, such as *left-wall-follow*, must be carefully designed to produce the desired behavior. This task may be time-consuming and is heavily dependent on the specific robot hardware and environmental characteristics.

Third, a behavior-based system may have multiple active behaviors at any one time. Even when individual behaviors are tuned to optimize performance, this fusion and rapid switching between multiple behaviors can negate that fine-tuning. Often, the addition of each new incremental behavior forces the robot designer to retune all of the existing behaviors again to ensure that the new interactions with the freshly introduced behavior are all stable.

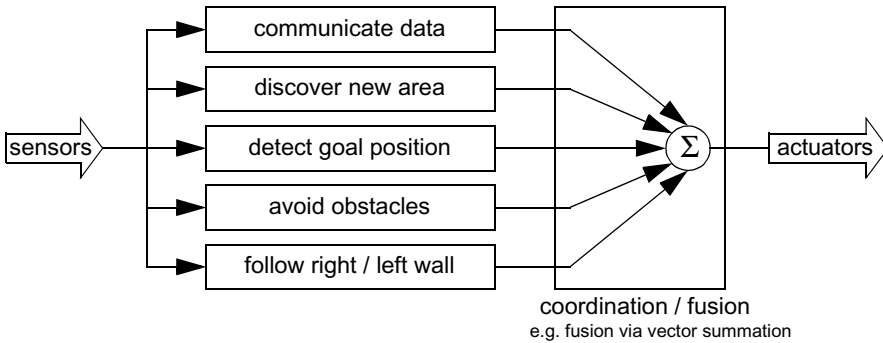


Figure 5.7
An architecture for behavior-based navigation.

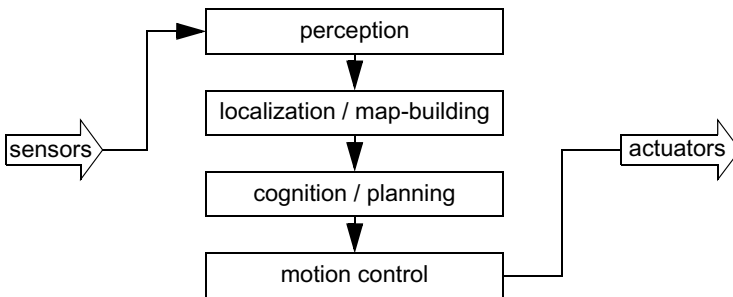


Figure 5.8
An architecture for map-based (or model-based) navigation.

In contrast to the behavior-based approach, the map-based approach includes both *localization* and *cognition* modules (see figure 5.8). In map-based navigation, the robot explicitly attempts to localize by collecting sensor data, then updating some belief about its position with respect to a map of the environment. The key advantages of the map-based approach for navigation are as follows:

- The explicit, map-based concept of position makes the system's belief about position transparently available to the human operators.
- The existence of the map itself represents a medium for communication between human and robot: the human can simply give the robot a new map if the robot goes to a new environment.

- The map, if created by the robot, can be used by humans as well, achieving two uses.

The map-based approach will require more up-front development effort to create a navigating mobile robot. The hope is that the development effort results in an architecture that can successfully map and navigate a variety of environments, thereby amortizing the up-front design cost over time.

Of course the key risk of the map-based approach is that an internal representation, rather than the real world itself, is being constructed and *trusted* by the robot. If that model diverges from reality (i.e., if the map is wrong), then the robot's behavior may be undesirable, even if the raw sensor values of the robot are only transiently incorrect.

In the remainder of this chapter, we focus on a discussion of map-based approaches and, specifically, the localization component of these techniques. These approaches are particularly appropriate for study given their significant recent successes in enabling mobile robots to navigate a variety of environments, from academic research buildings, to factory floors, and to museums around the world.

5.4 Belief Representation

The fundamental issue that differentiates various map-based localization systems is the issue of *representation*. There are two specific concepts that the robot must represent, and each has its own unique possible solutions. The robot must have a representation (a model) of the environment, or a map. What aspects of the environment are contained in this map? At what level of fidelity does the map represent the environment? These are the design questions for *map representation*.

The robot must also have a representation of its belief regarding its position on the map. Does the robot identify a single unique position as its current position, or does it describe its position in terms of a set of possible positions? If multiple possible positions are expressed in a single belief, how are those multiple positions ranked, if at all? These are the design questions for *belief representation*.

Decisions along these two design axes can result in varying levels of architectural complexity, computational complexity, and overall localization accuracy. We begin by discussing belief representation. The first major branch in a taxonomy of belief representation systems differentiates between single-hypothesis and multiple-hypothesis belief systems. The former covers solutions in which the robot postulates its unique position, whereas the latter enables a mobile robot to describe the degree to which it is uncertain about its position. A sampling of different belief and map representations is shown in figure 5.9.

5.4.1 Single-hypothesis belief

The single-hypothesis belief representation is the most direct possible postulation of mobile robot position. Given some environmental map, the robot's belief about position is

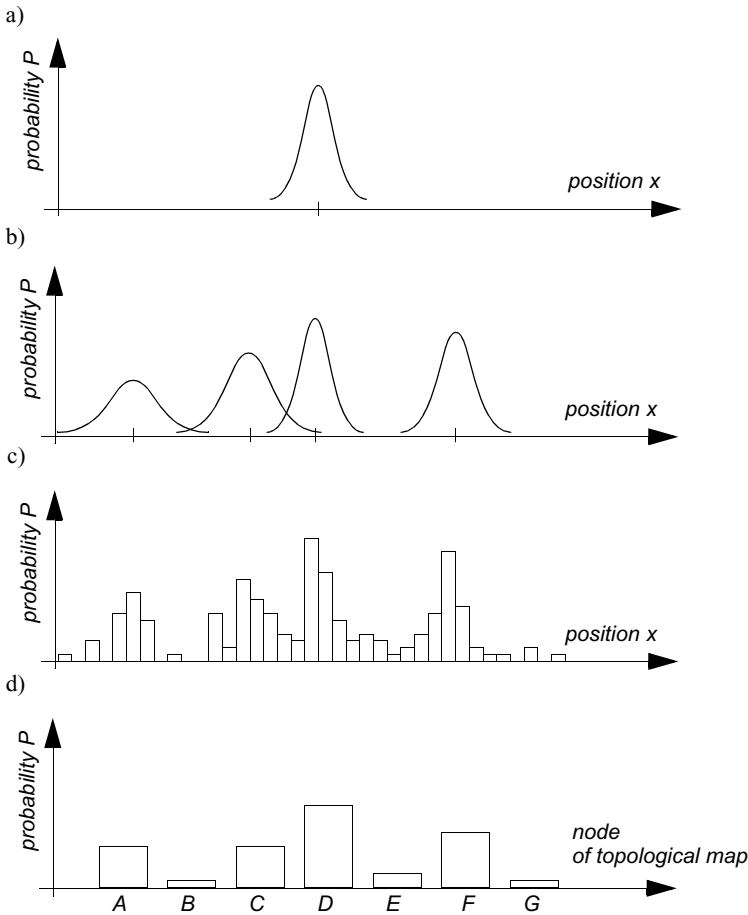


Figure 5.9

Belief representation regarding the robot position (1D) in continuous and discretized (tessellated) maps. (a) Continuous map with single-hypothesis belief, e.g., single Gaussian centered at a single continuous value. (b) Continuous map with multiple-hypothesis belief, e.g., multiple Gaussians centered at multiple continuous values. (c) Discretized (decomposed) grid map with probability values for all possible robot positions, e.g.; Markov approach. (d) Discretized topological map with probability value for all possible nodes (topological robot positions), e.g.; Markov approach.

expressed as a single unique point on the map. In figure 5.10, three examples of a single-hypothesis belief are shown using three different map representations of the same actual environment (figure 5.10a). In figure 5.10b, a single point is geometrically annotated as the robot's position in a continuous 2D geometric map. In figure 5.10c, the map is a discrete, tessellated map, and the position is noted at the same level of fidelity as the map cell size. In figure 5.10d, the map is not geometric at all but abstract and topological. In this case, the single hypothesis of position involves identifying a single node i in the topological graph as the robot's position.

The principal advantage of the single-hypothesis representation of position stems from the fact that, given a unique belief, there is no position ambiguity. The unambiguous nature of this representation facilitates decision-making at the robot's cognitive level (e.g., path planning). The robot can simply assume that its belief is correct, and can then select its future actions based on its unique position.

Just as decision-making is facilitated by a single-position hypothesis, so updating the robot's belief regarding position is also facilitated, since the single position must be updated by definition to a new, single position. The challenge with this position update approach, which ultimately is the principal disadvantage of single-hypothesis representation, is that robot motion often induces uncertainty due to effector and sensor noise. Therefore, forcing the position update process to always generate a *single* hypothesis of position is challenging and, often, impossible.

5.4.2 Multiple-hypothesis belief

In the case of multiple-hypothesis beliefs regarding position, the robot tracks not just a single possible position but a possibly infinite set of positions.

In one simple example originating in the work of Jean-Claude Latombe [21, 99], the robot's position is described in terms of a convex polygon positioned in a 2D map of the environment. This multiple-hypothesis representation communicates the set of possible robot positions geometrically, with no preference ordering over the positions. Each point in the map is simply either contained by the polygon and, therefore, in the robot's belief set, or outside the polygon and thereby excluded. Mathematically, the position polygon serves to partition the space of possible robot positions. Such a polygonal representation of the multiple-hypothesis belief can apply to a continuous, geometric map of the environment [35] or, alternatively, to a tessellated, discrete approximation to the continuous environment.

It may be useful, however, to incorporate some ordering on the possible robot positions, capturing the fact that some robot positions are likelier than others. A strategy for representing a continuous multiple-hypothesis belief state along with a preference ordering over possible positions is to model the belief as a mathematical distribution. For example, [50, 142] notate the robot's position belief using an $\{X, Y\}$ point in the 2D environment as the

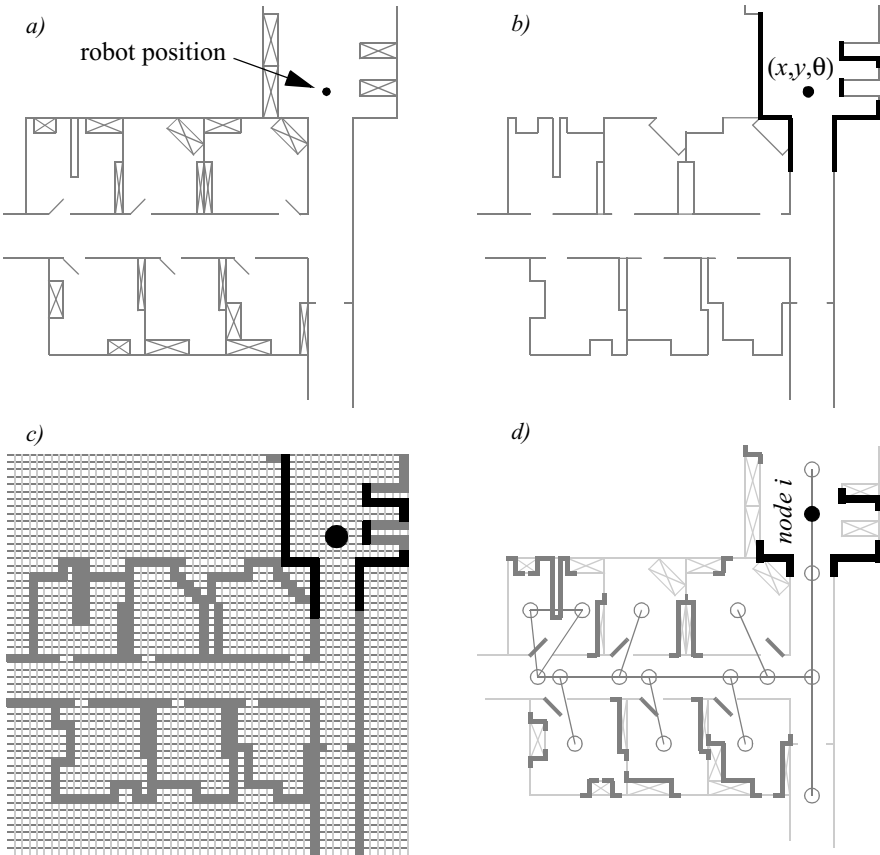


Figure 5.10

Three examples of single hypotheses of position using different map representations: (a) real map with walls, doors and furniture → around 100 lines with two parameters; (c) occupancy grid-based map → around 3000 grid cells size 50 x 50 cm; (d) topological map using line features (Z/S lines) and doors → around 50 features and 18 nodes.

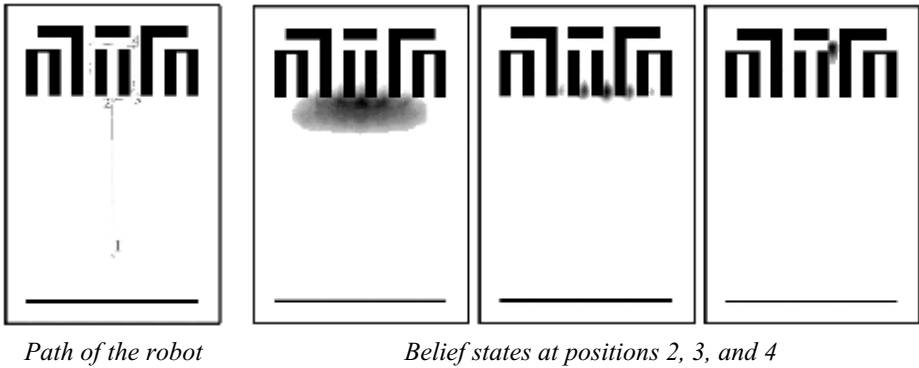


Figure 5.11

Example of multiple-hypothesis tracking (courtesy of W. Burgard [49]). The belief state that is largely distributed becomes very certain after moving to position 4. Note that darker coloring represents higher probability.

mean μ plus a standard deviation parameter σ , thereby defining a Gaussian distribution. The intended interpretation is that the distribution at each position represents the probability assigned to the robot being at that location. This representation is particularly amenable to mathematically defined tracking functions, such as the Kalman filter, that are designed to operate efficiently on Gaussian distributions.

An alternative is to represent the set of possible robot positions, not using a single Gaussian probability density function, but using discrete markers for each possible position. In this case, each possible robot position is individually noted along with a confidence or probability parameter (see figure 5.11). In the case of a highly tessellated map this can result in thousands or even tens of thousands of possible robot positions in a single-belief state.

The key advantage of the multiple-hypothesis representation is that the robot can explicitly maintain uncertainty regarding its position. If the robot only acquires partial information regarding position from its sensors and effectors, that information can conceptually be incorporated in an updated belief.

A more subtle advantage of this approach revolves around the robot's ability to explicitly measure its own degree of uncertainty regarding position. This advantage is the key to a class of localization and navigation solutions in which the robot not only reasons about reaching a particular goal but reasons about the future trajectory of its own belief state. For instance, a robot may choose paths that minimize its future position uncertainty. An example of this approach is [141], in which the robot plans a path from point *A* to point *B* that takes it near a series of landmarks in order to mitigate localization difficulties. This type of

explicit reasoning about the effect that trajectories will have on the quality of localization requires a multiple-hypothesis representation.

One of the fundamental disadvantages of multiple-hypothesis approaches involves decision-making. If the robot represents its position as a region or set of possible positions, then how shall it decide what to do next? Figure 5.11 provides an example. At position 3, the robot's belief state is distributed among five hallways separately. If the goal of the robot is to travel down one particular hallway, then given this belief state, what action should the robot choose?

The challenge occurs because some of the robot's possible positions imply a motion trajectory that is inconsistent with some of its other possible positions. One approach that we will see in the case studies below is to assume, for decision-making purposes, that the robot is physically at the most probable location in its belief state, then to choose a path based on that current position. But this approach demands that each possible position have an associated probability.

In general, the right approach to such decision-making problems would be to decide on trajectories that eliminate the ambiguity explicitly. But this leads us to the second major disadvantage of multiple-hypothesis approaches. In the most general case, they can be computationally very expensive. When one reasons in a 3D space of discrete possible positions, the number of possible belief states in the single-hypothesis case is limited to the number of possible positions in the 3D world. Consider this number to be N . When one moves to an arbitrary multiple-hypothesis representation, then the number of possible belief states is the power set of N , which is far larger: 2^N . Thus explicit reasoning about the possible trajectory of the belief state over time quickly becomes computationally untenable as the size of the environment grows.

There are, however, specific forms of multiple-hypothesis representations that are somewhat more constrained, thereby avoiding the computational explosion while allowing a limited type of multiple-hypothesis belief. For example, if one assumes a Gaussian distribution of probability centered at a single position, then the problem of representation and tracking of belief becomes equivalent to Kalman filtering, a straightforward mathematical process described below. Alternatively, a highly tessellated map representation combined with a limit of ten possible positions in the belief state, results in a discrete update cycle that is, at worst, only ten times more computationally expensive than a single-hypothesis belief update. And other ways to cope with the complexity problem, still being precise and computationally cheap, are hybrid metric-topological approaches [145, 147] or multi-Gaussian position estimation [35, 60, 81].

In conclusion, the most critical benefit of the multiple-hypothesis belief state is the ability to maintain a sense of position while explicitly annotating the robot's uncertainty about its own position. This powerful representation has enabled robots with limited sensory

information to navigate robustly in an array of environments, as we shall see in the case studies below.

5.5 Map Representation

The problem of representing the environment in which the robot moves is a dual of the problem of representing the robot's possible position or positions. Decisions made regarding the environmental representation can have impact on the choices available for robot position representation. Often the fidelity of the position representation is bounded by the fidelity of the map.

Three fundamental relationships must be understood when choosing a particular map representation:

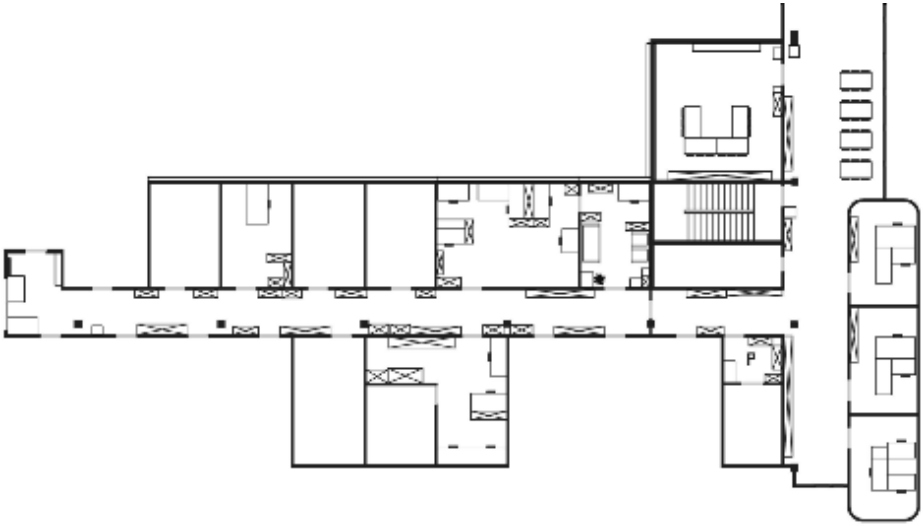
1. The precision of the map must appropriately match the precision with which the robot needs to achieve its goals.
2. The precision of the map and the type of features represented must match the precision and data types returned by the robot's sensors.
3. The complexity of the map representation has direct impact on the computational complexity of reasoning about mapping, localization, and navigation.

In the following sections, we identify and discuss critical design choices in creating a map representation. Each such choice has great impact on the relationships listed above and on the resulting robot localization architecture. As we shall see, the choice of possible map representations is broad. Selecting an appropriate representation requires understanding all of the trade-offs inherent in that choice as well as understanding the specific context in which a particular mobile robot implementation must perform localization. In general, the environmental representation and model can be roughly classified as presented in chapter 4, section 4.3.

5.5.1 Continuous representations

A continuous-valued map is one method for *exact* decomposition of the environment. The position of environmental features can be annotated precisely in continuous space. Mobile robot implementations to date use continuous maps only in 2D representations, as further dimensionality can result in computational explosion.

A common approach is to combine the exactness of a continuous representation with the compactness of the *closed-world assumption*. This means that one assumes that the representation will specify all environmental objects in the map, and that any area in the map that is devoid of objects has no objects in the corresponding portion of the environment. Thus, the total storage needed in the map is proportional to the density of objects in the environment, and a sparse environment can be represented by a low-memory map.

**Figure 5.12**

A continuous representation using polygons as environmental obstacles.

One example of such a representation, shown in figure 5.12, is a 2D representation in which polygons represent all obstacles in a continuous-valued coordinate space. This is similar to the method used by Latombe [21, 98] and others to represent environments for mobile robot path-planning techniques.

In the case of [21, 98], most of the experiments are in fact simulations run exclusively within the computer's memory. Therefore, no real effort would have been expended to attempt to use sets of polygons to describe a real-world environment, such as a park or office building.

In other work in which real environments must be captured by the maps, one sees a trend toward selectivity and abstraction. The human map maker tends to capture on the map, for localization purposes, only objects that can be detected by the robot's sensors and, furthermore, only a subset of the features of real-world objects.

It should be immediately apparent that geometric maps can capably represent the physical locations of objects without referring to their texture, color, elasticity, or any other such secondary features that do not relate directly to position and space. In addition to this level of simplification, a mobile robot map can further reduce memory usage by capturing only aspects of object geometry that are immediately relevant to localization. For example, all objects may be approximated using very simple convex polygons, sacrificing map fidelity for the sake of computational speed.

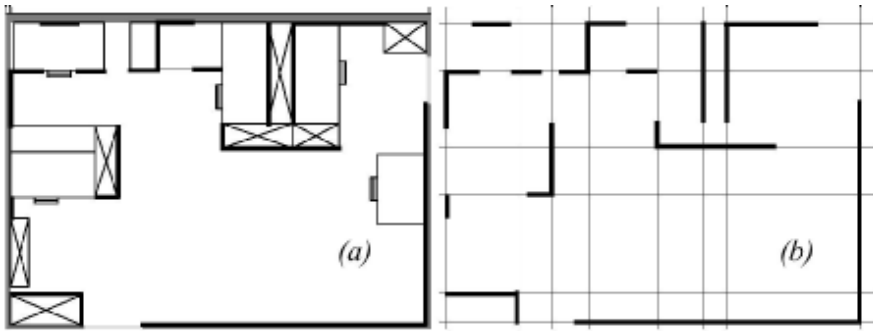


Figure 5.13

Example of a continuous-valued line representation of EPFL. (a) Real map. (b) Representation with a set of infinite lines.

One excellent example involves line extraction. Many indoor mobile robots rely upon laser rangefinding devices to recover distance readings to nearby objects. Such robots can automatically extract best-fit lines from the dense range data provided by thousands of points of laser strikes. Given such a line extraction sensor, an appropriate continuous mapping approach is to populate the map with a set of infinite lines. The continuous nature of the map guarantees that lines can be positioned at arbitrary positions in the plane and at arbitrary angles. The abstraction of real environmental objects such as walls and intersections captures only the information in the map representation that matches the type of information recovered by the mobile robot's rangefinding sensor.

Figure 5.13 shows a map of an indoor environment at EPFL using a continuous line representation. Note that the only environmental features captured by the map are straight lines, such as those found at corners and along walls. This represents not only a sampling of the real world of richer features but also a simplification, for an actual wall may have texture and relief that is not captured by the mapped line.

The impact of continuous map representations on position representation is primarily positive. In the case of single-hypothesis position representation, that position may be specified as any continuous-valued point in the coordinate space, and therefore extremely high accuracy is possible. In the case of multiple-hypothesis position representation, the continuous map enables two types of multiple position representation.

In one case, the possible robot position may be depicted as a geometric shape in the hyperplane, such that the robot is known to be within the bounds of that shape. This is shown in figure 5.29, in which the position of the robot is depicted by an oval bounding area.

Yet, the continuous representation does not disallow representation of position in the form of a discrete set of possible positions. For instance, in [62] the robot position belief state is captured by sampling nine continuous-valued positions from within a region near the robot's best-known position. This algorithm captures, within a continuous space, a discrete sampling of possible robot positions.

In summary, the key advantage of a continuous map representation is the potential for high accuracy and expressiveness with respect to the environmental configuration as well as the robot position within that environment. The danger of a continuous representation is that the map may be computationally costly. But this danger can be tempered by employing abstraction and capturing only the most relevant environmental features. Together with the use of the *closed-world assumption*, these techniques can enable a continuous-valued map to be no more costly, and sometimes even less costly, than a standard discrete representation.

5.5.2 Decomposition strategies

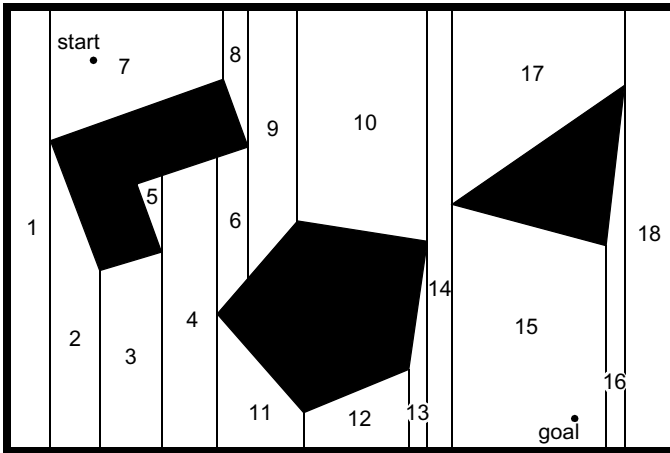
In the section above, we discussed one method of simplification, in which the continuous map representation contains a set of infinite lines that approximate real-world environmental lines based on a 2D slice of the world. Basically this transformation from the real world to the map representation is a filter that removes all nonstraight data and furthermore extends line segment data into infinite lines that require fewer parameters.

A more dramatic form of simplification is *abstraction*: a general decomposition and selection of environmental features. In this section, we explore decomposition as applied in its more extreme forms to the question of map representation.

Why would one radically decompose the real environment during the design of a map representation? The immediate disadvantage of decomposition and abstraction is the loss of fidelity between the map and the real world. Both qualitatively, in terms of overall structure, and quantitatively, in terms of geometric precision, a highly abstract map does not compare favorably to a high-fidelity map.

Despite this disadvantage, decomposition and abstraction may be useful if the abstraction can be planned carefully so as to capture the relevant, *useful* features of the world while discarding all other features. The advantage of this approach is that the map representation can potentially be minimized. Furthermore, if the decomposition is hierarchical, such as in a pyramid of recursive abstraction, then reasoning and planning with respect to the map representation may be computationally far superior to planning in a fully detailed world model.

A standard, lossless form of *opportunistic decomposition* is termed *exact cell decomposition*. This method, introduced by Latombe [21], achieves decomposition by selecting boundaries between discrete cells based on geometric criticality.

**Figure 5.14**

Example of exact cell decomposition.

Figure 5.14 depicts an exact decomposition of a planar workspace populated by polygonal obstacles. The map representation tessellates the space into areas of free space. The representation can be extremely compact because each such area is actually stored as a single node, resulting in a total of only eighteen nodes in this example.

The underlying assumption behind this decomposition is that the particular position of a robot within each area of free space does not matter. What matters is the robot's ability to traverse from each area of free space to the adjacent areas. Therefore, as with other representations we will see, the resulting graph captures the adjacency of map locales. If indeed the assumptions are valid and the robot does not care about its precise position within a single area, then this can be an effective representation that nonetheless captures the connectivity of the environment.

Such an exact decomposition is not always appropriate. Exact decomposition is a function of the particular environment obstacles and free space. If this information is expensive to collect or even unknown, then such an approach is not feasible.

An alternative is *fixed decomposition*, in which the world is tessellated, transforming the continuous real environment into a discrete approximation for the map. Such a transformation is demonstrated in figure 5.15, which depicts what happens to obstacle-filled and free areas during this transformation. The key disadvantage of this approach stems from its *inexact* nature. It is possible for narrow passageways to be lost during such a transformation, as shown in figure 5.15. Formally, this means that fixed decomposition is sound but not complete. Yet another approach is adaptive cell decomposition, as presented in figure 5.16.

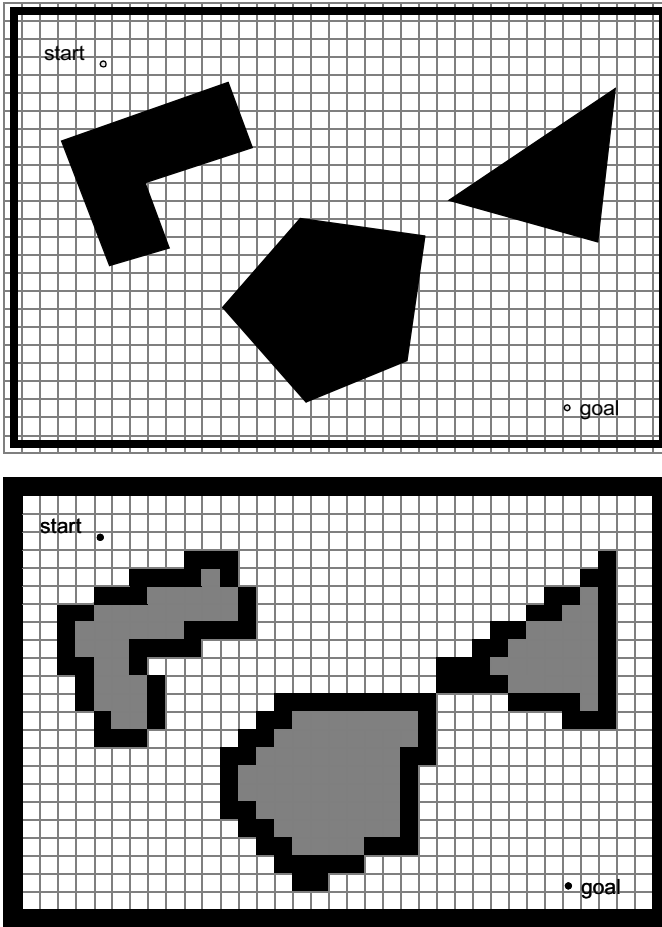


Figure 5.15
Fixed decomposition of the same space (narrow passage disappears).

The concept of fixed decomposition is extremely popular in mobile robotics; it is perhaps the single most common map representation technique currently utilized. One very popular version of fixed decomposition is known as the *occupancy grid* representation [112]. In an occupancy grid, the environment is represented by a discrete grid, where each cell is either filled (part of an obstacle) or empty (part of free space). This method is of particular value when a robot is equipped with range-based sensors because the range values

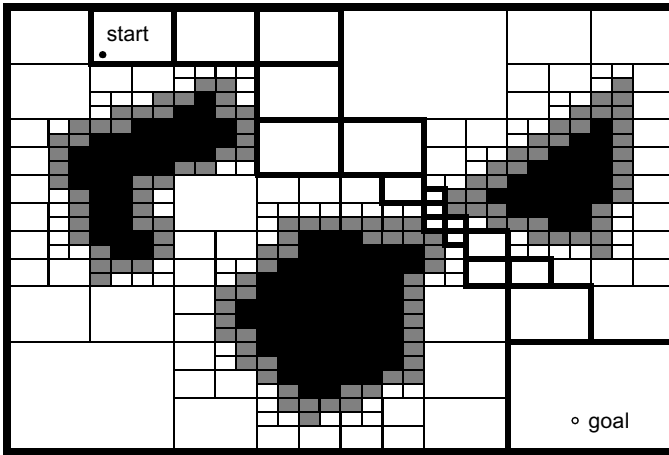


Figure 5.16

Example of adaptive (approximate variable-cell) decomposition of an environment [21]. The rectangle, bounding the free space, is decomposed into four identical rectangles. If the interior of a rectangle lies completely in free space or in the configuration space obstacle, it is not decomposed further. Otherwise, it is recursively decomposed into four rectangles until some predefined resolution is attained. The white cells lie outside the obstacles, the black inside, and the gray are part of both regions.

of each sensor, combined with the absolute position of the robot, can be used directly to update the filled or empty value of each cell.

In the occupancy grid, each cell may have a counter, whereby the value 0 indicates that the cell has not been “hit” by any ranging measurements and, therefore, it is likely free space. As the number of ranging strikes increases, the cell’s value is incremented and, above a certain threshold, the cell is deemed to be an obstacle. The values of cells are commonly discounted when a ranging strike travels *through* the cell, striking a further cell. By also discounting the values of cells over time, both hysteresis and the possibility of transient obstacles can be represented using this occupancy grid approach. Figure 5.17 depicts an occupancy grid representation in which the darkness of each cell is proportional to the value of its counter. One commercial robot that uses a standard occupancy grid for mapping and navigation is the Cye robot [163].

There remain two main disadvantages of the occupancy grid approach. First, the size of the map in robot memory grows with the size of the environment and if a small cell size is used, this size can quickly become untenable. This occupancy grid approach is not compatible with the *closed-world assumption*, which enabled continuous representations to have potentially very small memory requirements in large, sparse environments. In contrast, the

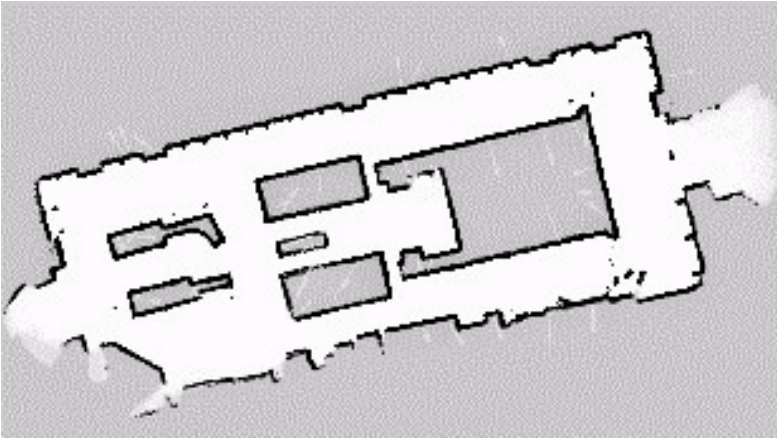


Figure 5.17

Example of an occupancy grid map representation (courtesy of S. Thrun [145]).

occupancy grid must have memory set aside for every cell in the matrix. Furthermore, any fixed decomposition method such as this imposes a geometric grid on the world *a priori*, regardless of the environmental details. This can be inappropriate in cases where geometry is not the most salient feature of the environment.

For these reasons, an alternative, called *topological* decomposition, has been the subject of some exploration in mobile robotics. Topological approaches avoid direct measurement of geometric environmental qualities, instead concentrating on characteristics of the environment that are most relevant to the robot for localization.

Formally, a topological representation is a graph that specifies two things: *nodes* and the *connectivity* between those nodes. Insofar as a topological representation is intended for the use of a mobile robot, nodes are used to denote areas in the world and arcs are used to denote adjacency of pairs of nodes. When an arc connects two nodes, then the robot can traverse from one node to the other without requiring traversal of any other intermediary node.

Adjacency is clearly at the heart of the topological approach, just as adjacency in a cell decomposition representation maps to geometric adjacency in the real world. However, the topological approach diverges in that the nodes are not of fixed size or even specifications of free space. Instead, nodes document an area based on any sensor discriminant such that the robot can recognize entry and exit of the node.

Figure 5.18 depicts a topological representation of a set of hallways and offices in an indoor environment. In this case, the robot is assumed to have an intersection detector, perhaps using sonar and vision to find intersections between halls and between halls and

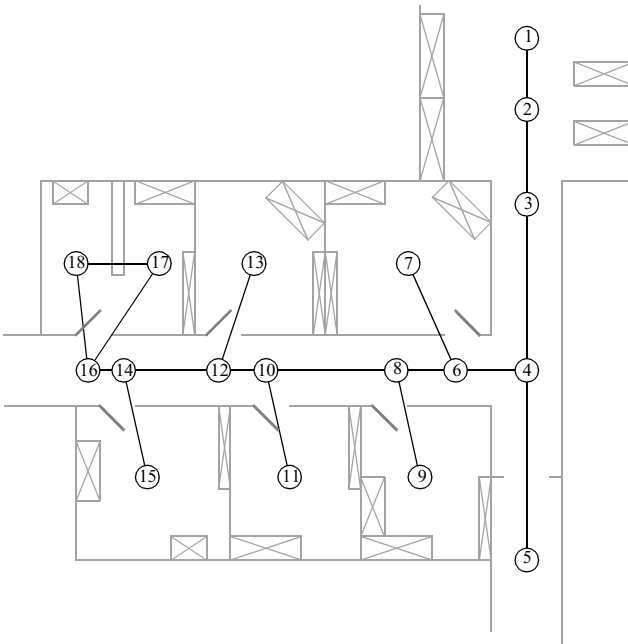


Figure 5.18
A topological representation of an indoor office area.

rooms. Note that nodes capture geometric space, and arcs in this representation simply represent connectivity.

Another example of topological representation is the work of Simhon and Dudek [134], in which the goal is to create a mobile robot that can capture the most interesting aspects of an area for human consumption. The nodes in their representation are visually striking locales rather than route intersections.

In order to navigate using a topological map robustly, a robot must satisfy two constraints. First, it must have a means for detecting its current position in terms of the nodes of the topological graph. Second, it must have a means for traveling between nodes using robot motion. The node sizes and particular dimensions must be optimized to match the sensory discrimination of the mobile robot hardware. This ability to “tune” the representation to the robot’s particular sensors can be an important advantage of the topological approach. However, as the map representation drifts further away from true geometry, the expressiveness of the representation for accurately and precisely describing a robot position is lost. Therein lies the compromise between the discrete cell-based map representations and the topological representations. Interestingly, the continuous map representation has

**Figure 5.19**

An artificial landmark used by Chips during autonomous docking.

the potential to be both compact like a topological representation and precise as with all direct geometric representations.

Yet, a chief motivation of the topological approach is that the environment may contain important nongeometric features – features that have no ranging relevance but are useful for localization. In chapter 4 we described such whole-image vision-based features.

In contrast to these whole-image feature extractors, often spatially localized landmarks are artificially placed in an environment to impose a particular visual-topological connectivity upon the environment. In effect, the artificial landmark can impose artificial structure. Examples of working systems operating with this landmark-based strategy have also demonstrated success. Latombe's landmark-based navigation research [99] has been implemented on real-world indoor mobile robots that employ paper landmarks attached to the ceiling as the locally observable features. Chips, the museum robot, is another robot that uses man-made landmarks to obviate the localization problem. In this case, a bright pink square serves as a landmark with dimensions and color signature that would be hard to accidentally reproduce in a museum environment [118]. One such museum landmark is shown in figure 5.19.

In summary, range is clearly not the only measurable and useful environmental value for a mobile robot. This is particularly true with the advent of color vision, as well as laser

rangefinding, which provides reflectance information in addition to range information. Choosing a map representation for a particular mobile robot requires, first, understanding the sensors available on the mobile robot, and, second, understanding the mobile robot's functional requirements (e.g., required goal precision and accuracy).

5.5.3 State of the art: current challenges in map representation

The sections above describe major design decisions in regard to map representation choices. There are, however, fundamental real-world features that mobile robot map representations do not yet represent well. These continue to be the subject of open research, and several such challenges are described below.

The real world is dynamic. As mobile robots come to inhabit the same spaces as humans, they will encounter moving people, cars, strollers, and the transient obstacles placed and moved by humans as they go about their activities. This is particularly true when one considers the home environment with which domestic robots will someday need to contend.

The map representations described above do not, in general, have explicit facilities for identifying and distinguishing between permanent obstacles (e.g., walls, doorways, etc.) and transient obstacles (e.g., humans, shipping packages, etc.). The current state of the art in terms of mobile robot sensors is partly to blame for this shortcoming. Although vision research is rapidly advancing, robust sensors that discriminate between moving animals and static structures *from a moving reference frame* are not yet available. Furthermore, estimating the motion vector of transient objects remains a research problem.

Usually, the assumption behind the above map representations is that all objects on the map are effectively static. Partial success can be achieved by discounting mapped objects over time. For example, occupancy grid techniques can be more robust to dynamic settings by introducing temporal discounting, effectively treating transient obstacles as noise. The more challenging process of map creation is particularly fragile to environmental dynamics; most mapping techniques generally require that the environment be free of moving objects during the mapping process. One exception to this limitation involves topological representations. Because precise geometry is not important, transient objects have little effect on the mapping or localization process, subject to the critical constraint that the transient objects must not change the topological connectivity of the environment. Still, neither the occupancy grid representation nor a topological approach is actively recognizing and representing transient objects as distinct from both sensor error and permanent map features.

As vision sensing provides more robust and more informative content regarding the transience and motion details of objects in the world, mobile roboticists will in time propose representations that make use of that information. A classic example involves occlusion by human crowds. Museum tour guide robots generally suffer from an extreme amount of occlusion. If the robot's sensing suite is located along the robot's body, then the robot is

effectively blind when a group of human visitors completely surround the robot. This is because its map contains only environmental features that are, at that point, fully hidden from the robot's sensors by the wall of people. In the best case, the robot should recognize its occlusion and make no effort to localize using these invalid sensor readings. In the worst case, the robot will localize with the fully occluded data, and will update its location incorrectly. A vision sensor that can discriminate the local conditions of the robot (e.g., we are surrounded by people) can help eliminate this error mode.

A second open challenge in mobile robot localization involves the traversal of open spaces. Existing localization techniques generally depend on local measures such as range, thereby demanding environments that are somewhat densely filled with objects that the sensors can detect and measure. Wide-open spaces such as parking lots, fields of grass, and indoor atriums such as those found in convention centers pose a difficulty for such systems because of their relative sparseness. Indeed, when populated with humans, the challenge is exacerbated because any mapped objects are almost certain to be occluded from view by the people.

Once again, more recent technologies provide some hope of overcoming these limitations. Both vision and state-of-the-art laser rangefinding devices offer outdoor performance with ranges of up to a hundred meters and more. Of course, GPS performs even better. Such long-range sensing may be required for robots to localize using distant features.

This trend teases out a hidden assumption underlying most topological map representations. Usually, topological representations make assumptions regarding spatial locality: a node contains objects and features that are themselves within that node. The process of map creation thus involves making nodes that are, in their own self-contained way, recognizable by virtue of the objects contained within the node. Therefore, in an indoor environment, each room can be a separate node, and this is reasonable because each room will have a layout and a set of belongings that are unique to that room.

However, consider the outdoor world of a wide-open park. Where should a single node end and the next node begin? The answer is unclear because objects that are far away from the current node, or position, can yield information for the localization process. For example, the hump of a hill at the horizon, the position of a river in the valley, and the trajectory of the sun all are nonlocal features that have great bearing on one's ability to infer current position. The spatial locality assumption is violated and, instead, replaced by a visibility criterion: the node or cell may need a mechanism for representing objects that are measurable and visible from that cell. Once again, as sensors improve and, in this case, as outdoor locomotion mechanisms improve, there will be greater urgency to solve problems associated with localization in wide-open settings, with and without GPS-type global localization sensors.

We end this section with one final open challenge that represents one of the fundamental academic research questions of robotics: sensor fusion. A variety of measurement types are

possible using off-the-shelf robot sensors, including heat, range, acoustic and light-based reflectivity, color, texture, friction, and so on. Sensor fusion is a research topic closely related to map representation. Just as a map must embody an environment in sufficient detail for a robot to perform localization and reasoning, sensor fusion demands a representation of the world that is sufficiently general and expressive that a variety of sensor types can have their data correlated appropriately, strengthening the resulting percepts well beyond that of any individual sensor's readings.

Perhaps the only general implementation of sensor fusion to date is that of neural network classifier. Using this technique, any number and any type of sensor values may be jointly combined in a network that will use whatever means necessary to optimize its classification accuracy. For the mobile robot that must use a human-readable internal map representation, no equally general sensor fusion scheme has yet been born. It is reasonable to expect that, when the sensor fusion problem is solved, integration of a large number of disparate sensor types may easily result in sufficient discriminatory power for robots to achieve real-world navigation, even in wide-open and dynamic circumstances such as a public square filled with people.

5.6 Probabilistic Map-Based Localization

5.6.1 Introduction

As stated earlier, multiple-hypothesis position representation is advantageous because the robot can explicitly track its own beliefs regarding its possible positions in the environment. Ideally, the robot's *belief state* will change, over time, as is consistent with its motor outputs and perceptual inputs. One geometric approach to multiple-hypothesis representation, mentioned earlier, involves identifying the possible positions of the robot by specifying a polygon in the environmental representation [98]. This method does not provide any indication of the relative chances between various possible robot positions.

Probabilistic techniques differ from this because they explicitly identify probabilities with the possible robot positions, and for this reason these methods have been the focus of recent research. In the following sections we present two classes of probabilistic localization. The first class, *Markov localization*, uses an explicitly specified probability distribution across all possible robot positions. The second method, *Kalman filter localization*, uses a Gaussian probability density representation of robot position and scan matching for localization. Unlike Markov localization, Kalman filter localization does not independently consider each possible pose in the robot's configuration space. Interestingly, the Kalman filter localization process results from the Markov localization axioms if the robot's position uncertainty is assumed to have a Gaussian form [3, pp. 43-44].

Before discussing each method in detail, we present the general robot localization problem and solution strategy. Consider a mobile robot moving in a known environment. As it

starts to move, say from a precisely known location, it can keep track of its motion using odometry. Due to odometry uncertainty, after some movement the robot will become very uncertain about its position (see section 5.2.4). To keep position uncertainty from growing unbounded, the robot must localize itself in relation to its environment map. To localize, the robot might use its on-board sensors (ultrasonic, range sensor, vision) to make observations of its environment. The information provided by the robot's odometry, plus the information provided by such exteroceptive observations, can be combined to enable the robot to localize as well as possible with respect to its map. The processes of updating based on proprioceptive sensor values and exteroceptive sensor values are often separated logically, leading to a general two-step process for robot position update.

Action update represents the application of some action model Act to the mobile robot's proprioceptive encoder measurements o_t and prior belief state s_{t-1} to yield a new belief state representing the robot's belief about its current position. Note that throughout this chapter we assume that the robot's proprioceptive encoder measurements are used as the best possible measure of its actions over time. If, for instance, a differential-drive robot had motors without encoders connected to its wheels and employed open-loop control, then instead of encoder measurements the robot's highly uncertain estimates of wheel spin would need to be incorporated. We ignore such cases and therefore have a simple formula:

$$s'_t = Act(o_t, s_{t-1}). \quad (5.16)$$

Perception update represents the application of some perception model See to the mobile robot's exteroceptive sensor inputs i_t and updated belief state s'_t to yield a refined belief state representing the robot's current position:

$$s_t = See(i_t, s'_t) \quad (5.17)$$

The perception model See and sometimes the action model Act are abstract functions of both the map and the robot's physical configuration (e.g., sensors and their positions, kinematics, etc.).

In general, the action update process contributes uncertainty to the robot's belief about position: encoders have error and therefore motion is somewhat nondeterministic. By contrast, perception update generally refines the belief state. Sensor measurements, when compared to the robot's environmental model, tend to provide clues regarding the robot's possible position.

In the case of Markov localization, the robot's belief state is usually represented as separate probability assignments for every possible robot pose in its map. The action update and perception update processes must update the probability of every cell in this case. Kalman filter localization represents the robot's belief state using a single, well-defined

Gaussian probability density function, and thus retains just a μ and σ parameterization of the robot's belief about position with respect to the map. Updating the parameters of the Gaussian distribution is all that is required. This fundamental difference in the representation of belief state leads to the following advantages and disadvantages of the two methods, as presented in [73]:

- Markov localization allows for localization starting from any unknown position and can thus recover from ambiguous situations because the robot can track multiple, completely disparate possible positions. However, to update the probability of all positions within the whole state space at any time requires a discrete representation of the space, such as a geometric grid or a topological graph (see section 5.5.2). The required memory and computational power can thus limit precision and map size.
- Kalman filter localization tracks the robot from an initially known position and is inherently both precise and efficient. In particular, Kalman filter localization can be used in continuous world representations. However, if the uncertainty of the robot becomes too large (e.g., due to a robot collision with an object) and thus not truly unimodal, the Kalman filter can fail to capture the multitude of possible robot positions and can become irrevocably lost.

In recent research projects improvements are achieved or proposed by either only updating the state space of interest within the Markov approach [49] or by tracking multiple hypotheses with Kalman filters [35], or by combining both methods to create a hybrid localization system [73, 147]. In the next two sections we will each approach in detail.

5.6.2 Markov localization

Markov localization tracks the robot's belief state using an arbitrary probability density function to represent the robot's position (see also [50, 88, 116, 119]). In practice, all known Markov localization systems implement this generic belief representation by first tessellating the robot configuration space into a finite, discrete number of possible robot poses in the map. In actual applications, the number of possible poses can range from several hundred to millions of positions.

Given such a generic conception of robot position, a powerful update mechanism is required that can compute the belief state that results when new information (e.g., encoder values and sensor values) is incorporated into a prior belief state with arbitrary probability density. The solution is born out of probability theory, and so the next section describes the foundations of probability theory that apply to this problem, notably the Bayes formula. Then, two subsequent sections provide case studies, one robot implementing a simple feature-driven topological representation of the environment [88, 116, 119] and the other using a geometric grid-based map [49, 50].

5.6.2.1 Introduction: applying probability theory to robot localization

Given a discrete representation of robot positions, in order to express a belief state we wish to assign to each possible robot position a probability that the robot is indeed at that position. From probability theory we use the term $p(A)$ to denote the probability that A is true. This is also called the *prior probability* of A because it measures the probability that A is true independent of any additional knowledge we may have. For example we can use $p(r_t = l)$ to denote the prior probability that the robot r is at position l at time t .

In practice, we wish to compute the probability of each individual robot position given the encoder and sensor evidence the robot has collected. In probability theory, we use the term $p(A|B)$ to denote the *conditional* probability of A given that we know B . For example, we use $p(r_t = l|i_t)$ to denote the probability that the robot is at position l given that the robot's sensor inputs i .

The question is, how can a term such as $p(r_t = l|i_t)$ be simplified to its constituent parts so that it can be computed? The answer lies in the product rule, which states

$$p(A \wedge B) = p(A|B)p(B) \quad (5.18)$$

Equation (5.18) is intuitively straightforward, as the probability of both A and B being true is being related to B being true *and* the other being conditionally true. But you should be able to convince yourself that the alternate equation is equally correct:

$$p(A \wedge B) = p(B|A)p(A) \quad (5.19)$$

Using equations (5.18) and (5.19) together, we can derive the Bayes formula for computing $p(A|B)$:

$$p(A|B) = \frac{p(B|A)p(A)}{p(B)} \quad (5.20)$$

We use the Bayes rule to compute the robot's new belief state as a function of its sensory inputs and its former belief state. But to do this properly, we must recall the basic goal of the Markov localization approach: a discrete set of possible robot positions L are represented. The belief state of the robot must assign a probability $p(r_t = l)$ for each location l in L .

The *See* function described in equation (5.17) expresses a mapping from a belief state and sensor input to a refined belief state. To do this, we must update the probability associated with each position l in L , and we can do this by directly applying the Bayes formula to every such l . In denoting this, we will stop representing the temporal index t for simplicity and will further use $p(l)$ to mean $p(r = l)$:

$$p(l|i) = \frac{p(i|l)p(l)}{p(i)} \quad (5.21)$$

The value of $p(i|l)$ is key to equation (5.21), and this probability of a sensor input at each robot position must be computed using some model. An obvious strategy would be to consult the robot's map, identifying the probability of particular sensor readings with each possible map position, given knowledge about the robot's sensor geometry and the mapped environment. The value of $p(l)$ is easy to recover in this case. It is simply the probability $p(r = l)$ associated with the belief state before the perceptual update process. Finally, note that the denominator $p(i)$ does not depend upon l ; that is, as we apply equation (5.21) to all positions l in L , the denominator never varies. Because it is effectively constant, in practice this denominator is usually dropped and, at the end of the perception update step, all probabilities in the belief state are re-normalized to sum at 1.0.

Now consider the *Act* function of equation (5.16). *Act* maps a former belief state and encoder measurement (i.e., robot action) to a new belief state. In order to compute the probability of position l in the new belief state, one must integrate over all the possible ways in which the robot may have reached l according to the potential positions expressed in the former belief state. This is subtle but fundamentally important. The same location l can be reached from multiple source locations with the same encoder measurement o because the encoder measurement is uncertain. Temporal indices are required in this update equation:

$$p(l_i|o_i) = \int p(l_i|l'_{t-1}, o_i) p(l'_{t-1}) dl'_{t-1} \quad (5.22)$$

Thus, the total probability for a specific position l is built up from the individual contributions from every location l' in the former belief state given encoder measurement o .

Equations (5.21) and (5.22) form the basis of Markov localization, and they incorporate the *Markov assumption*. Formally, this means that their output is a function only of the robot's previous state and its most recent actions (odometry) and perception. In a general, non-Markovian situation, the state of a system depends upon all of its history. After all, the values of a robot's sensors at time t do not really depend only on its position at time t . They depend to some degree on the trajectory of the robot over time; indeed, on the entire history of the robot. For example, the robot could have experienced a serious collision recently that has biased the sensor's behavior. By the same token, the position of the robot at time t does not really depend only on its position at time $t - 1$ and its odometric measurements. Due to its history of motion, one wheel may have worn more than the other, causing a left-turning bias over time that affects its current position.

So the Markov assumption is, of course, not a valid assumption. However the Markov assumption greatly simplifies tracking, reasoning, and planning and so it is an approximation that continues to be extremely popular in mobile robotics.



Figure 5.20
Dervish exploring its environment.

5.6.2.2 Case study 1: Markov localization using a topological map

A straightforward application of Markov localization is possible when the robot's environment representation already provides an appropriate decomposition. This is the case when the environmental representation is purely topological.

Consider a contest in which each robot is to receive a topological description of the environment. The description would include only the connectivity of hallways and rooms, with no mention of geometric distance. In addition, this supplied *map* would be imperfect, containing several false arcs (e.g., a closed door). Such was the case for the 1994 American Association for Artificial Intelligence (AAAI) National Robot Contest, at which each robot's mission was to use the supplied map and its own sensors to navigate from a chosen starting position to a target room.

Dervish, the winner of this contest, employed probabilistic Markov localization and used a multiple-hypothesis belief state over a topological environmental representation. We now describe Dervish as an example of a robot with a discrete, topological representation and a probabilistic localization algorithm.

Dervish, shown in figure 5.20, includes a sonar arrangement custom-designed for the 1994 AAAI National Robot Contest. The environment in this contest consisted of a recti-

linear indoor office space filled with real office furniture as obstacles. Traditional sonars were arranged radially around the robot in a ring. Robots with such sensor configurations are subject to both tripping over short objects below the ring and to decapitation by tall objects (such as ledges, shelves, and tables) that are above the ring.

Dervish's answer to this challenge was to arrange one pair of sonars diagonally upward to detect ledges and other overhangs. In addition, the diagonal sonar pair also proved to ably detect tables, enabling the robot to avoid wandering underneath tall tables. The remaining sonars were clustered in sets of sonars, such that each individual transducer in the set would be at a slightly varied angle to minimize specularly. Finally, two sonars near the robot's base were positioned to detect low obstacles, such as paper cups, on the floor.

We have already noted that the representation provided by the contest organizers was purely topological, noting the connectivity of hallways and rooms in the office environment. Thus, it would be appropriate to design Dervish's perceptual system to detect matching perceptual events: the detection and passage of connections between hallways and offices.

This *abstract* perceptual system was implemented by viewing the trajectory of sonar strikes to the left and right sides of Dervish over time. Interestingly, this perceptual system would use time alone and no concept of encoder value to trigger perceptual events. Thus, for instance, when the robot detects a 7 to 17 cm indentation in the width of the hallway for more than 1 second continuously, a *closed door* sensory event is triggered. If the sonar strikes jump well beyond 17 cm for more than 1 second, an *open door* sensory event triggers.

To reduce coherent reflection sensor noise (see section 4.1.6) associated with Dervish's sonars, the robot would track its angle relative to the hallway centerline and completely suppress sensor events when its angle to the hallway exceeded 9 degrees. Interestingly, this would result in a conservative perceptual system that frequently misses features, particularly when the hallway is crowded with obstacles that Dervish must negotiate. Once again, the conservative nature of the perceptual system, and in particular its tendency to issue false negatives, would point to a probabilistic solution to the localization problem so that a complete trajectory of perceptual inputs could be considered.

Dervish's environmental representation was a discrete topological map, identical in abstraction and information to the map provided by the contest organizers. Figure 5.21 depicts a geometric representation of a typical office environment overlaid with the topological map for the same office environment. Recall that for a topological representation the key decision involves assignment of nodes and connectivity between nodes (see section 5.5.2). As shown on the left in figure 5.21 Dervish uses a topology in which node boundaries are marked primarily by doorways (and hallways and foyers). The topological graph shown on the right depicts the information captured in the example shown.

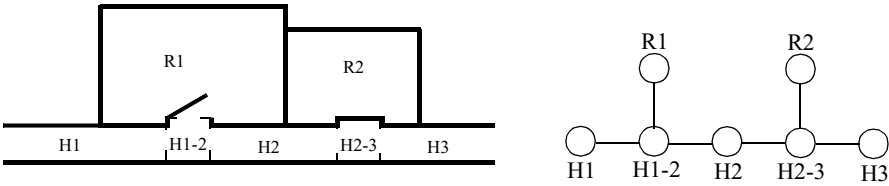


Figure 5.21
A geometric office environment (left) and its topological analog (right).

Note that in this particular topological model arcs are zero-length while nodes have spatial expansiveness and together cover the entire space. This particular topological representation is particularly apt for Dervish given its task of navigating through hallways into a specific room and its perceptual capability of recognizing discontinuities in hallway walls.

In order to represent a specific belief state, Dervish associated with each topological node n a probability that the robot is at a physical position within the boundaries of n : $p(r_i = n)$. As will become clear below, the probabilistic update used by Dervish was approximate, therefore technically one should refer to the resulting values as *likelihoods* rather than probabilities.

Table 5.1
Dervish's certainty matrix.

	Wall	Closed door	Open door	Open hallway	Foyer
Nothing detected	0.70	0.40	0.05	0.001	0.30
Closed door detected	0.30	0.60	0	0	0.05
Open door detected	0	0	0.90	0.10	0.15
Open hallway detected	0	0	0.001	0.90	0.50

The perception update process for Dervish functions precisely as in equation (5.21). Perceptual events are generated asynchronously, each time the feature extractor is able to recognize a large scale feature (e.g., doorway, intersection) based on recent ultrasonic values. Each perceptual event consists of a percept-pair (a feature on one side of the robot or two features on both sides).

Given a specific percept-pair i , equation (5.21) enables the likelihood of each possible position n to be updated using the formula:

$$p(n|i) = p(i|n)p(n) \quad (5.23)$$

The value of $p(n)$ is already available from the current belief state of Dervish, and so the challenge lies in computing $p(i|n)$. The key simplification for Dervish is based upon the realization that, because the feature extraction system only extracts four total features and because a node contains (on a single side) one of five total features, every possible combination of node type and extracted feature can be represented in a 4 x 5 table.

Dervish's *certainty matrix* (shown in table 5.1) is just this lookup table. Dervish makes the simplifying assumption that the performance of the feature detector (i.e., the probability that it is correct) is only a function of the feature extracted and the actual feature in the node. With this assumption in hand, we can populate the *certainty matrix* with confidence estimates for each possible pairing of perception and node type. For each of the five world features that the robot can encounter (wall, closed door, open door, open hallway-and foyer) this matrix assigns a likelihood for each of the three one-sided percepts that the sensory system can issue. In addition, this matrix assigns a likelihood that the sensory system will fail to issue a perceptual event altogether (*nothing detected*).

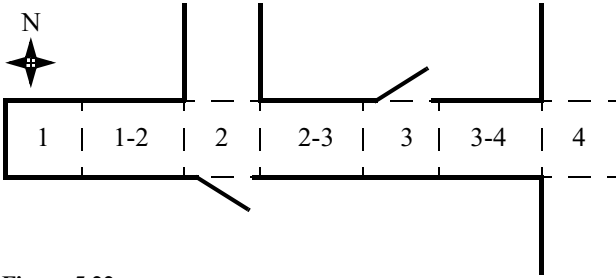
For example, using the specific values in table 5.1, if Dervish is next to an open hallway, the likelihood of mistakenly recognizing it as an open door is 0.10. This means that for any node n that is of type *open hallway* and for the sensor value $i = \text{open door}$, $p(i|n) = 0.10$. Together with a specific topological map, the certainty matrix enables straightforward computation of $p(i|n)$ during the perception update process.

For Dervish's particular sensory suite and for any specific environment it intends to navigate, humans generate a specific certainty matrix that loosely represents its perceptual confidence, along with a global measure for the probability that any given door will be closed versus opened in the real world.

Recall that Dervish has no encoders and that perceptual events are triggered asynchronously by the feature extraction processes. Therefore, Dervish has no action update step as depicted by equation (5.22). When the robot does detect a perceptual event, multiple perception update steps will need to be performed to update the likelihood of every possible robot position given Dervish's former belief state. This is because there is a chance that the robot has traveled *multiple* topological nodes since its previous perceptual event (i.e., false-negative errors). Formally, the perception update formula for Dervish is in reality a combination of the general form of action update and perception update. The likelihood of position n given perceptual event i is calculated as in equation (5.22):

$$p(n_t|i_t) = \int p(n_t|n'_{t-i}, i_t) p(n'_{t-i}) dn'_{t-i} \quad (5.24)$$

The value of $p(n'_{t-i})$ denotes the likelihood of Dervish being at position n' as represented by Dervish's former belief state. The temporal subscript $t-i$ is used in lieu of $t-1$

**Figure 5.22**

A realistic indoor topological environment.

because for each possible position n' the discrete topological distance from n' to n can vary depending on the specific topological map. The calculation of $p(n_t | n'_{t-i}, i_t)$ is performed by multiplying the probability of generating perceptual event i at position n by the probability of having failed to generate perceptual events at all nodes between n' and n :

$$p(n_t | n'_{t-i}, i_t) = p(i_t, n_t) \cdot p(\emptyset, n_{t-1}) \cdot p(\emptyset, n_{t-2}) \cdot \dots \cdot p(\emptyset, n_{t-i+1}) \quad (5.25)$$

For example (figure 5.22), suppose that the robot has only two nonzero nodes in its belief state, $\{1-2, 2-3\}$, with likelihoods associated with each possible position: $p(1-2) = 1.0$ and $p(2-3) = 0.2$. For simplicity assume the robot is facing east with certainty. Note that the likelihoods for nodes 1-2 and 2-3 do not sum to 1.0. These values are not formal probabilities, and so computational effort is minimized in Dervish by avoiding normalization altogether. Now suppose that a perceptual event is generated: the robot detects an open hallway on its left and an open door on its right simultaneously.

State 2-3 will progress potentially to states 3, 3-4, and 4. But states 3 and 3-4 can be eliminated because the likelihood of detecting an open door when there is only wall is zero. The likelihood of reaching state 4 is the product of the initial likelihood for state 2-3, 0.2, the likelihood of not detecting anything at node 3, (a), and the likelihood of detecting a hallway on the left and a door on the right at node 4, (b). Note that we assume the likelihood of detecting nothing at node 3-4 is 1.0 (a simplifying approximation).

(a) occurs only if Dervish fails to detect the door on its left at node 3 (either closed or open), $[0.6 \cdot 0.4 + (1 - 0.6) \cdot 0.05]$, and correctly detects nothing on its right, 0.7.

(b) occurs if Dervish correctly identifies the open hallway on its left at node 4, 0.90, and mistakes the right hallway for an open door, 0.10.

The final formula, $0.2 \cdot [0.6 \cdot 0.4 + 0.4 \cdot 0.05] \cdot 0.7 \cdot [0.9 \cdot 0.1]$, yields a likelihood of 0.003 for state 4. This is a partial result for $p(4)$ following from the prior belief state node 2-3.

Turning to the other node in Dervish's prior belief state, 1-2 will potentially progress to states 2, 2-3, 3, 3-4, and 4. Again, states 2-3, 3, and 3-4 can all be eliminated since the likelihood of detecting an open door when a wall is present is zero. The likelihood of state 2 is the product of the prior likelihood for state 1-2, (1.0), the likelihood of detecting the door on the right as an open door, $[0.6 \cdot 0 + 0.4 \cdot 0.9]$, and the likelihood of correctly detecting an open hallway to the left, 0.9. The likelihood for being at state 2 is then $1.0 \cdot 0.4 \cdot 0.9 \cdot 0.9 = 0.3$. In addition, 1-2 progresses to state 4 with a certainty factor of $4.3 \cdot 10^{-6}$, which is added to the certainty factor above to bring the total for state 4 to 0.00328. Dervish would therefore track the new belief state to be $\{2, 4\}$, assigning a very high likelihood to position 2 and a low likelihood to position 4.

Empirically, Dervish's map representation and localization system have proved to be sufficient for navigation of four indoor office environments: the artificial office environment created explicitly for the 1994 National Conference on Artificial Intelligence; and the psychology, history, and computer science departments at Stanford University. All of these experiments were run while providing Dervish with no notion of the distance between adjacent nodes in its topological map. It is a demonstration of the power of probabilistic localization that, in spite of the tremendous lack of action and encoder information, the robot is able to navigate several real-world office buildings successfully.

One open question remains with respect to Dervish's localization system. Dervish was not just a localizer but also a navigator. As with all multiple hypothesis systems, one must ask the question, how does the robot decide how to move, given that it has multiple possible robot positions in its representation? The technique employed by Dervish is a common technique in the mobile robotics field: plan the robot's actions by assuming that the robot's actual position is its most likely node in the belief state. Generally, the most likely position is a good measure of the robot's actual world position. However, this technique has shortcomings when the highest and second highest most likely positions have similar values. In the case of Dervish, it nonetheless goes with the highest-likelihood position at all times, save at one critical juncture. The robot's goal is to enter a target room and remain there. Therefore, from the point of view of its goal, it is critical that Dervish finish navigating only when the robot has strong confidence in being at the correct final location. In this particular case, Dervish's execution module refuses to enter a room if the gap between the most likely position and the second likeliest position is below a preset threshold. In such a case, Dervish will actively plan a path that causes it to move further down the hallway in an attempt to collect more sensor data and thereby increase the relative likelihood of one position in the multiple-hypothesis belief state.

Although computationally unattractive, one can go further, imagining a planning system for robots such as Dervish for which one specifies a *goal belief state* rather than a goal position. The robot can then reason and plan in order to achieve a goal confidence level, thus explicitly taking into account not only robot position but also the measured likelihood of

each position. An example of just such a procedure is the sensory uncertainty field of Latombe [141], in which the robot must find a trajectory that reaches its goal while maximizing its localization confidence on-line.

5.6.2.3 Case study 2: Markov localization using a grid map

The major weakness of a purely topological decomposition of the environment is the resolution limitation imposed by such a granular representation. The position of the robot is usually limited to the resolution of a single node in such cases, and this may be undesirable for certain applications.

In this case study, we examine the work of Burgard and colleagues [49, 50] in which far more precise navigation is made possible using a grid-based representation while still employing the Markov localization technique.

The robot used by this research, Rhino, is an RWI B24 robot with twenty-four sonars and two Sick laser rangefinders. Clearly, at the sensory level this robot accumulates greater and more accurate range data than is possible with the handful of sonar sensors mounted on Dervish. In order to make maximal use of these fine-grained sensory data, Rhino uses a 2D geometric environmental representation of free and occupied space. This metric map is tessellated regularly into a *fixed decomposition* grid with each cell occupying 4 to 64 cm in various instantiations.

Like Dervish, Rhino uses multiple-hypothesis belief representation. In line with the far improved resolution of the environmental representation, the belief state representation of Rhino consists of a $15 \times 15 \times 15$ 3D array representing the probability of 15^3 possible robot positions (see figure 5.23). The resolution of the array is $15 \text{ cm} \times 15 \text{ cm} \times 1^\circ$. Note that unlike Dervish, which assumes its orientation is approximate and known, Rhino explicitly represents fine-grained alternative orientations, and so its belief state formally represents three degrees of freedom. As we have stated before, the resolution of the belief state representation must match the environmental representation in order for the overall system to function well.

Whereas Dervish made use of only perceptual events, ignoring encoder inputs and therefore metric distance altogether, Rhino uses the complete Markov probabilistic localization approach summarized in section 5.6.2.1, including both an explicit action update phase and a perception update phase at every cycle.

The discrete Markov chain version of action update is performed because of the tessellated representation of position. Given encoder measurements o at time t , each updated position probability in the belief state is expressed as a sum over previous possible positions and the motion model:

$$P(l_t | o_t) = \sum_l P(l_t | l_{t-1}, o_t) \cdot p(l_{t-1}) \quad (5.26)$$

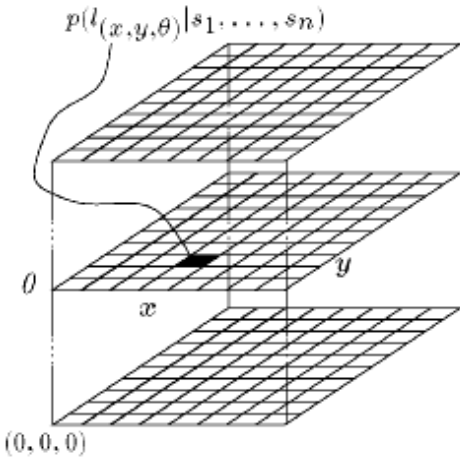


Figure 5.23

The belief state representation 3D array used by Rhino (courtesy of W. Burgard and S. Thrun).

Note that equation (5.26) is simply a discrete version of equation (5.22). The specific motion model used by Rhino represents the result of motion as a Gaussian that is bounded (i.e., the tails of the distribution are finite). Rhino's kinematic configuration is a three-wheel synchro-drive rather than a differential-drive robot. Nevertheless, the error ellipses depicted in figures 5.4 and 5.5 are similar to the Gaussian bounds that result from Rhino's motion model.

The perception model follows the Bayes formula precisely, as in equation (5.21). Given a range perception i the probability of the robot being at each location l is updated as follows:

$$p(l|i) = \frac{p(i|l)p(l)}{p(i)} \quad (5.27)$$

Note that a denominator is used by Rhino, although the denominator is constant for varying values of l . This denominator acts as a normalizer to ensure that the probability measures in the belief state continue to sum to 1.

The critical challenge is, of course, the calculation of $p(i|l)$. In the case of Dervish, the number of possible values for i and l were so small that a simple table could suffice. However, with the fine-grained metric representation of Rhino, the number of possible sensor readings and environmental geometric contexts is extremely large. Thus, Rhino computes

$p(i|l)$ directly using a model of the robot's sensor behavior, its position l , and the local environmental metric map around l .

The sensor model must calculate the probability of a specific perceptual measurement given that its likelihood is justified by known errors of the sonar or laser rangefinder sensors. Three key assumptions are used to construct this sensor model:

1. If an object in the metric map is detected by a range sensor, the measurement error can be described with a distribution that has a mean at the correct reading.
2. There should always be a nonzero chance that a range sensor will read any measurement value, even if this measurement disagrees sharply with the environmental geometry.
3. In contrast to the generic error described in (2), there is a specific failure mode in ranging sensors whereby the signal is absorbed or coherently reflected, causing the sensor's range measurement to be maximal. Therefore, there is a local peak in the probability density distribution at the maximal reading of a range sensor.

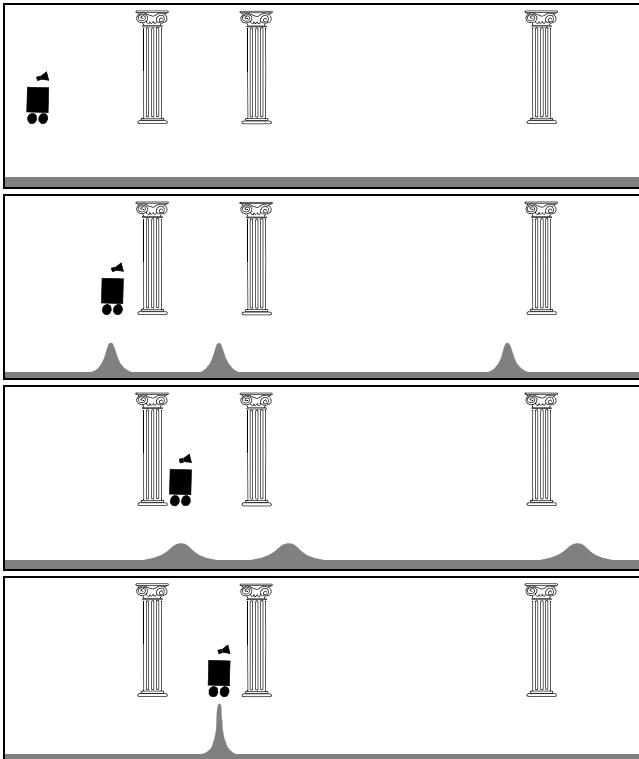
By validating these assumptions using empirical sonar trials in multiple environments, the research group has delivered to Rhino a conservative and powerful sensor model for its particular sensors.

Figure 5.24 provides a simple 1D example of the grid-based Markov localization algorithm. The robot begins with a flat probability density function for its possible location. In other words, it initially has no bias regarding position. As the robot encounters first one door and then a second door, the probability density function over possible positions becomes first multimodal and finally unimodal and sharply defined. The ability of a Markov localization system to *localize* the robot from an initially lost belief state is its key distinguishing feature.

The resulting robot localization system has been part of a navigation system that has demonstrated great success both at the University of Bonn (Germany) and at a public museum in Bonn. This is a challenging application because of the dynamic nature of the environment, as the robot's sensors are frequently subject to occlusion due to humans gathering around the robot. Rhino's ability to function well in this setting is a demonstration of the power of the Markov localization approach.

Reducing computational complexity: randomized sampling. A great many steps are taken in real-world implementations such as Rhino in order to effect computational gains. These are valuable because, with an exact cell decomposition representation and use of raw sensor values rather than abstraction to features, such a robot has a massive computational effort associated with each perceptual update.

One class of techniques deserves mention because it can significantly reduce the computational overhead of techniques that employ fixed-cell decomposition representations.

**Figure 5.24**

Improving belief state by moving.

The basic idea, which we call *randomized sampling*, is known alternatively as particle filter algorithms, condensation algorithms, and Monte Carlo algorithms [68, 144].

Irrespective of the specific technique, the basic algorithm is the same in all these cases. Instead of representing *every* possible robot position by representing the complete and correct belief state, an approximate belief state is constructed by representing only a *subset* of the complete set of possible locations that should be considered.

For example, consider a robot with a complete belief state of 10,000 possible locations at time t . Instead of tracking and updating all 10,000 possible locations based on a new sensor measurement, the robot can select only 10% of the stored locations and update only those locations. By weighting this sampling process with the probability values of the locations, one can bias the system to generate more samples at local peaks in the probability

density function. So the resulting 1000 locations will be concentrated primarily at the highest probability locations. This biasing is desirable, but only to a point.

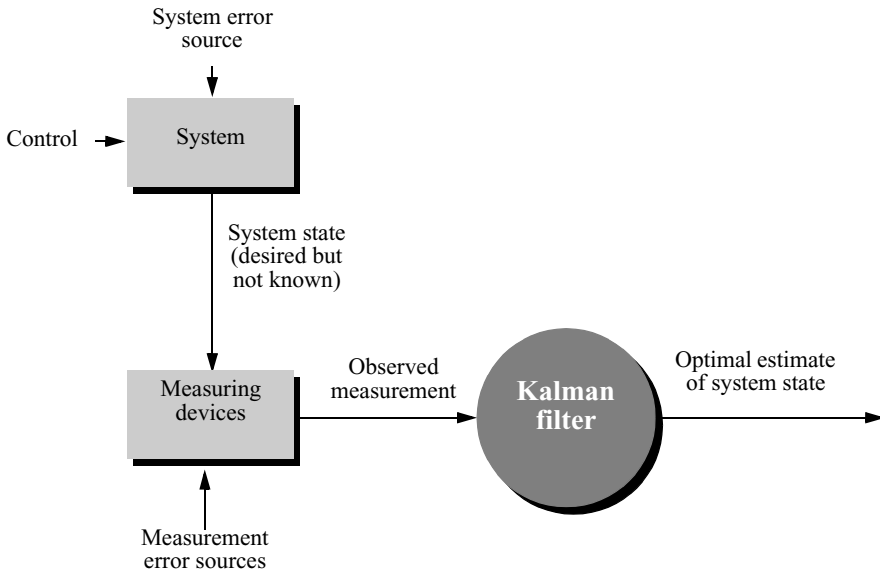
We also wish to ensure that *some* less likely locations are tracked, as otherwise, if the robot does indeed receive unlikely sensor measurements, it will fail to localize. This *randomization* of the sampling process can be performed by adding additional samples from a flat distribution, for example. Further enhancements of these randomized methods enable the number of statistical samples to be varied on the fly, based, for instance, on the ongoing localization confidence of the system. This further reduces the number of samples required on average while guaranteeing that a large number of samples will be used when necessary [68].

These sampling techniques have resulted in robots that function indistinguishably as compared to their full belief state set ancestors, yet use computationally a fraction of the resources. Of course, such sampling has a penalty: completeness. The probabilistically complete nature of Markov localization is violated by these sampling approaches because the robot is failing to update *all* the nonzero probability locations, and thus there is a danger that the robot, due to an unlikely but correct sensor reading, could become truly lost. Of course, recovery from a lost state is feasible just as with all Markov localization techniques.

5.6.3 Kalman filter localization

The Markov localization model can represent any probability density function over robot position. This approach is very general but, due to its generality, inefficient. Consider instead the key demands on a robot localization system. One can argue that it is not the exact replication of a probability density curve but the *sensor fusion* problem that is key to robust localization. Robots usually include a large number of heterogeneous sensors, each providing clues as to robot position and, critically, each suffering from its own failure modes. Optimal localization should take into account the information provided by all of these sensors. In this section we describe a powerful technique for achieving this sensor fusion, called the Kalman filter. This mechanism is in fact more efficient than Markov localization because of key simplifications when representing the probability density function of the robot's belief state and even its individual sensor readings, as described below. But the benefit of this simplification is a resulting *optimal recursive data-processing algorithm*. It incorporates all information, regardless of precision, to estimate the current value of the variable of interest (i.e., the robot's position). A general introduction to Kalman filters can be found in [106] and a more detailed treatment is presented in [3].

Figure 5.25 depicts the general scheme of Kalman filter estimation, where a system has a control signal and system error sources as inputs. A measuring device enables measuring some system states with errors. The Kalman filter is a mathematical mechanism for producing an optimal estimate of the system state based on the knowledge of the *system* and the *measuring device*, the description of the system noise and measurement errors and the

**Figure 5.25**

Typical Kalman filter application [106].

uncertainty in the dynamics models. Thus the Kalman filter *fuses* sensor signals and system knowledge in an optimal way. Optimality depends on the criteria chosen to evaluate the performance and on the assumptions. Within the Kalman filter theory the system is assumed to be *linear* and *white* with *Gaussian* noise. As we have discussed earlier, the assumption of Gaussian error is invalid for our mobile robot applications but, nevertheless, the results are extremely useful. In other engineering disciplines, the Gaussian error assumption has in some cases been shown to be quite accurate [106].

We begin with a section that introduces Kalman filter theory, then we present an application of that theory to the problem of mobile robot localization (5.6.3.2). Finally, section 5.6.3.2 presents a case study of a mobile robot that navigates indoor spaces by virtue of Kalman filter localization.

5.6.3.1 Introduction to Kalman filter theory

The basic Kalman filter method allows multiple measurements to be incorporated optimally into a single estimate of state. In demonstrating this, first we make the simplifying assumption that the state does not change (e.g., the robot does not move) between the acquisition of the first and second measurement. After presenting this static case, we can introduce *dynamic prediction* readily.

Static estimation. Suppose that our robot has two sensors, an ultrasonic range sensor and a laser rangefinding sensor. The laser rangefinder provides far richer and more accurate data for localization, but it will suffer from failure modes that differ from that of the sonar ranger. For instance, a glass wall will be transparent to the laser but, when measured head-on, the sonar will provide an accurate reading. Thus we wish to combine the information provided by the two sensors, recognizing that such sensor fusion, when done in a principled way, can only result in information gain.

The Kalman filter enables such fusion extremely efficiently, as long as we are willing to approximate the error characteristics of these sensors with unimodal, zero-mean, Gaussian noise. Specifically, assume we have taken two measurements, one with the sonar sensor at time k and one with the laser rangefinder at time $k + 1$. Based on each measurement individually we can estimate the robot's position. Such an estimate derived from the sonar is q_1 and the estimate of position based on the laser is q_2 . As a simplified way of characterizing the error associated with each of these estimates, we presume a (unimodal) Gaussian probability density curve and thereby associate one variance with each measurement: σ_1^2 and σ_2^2 . The two dashed probability densities in figure 5.26 depict two such measurements. In summary, this yields two robot position estimates:

$$\hat{q}_1 = q_1 \text{ with variance } \sigma_1^2 \quad (5.28)$$

$$\hat{q}_2 = q_2 \text{ with variance } \sigma_2^2. \quad (5.29)$$

The question is, how do we *fuse* (combine) these data to get the best estimate $\hat{q}$ for the robot position? We are assuming that there was no robot motion between time k and time $k + 1$, and therefore we can directly apply the same weighted least-squares technique of equation (5.26) in section 4.3.1.1. Thus we write

$$S = \sum_{i=1}^n w_i (\hat{q} - q_i)^2 \quad (5.30)$$

with w_i being the weight of measurement i . To find the minimum error we set the derivative of S equal to zero.

$$\frac{\partial S}{\partial \hat{q}} = \frac{\partial}{\partial \hat{q}} \sum_{i=1}^n w_i (\hat{q} - q_i)^2 = 2 \sum_{i=1}^n w_i (\hat{q} - q_i) = 0 \quad (5.31)$$

$$\sum_{i=1}^n w_i \hat{q} - \sum_{i=1}^n w_i q_i = 0 \quad (5.32)$$

$$\hat{q} = \frac{\sum_{i=1}^n w_i q_i}{\sum_{i=1}^n w_i} \quad (5.33)$$

If we take as the weight w_i

$$w_i = \frac{1}{\sigma_i^2} \quad (5.34)$$

then the value of $\hat{q}$ in terms of two measurements can be defined as follows:

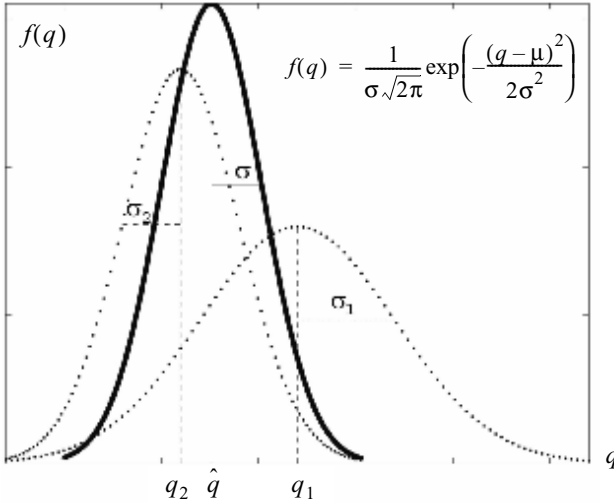
$$\hat{q} = \frac{\frac{1}{\sigma_1^2} q_1 + \frac{1}{\sigma_2^2} q_2}{\frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}} = \frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2} q_1 + \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2} q_2 \quad (5.35)$$

$$\frac{1}{\sigma^2} = \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2} = \frac{\sigma_2^2 + \sigma_1^2}{\sigma_1^2 \sigma_2^2} \quad ; \quad \sigma^2 = \frac{\sigma_1^2 \sigma_2^2}{\sigma_2^2 + \sigma_1^2} \quad (5.36)$$

Note that from equation (5.36) we can see that the resulting variance σ^2 is less than all the variances σ_i^2 of the individual measurements. Thus the uncertainty of the position estimate has been decreased by combining the two measurements. The solid probability density curve represents the result of the Kalman filter in figure 5.26, depicting this result. Even poor measurements, such as are provided by the sonar, will only increase the precision of an estimate. This is a result that we expect based on information theory.

Equation (5.35) can be rewritten as

$$\hat{q} = q_1 + \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2} (q_2 - q_1) \quad (5.37)$$

**Figure 5.26**

Fusing probability density of two estimates [106].

or, in the final form that is used in Kalman filter implementation,

$$\hat{x}_{k+1} = \hat{x}_k + K_{k+1}(z_{k+1} - \hat{x}_k) \quad (5.38)$$

where

$$K_{k+1} = \frac{\sigma_k^2}{\sigma_k^2 + \sigma_z^2} ; \quad \sigma_k^2 = \sigma_1^2 ; \quad \sigma_z^2 = \sigma_2^2 \quad (5.39)$$

Equation (5.38) tells us, that the best estimate $\hat{x}_{k+1}$ of the state x_{k+1} at time $k+1$ is equal to the best prediction of the value $\hat{x}_k$ before the new measurement z_{k+1} is taken, plus a correction term of an optimal weighting value times the difference between z_{k+1} and the best prediction $\hat{x}_k$ at time $k+1$. The updated variance of the state $\hat{x}_{k+1}$ is given using equation (5.36)

$$\sigma_{k+1}^2 = \sigma_k^2 - K_{k+1}\sigma_k^2 \quad (5.40)$$

The new, fused estimate of robot position provided by the Kalman filter is again subject to a Gaussian probability density curve. Its mean and variance are simply functions of the inputs' means and variances. Thus the Kalman filter provides both a compact, simplified representation of uncertainty and an extremely efficient technique for combining heterogeneous estimates to yield a new estimate for our robot's position.

Dynamic estimation. Next, consider a robot that moves between successive sensor measurements. Suppose that the motion of the robot between times k and $k+1$ is described by the velocity u and the noise w which represents the uncertainty of the actual velocity:

$$\frac{dx}{dt} = u + w \quad (5.41)$$

If we now start at time k , knowing the variance σ_k^2 of the robot position at this time and knowing the variance σ_w^2 of the motion, we obtain for the time k' just when the measurement is taken,

$$\hat{x}_{k'} = \hat{x}_k + u(t_{k+1} - t_k) \quad (5.42)$$

$$\sigma_{k'}^2 = \sigma_k^2 + \sigma_w^2[t_{k+1} - t_k] \quad (5.43)$$

where

$$t_{k'} = t_{k+1};$$

t_{k+1} and t_k are the time in seconds at $k+1$ and k respectively.

Thus $\hat{x}_{k'}$ is the optimal prediction of the robot's position just as the measurement is taken at time $k+1$. It describes the growth of position error until a new measurement is taken (figure 5.27).

We can now rewrite equations (5.38) and (5.39) using equations (5.42) and (5.43).

$$\begin{aligned} \hat{x}_{k+1} &= \hat{x}_{k'} + K_{k+1}(z_{k+1} - \hat{x}_{k'}) \\ &= [\hat{x}_k + u(t_{k+1} - t_k)] + K_{k+1}[z_{k+1} - \hat{x}_k - u(t_{k+1} - t_k)] \end{aligned} \quad (5.44)$$

$$K_{k+1} = \frac{\sigma_{k'}^2}{\sigma_{k'}^2 + \sigma_z^2} = \frac{\sigma_k^2 + \sigma_w^2[t_{k+1} - t_k]}{\sigma_k^2 + \sigma_w^2[t_{k+1} - t_k] + \sigma_z^2} \quad (5.45)$$

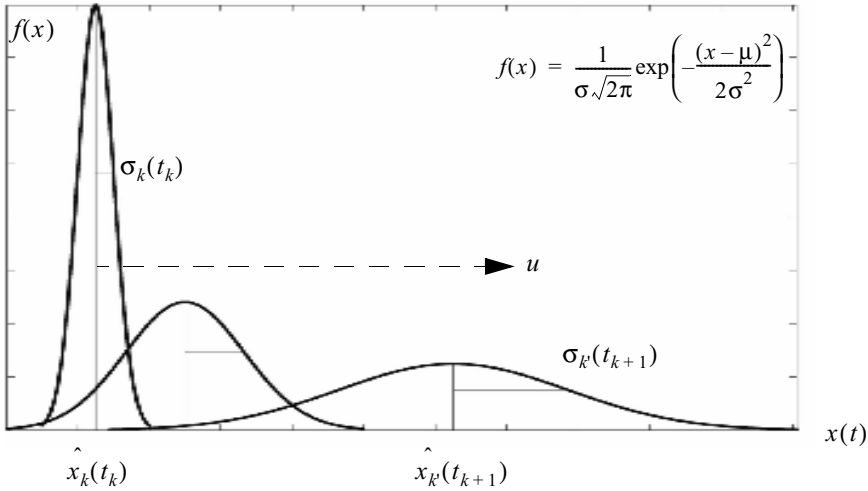


Figure 5.27
Propagation of probability density of a moving robot [106].

The optimal estimate at time $k + 1$ is given by the last estimate at k and the estimate of the robot motion including the estimated movement errors.

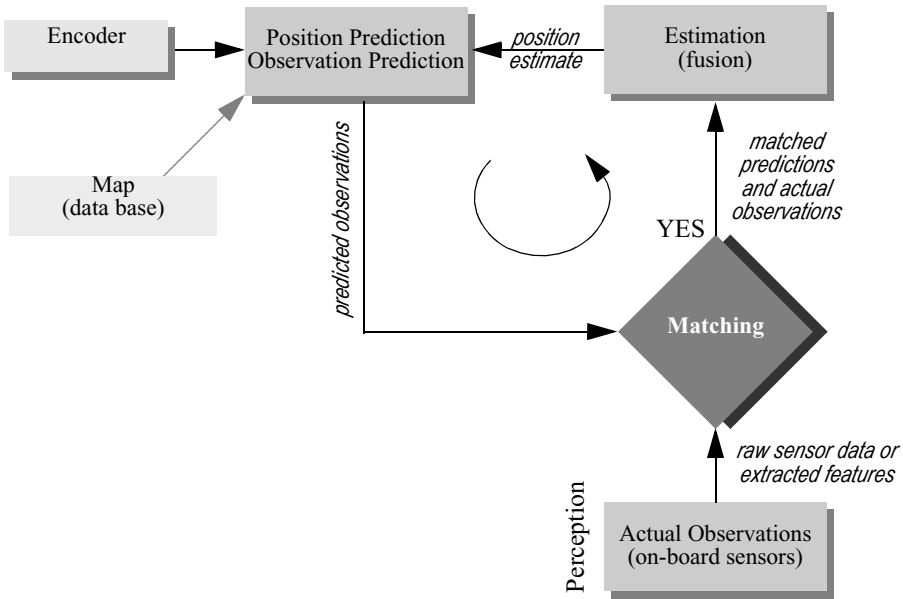
By extending the above equations to the vector case and allowing time-varying parameters in the system and a description of noise, we can derive the Kalman filter localization algorithm.

5.6.3.2 Application to mobile robots: Kalman filter localization

The Kalman filter is an optimal and efficient sensor fusion technique. Application of the Kalman filter to localization requires posing the robot localization problem as a sensor fusion problem. Recall that the basic probabilistic update of robot belief state can be segmented into two phases, *perception update* and *action update*, as specified by equations (5.21) and (5.22).

The key difference between the Kalman filter approach and our earlier Markov localization approach lies in the perception update process. In Markov localization, the entire perception, that is, the robot's set of instantaneous sensor measurements, is used to update each possible robot position in the belief state individually using the Bayes formula. In some cases, the perception is abstract, having been produced by a feature extraction mechanism, as in Dervish. In other cases, as with Rhino, the perception consists of raw sensor readings.

By contrast, perception update using a Kalman filter is a multistep process. The robot's total sensory input is treated not as a monolithic whole but as a set of extracted features that

**Figure 5.28**

Schematic for Kalman filter mobile robot localization (see [23]).

each relate to objects in the environment. Given a set of possible features, the Kalman filter is used to fuse the distance estimate from each feature to a matching object in the map. Instead of carrying out this matching process for many possible robot locations individually as in the Markov approach, the Kalman filter accomplishes the same probabilistic update by treating the whole, unimodal, and Gaussian belief state at once. Figure 5.28 depicts the particular schematic for Kalman filter localization.

The first step is action update or *position prediction*, the straightforward application of a Gaussian error motion model to the robot's measured encoder travel. The robot then collects actual sensor data and extracts appropriate features (e.g., lines, doors, or even the value of a specific sensor) in the *observation* step. At the same time, based on its predicted position in the map, the robot generates a *measurement prediction* which identifies the features that the robot expects to find and the positions of those features. In *matching* the robot identifies the best pairings between the features actually extracted during observation and the expected features due to measurement prediction. Finally, the Kalman filter can fuse the information provided by all of these matches to update the robot belief state in *estimation*.

In the following sections these five steps are described in greater detail. The presentation is based on the work of Leonard and Durrant-Whyte [23, pp. 61–65].

1. Robot position prediction. The robot's position at timestep $k + 1$ is predicted based on its old location (at timestep k) and its movement due to the control input $u(k)$:

$$\hat{p}(k + 1|k) = f(\hat{p}(k|k), u(k)) \quad (5.46)$$

For the differential-drive robot, $\hat{p}(k + 1|k) = p'$ is derived in equations (5.6) and (5.7) respectively.

Knowing the plant and error model, we can also compute the variance $\Sigma_p(k + 1|k)$ associated with this prediction [see equation. (5.9), section 5.2.4]:

$$\Sigma_p(k + 1|k) = \nabla_p f \cdot \Sigma_p(k|k) \cdot \nabla_p f^T + \nabla_u f \cdot \Sigma_u(k) \cdot \nabla_u f^T \quad (5.47)$$

This allows us to predict the robot's position and its uncertainty after a movement specified by the control input $u(k)$. Note that the belief state is assumed to be Gaussian, and so we can characterize the belief state with just the two parameters $\hat{p}(k + 1|k)$ and $\Sigma_p(k + 1|k)$.

2. Observation. The second step is to obtain sensor measurements $Z(k + 1)$ from the robot at time $k + 1$. In this presentation, we assume that the observation is the result of a feature extraction process executed on the raw sensor data. Therefore, the observation consists of a set n_0 of single observations $z_j(k + 1)$ extracted from various sensors. Formally, each single observation can represent an extracted feature such as a line or door, or even a single, raw sensor value.

The parameters of the features are usually specified in the sensor frame and therefore in a local reference frame of the robot. However, for matching we need to represent the observations and measurement predictions in the same frame $\{S\}$. In our presentation we will transform the measurement predictions from the global coordinate frame to the sensor frame $\{S\}$. This transformation is specified in the function h_i discussed in the next paragraph.

3. Measurement prediction. We use the predicted robot position $\hat{p}(k + 1|k)$ and the map $M(k)$ to generate multiple predicted feature observations z_i . Each predicted feature has its position transformed into the sensor frame:

$$\hat{z}_i(k + 1) = h_i(z_i, \hat{p}(k + 1|k)) \quad (5.48)$$

We can define the measurement prediction as the set containing all n_t predicted feature observations:

$$\hat{Z}(k+1) = \{\hat{z}_i(k+1) | (1 \leq i \leq n_t)\} \quad (5.49)$$

The predicted state estimate $\hat{p}(k+1|k)$ is used to compute the measurement Jacobian ∇h_i for each prediction. As you will see in the example below, the function h_i is mainly a coordinate transformation between the world frame and the sensor frame.

4. Matching. At this point we have a set of actual, single observations, which are features in sensor space, and we also have a set of predicted features, also positioned in sensor space. The matching step has the purpose of identifying all of the single observations that match specific predicted features well enough to be used during the estimation process. In other words, we will, for a subset of the observations and a subset of the predicted features, find pairings that intuitively say “this observation is the robot’s measurement of this predicted feature based on the map.”

Formally, the goal of the matching procedure is to produce an assignment from observations $z_j(k+1)$ to the targets z_i (stored in the map). For each measurement prediction for which a corresponding observation is found we calculate the innovation $v_{ij}(k+1)$. *Innovation* is a measure of the difference between the predicted and observed measurements:

$$\begin{aligned} v_{ij}(k+1) &= [z_j(k+1) - \hat{z}_i(k+1)] \\ &= [z_j(k+1) - h_i(z_i, \hat{p}(k+1|k))] \end{aligned} \quad (5.50)$$

The innovation covariance $\Sigma_{IN,ij}(k+1)$ can be found by applying the error propagation law [section 4.2.2, equation (4.60)]:

$$\Sigma_{IN,ij}(k+1) = \nabla h_i \cdot \Sigma_p(k+1|k) \cdot \nabla h_i^T + \Sigma_{R,i}(k+1) \quad (5.51)$$

where $\Sigma_{R,i}(k+1)$ represents the covariance (noise) of the measurement $z_i(k+1)$.

To determine the validity of the correspondence between measurement prediction and observation, a *validation gate* has to be specified. A possible definition of the validation gate is the Mahalanobis distance:

$$v_{ij}^T(k+1) \cdot \Sigma_{IN,ij}^{-1}(k+1) \cdot v_{ij}(k+1) \leq g^2 \quad (5.52)$$

However, dependent on the application, the sensors, and the environment models, more sophisticated validation gates might be employed.

The validation equation is used to test observation $z_j(k+1)$ for membership in the validation gate for each predicted measurement. When a single observation falls in the validation gate, we get a successful match. If one observation falls in multiple validation gates,

the best matching candidate is selected or multiple hypotheses are tracked. Observations that do not fall in the validation gate are simply ignored for localization. Such observations could have resulted from objects not in the map, such as new objects (e.g., someone places a large box in the hallway) or transient objects (e.g., humans standing next to the robot may form a line feature). One approach is to take advantage of such unmatched observations to populate the robot's map.

5. Estimation: applying the Kalman filter. Next we compute the best estimate $\hat{p}(k+1|k+1)$ of the robot's position based on the position prediction and all the observations at time $k+1$. To do this position update, we first stack the validated observations $z_j(k+1)$ into a single vector to form $z(k+1)$ and designate the composite innovation $v(k+1)$. Then we stack the measurement Jacobians ∇h_i for each validated measurement together to form the composite Jacobian ∇h and the measurement error (noise) vector $\Sigma_R(k+1) = \text{diag}[\Sigma_{R,i}(k+1)]$. We can then compute the composite innovation covariance $\Sigma_{IN}(k+1)$ according to equation (5.51) and by utilizing the well-known result [3] that the Kalman gain can be written as

$$K(k+1) = \Sigma_p(k+1|k) \cdot \nabla h^T \cdot \Sigma_{IN}^{-1}(k+1) \quad (5.53)$$

we can update the robot's position estimate

$$\hat{p}(k+1|k+1) = \hat{p}(k+1|k) + K(k+1) \cdot v(k+1) \quad (5.54)$$

with the associated variance

$$\Sigma_p(k+1|k+1) = \Sigma_p(k+1|k) - K(k+1) \cdot \Sigma_{IN}(k+1) \cdot K^T(k+1) \quad (5.55)$$

For the 1D case and with $h_i(z_t, \hat{p}(k+1|k)) = z_t$ we can show that this formula corresponds to the 1D case derived earlier

Equation (5.53) is simplified to

$$K(k+1) = \frac{\sigma_p^2(k+1|k)}{\sigma_{IN}^2(k+1)} = \frac{\sigma_p^2(k+1|k)}{\sigma_p^2(k+1|k) + \sigma_R^2(k+1)} \quad (5.56)$$

corresponding to equation (5.45), and equation (5.54) simplifies to

$$\begin{aligned}
\hat{p}(k+1|k+1) &= \hat{p}(k+1|k) + K(k+1) \cdot v(k+1) \\
&= \hat{p}(k+1|k) + K(k+1) \cdot [z_j(k+1) - h_i(z_p, \hat{p}(k+1|k))] \\
&= \hat{p}(k+1|k) + K(k+1) \cdot [z_j(k+1) - z_t]
\end{aligned} \tag{5.57}$$

corresponding to equation (5.44).

5.6.3.3 Case study: Kalman filter localization with line feature extraction

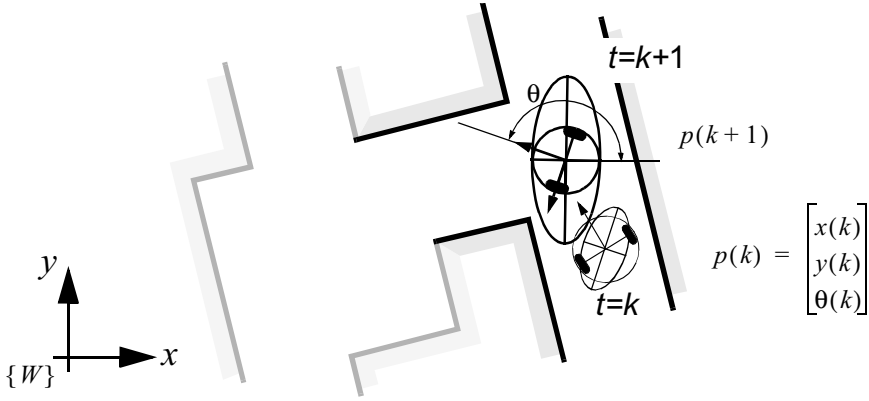
The Pygmalion robot at EPFL is a differential-drive robot that uses a laser rangefinder as its primary sensor [37, 38]. In contrast to both Dervish and Rhino, the environmental representation of Pygmalion is continuous and abstract: the map consists of a set of infinite lines describing the environment. Pygmalion's belief state is, of course, represented as a Gaussian distribution since this robot uses the Kalman filter localization algorithm. The value of its mean position μ is represented to a high level of precision, enabling Pygmalion to localize with very high precision when desired. Below, we present details for Pygmalion's implementation of the five Kalman filter localization steps. For simplicity we assume that the sensor frame $\{S\}$ is equal to the robot frame $\{R\}$. If not specified all the vectors are represented in the world coordinate system $\{W\}$.

1. Robot position prediction. At the time increment k the robot is at position $p(k) = [x(k) \ y(k) \ \theta(k)]^T$ and its best position estimate is $\hat{p}(k|k)$. The control input $u(k)$ drives the robot to the position $p(k+1)$ (figure 5.29).

The robot position prediction $\hat{p}(k+1)$ at the time increment $k+1$ can be computed from the previous estimate $\hat{p}(k|k)$ and the odometric integration of the movement. For the differential drive that Pygmalion has we can use the model (odometry) developed in section 5.2.4:

$$\hat{p}(k+1|k) = \hat{p}(k|k) + u(k) = \hat{p}(k|k) + \begin{bmatrix} \frac{\Delta s_r + \Delta s_l}{2} \cos\left(\theta + \frac{\Delta s_r - \Delta s_l}{2b}\right) \\ \frac{\Delta s_r + \Delta s_l}{2} \sin\left(\theta + \frac{\Delta s_r - \Delta s_l}{2b}\right) \\ \frac{\Delta s_r - \Delta s_l}{b} \end{bmatrix} \tag{5.58}$$

with the updated covariance matrix

**Figure 5.29**

Prediction of the robot's position (thick) based on its former position (thin) and the executed movement. The ellipses drawn around the robot positions represent the uncertainties in the x, y direction (e.g.; 3σ). The uncertainty of the orientation θ is not represented in the picture.

$$\Sigma_p(k+1|k) = \nabla_p f \cdot \Sigma_p(k|k) \cdot \nabla_p f^T + \nabla_u f \cdot \Sigma_u(k) \cdot \nabla_u f^T \quad (5.59)$$

where

$$\Sigma_u = \text{cov}(\Delta s_r, \Delta s_l) = \begin{bmatrix} k_r |\Delta s_r| & 0 \\ 0 & k_l |\Delta s_l| \end{bmatrix} \quad (5.60)$$

2. Observation. For line-based localization, each single observation (i.e., a line feature) is extracted from the raw laser rangefinder data and consists of the two line parameters $\beta_{0,j}$, $\beta_{1,j}$ or α_j , r_j (figure 4.36) respectively. For a rotating laser rangefinder, a representation in the polar coordinate frame is more appropriate and so we use this coordinate frame here:

$$z_j(k+1) = {}^R \begin{bmatrix} \alpha_j \\ r_j \end{bmatrix} \quad (5.61)$$

After acquiring the raw data at time $k+1$, lines and their uncertainties are extracted (figure 5.30a, b). This leads to n_0 observed lines with $2n_0$ line parameters (figure 5.30c) and a covariance matrix for each line that can be calculated from the uncertainties of all the

measurement points contributing to each line as developed for line extraction in section 4.3.1.1:

$$\Sigma_{R,j} = \begin{bmatrix} \sigma_{\alpha\alpha} & \sigma_{\alpha r} \\ \sigma_{r\alpha} & \sigma_{rr} \end{bmatrix}_j \quad (5.62)$$

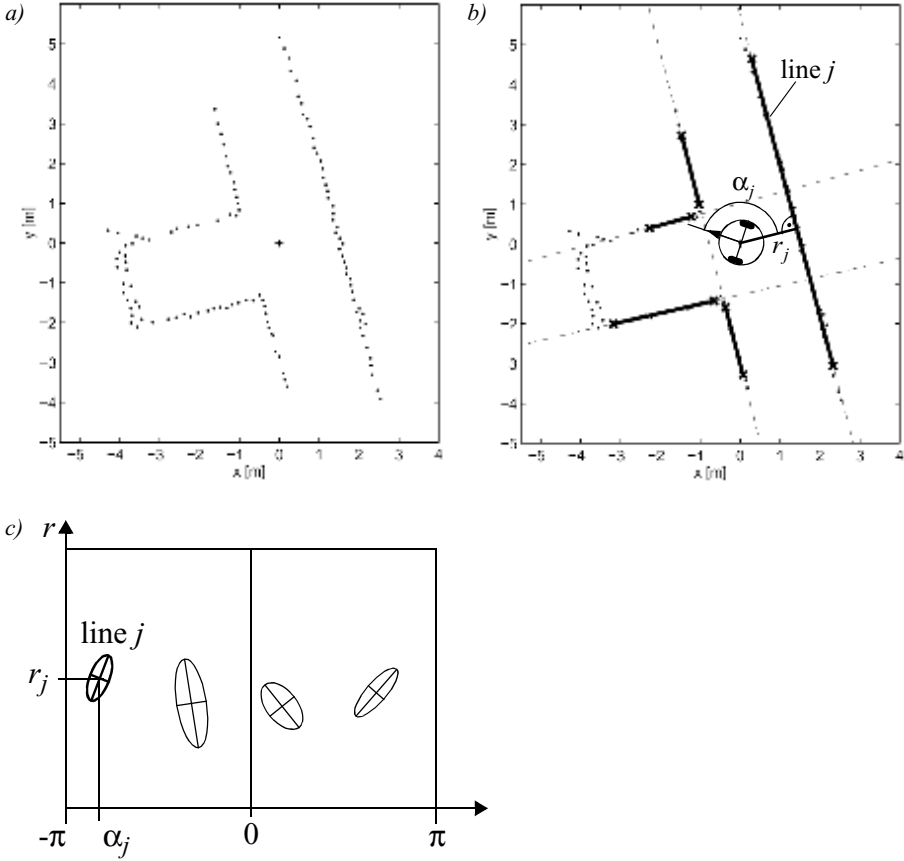


Figure 5.30

Observation: From the raw data (a) acquired by the laser scanner at time $k + 1$, lines are extracted (b). The line parameters α_j and r_j and its uncertainties can be represented in the model space (c).

3. Measurement prediction. Based on the stored map and the predicted robot position $\hat{p}(k|k)$, the measurement predictions of expected features $z_{t,i}$ are generated (figure 5.31). To reduce the required calculation power, there is often an additional step that first selects the possible features, in this case lines, from the whole set of features in the map. These lines are stored in the map and specified in the world coordinate system $\{W\}$. Therefore they need to be transformed to the robot frame $\{R\}$:

$${}^W z_{t,i} = \begin{bmatrix} \alpha_{t,i} \\ r_{t,i} \end{bmatrix} \rightarrow {}^R z_{t,i} = \begin{bmatrix} \alpha_{t,i} \\ r_{t,i} \end{bmatrix} \quad (5.63)$$

According to figure (5.31), the transformation is given by

$$\begin{aligned} \hat{z}_i(k+1) &= \begin{bmatrix} \alpha_{t,i} \\ r_{t,i} \end{bmatrix} = h_i(z_{t,i}, \hat{p}(k+1|k)) \\ &= \begin{bmatrix} {}^W \alpha_{t,i} - {}^W \hat{\theta}(k+1|k) \\ {}^W r_{t,i} - ({}^W \hat{x}(k+1|k) \cos({}^W \alpha_{t,i}) + {}^W \hat{y}(k+1|k) \sin({}^W \alpha_{t,i})) \end{bmatrix} \end{aligned} \quad (5.64)$$

and its Jacobian ∇h_i by

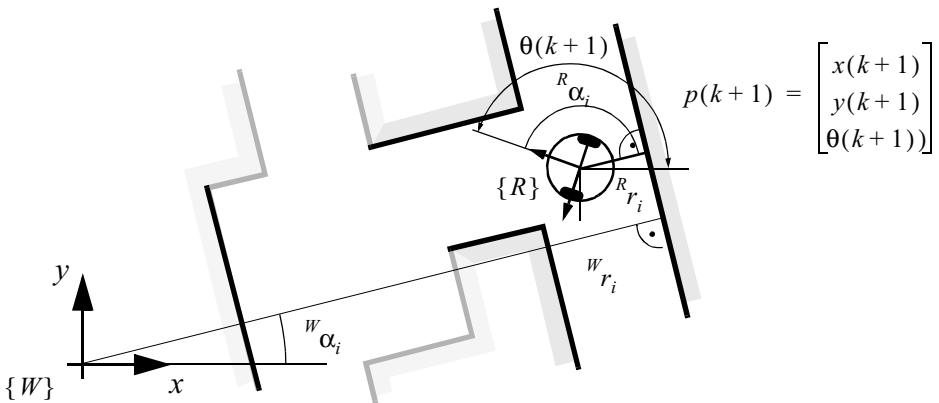


Figure 5.31

Representation of the target position in the world coordinate frame $\{W\}$ and robot coordinate frame $\{R\}$.

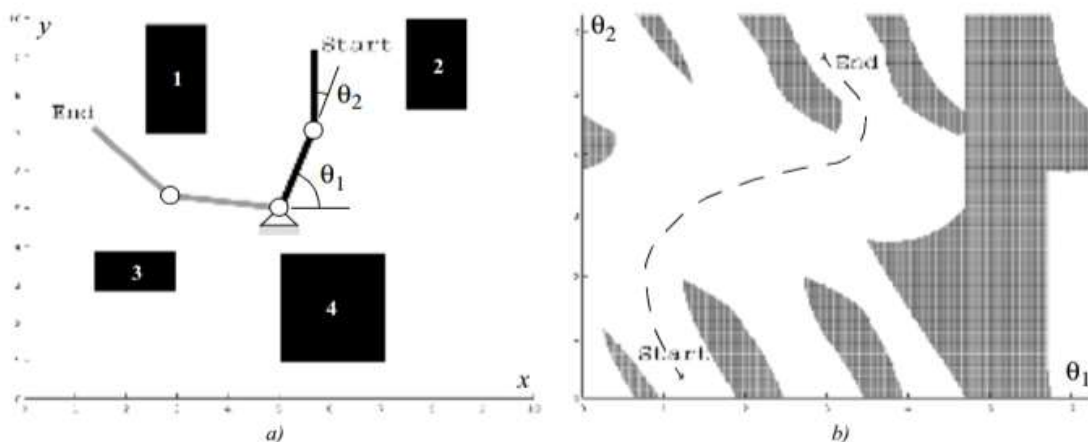
UNIT – 5

ROBOT PLANNING AND NAVIGATION

Planning and navigation – planning and reacting – path planning – obstacle avoidance – navigation architectures.

Planning and Navigation

- Planning and Navigation has focused on the elements of a mobile robot that are
 - Critical to robust mobility
 - The kinematics of locomotion
 - Sensors for determining the robot's environmental context
 - Techniques for localizing with respect to its map
- We now turn our attention to the robot's cognitive level. Cognition generally represents the purposeful decision-making and execution that a system utilizes to achieve its highest-order goals.
- In the case of a mobile robot, the specific aspect of cognition directly linked to robust mobility is navigation competence. Given partial knowledge about its environment and a goal position or series of positions, navigation encompasses the ability of the robot to act based on its knowledge and sensor values so as to reach its goal positions as efficiently and as reliably as possible.
- Given a map and a goal location, path planning involves identifying a trajectory that will cause the robot to reach the goal location when executed. Path planning is a strategic problem-solving competence, as the robot must decide what to do over the long term to achieve its goals.
- The second competence is equally important but occupies the opposite, tactical extreme. Given real-time sensor readings, obstacle avoidance means modulating the trajectory of the robot in order to avoid collisions. A great variety of approaches have demonstrated competent obstacle avoidance, and we survey a number of these approaches as well.



Planning and Reacting

- Planning and reacting are essential components of AI and robotics. Planning involves generating a sequence of actions to achieve a goal, while reacting involves responding to unexpected situations in real-time.
- AI and robotics systems often combine both to navigate complex environments effectively.

- AI algorithms like pathfinding and reinforcement learning help plan actions, while sensors and feedback mechanisms allow robots to react to changes in their surroundings.
- The balance between planning and reacting depends on the specific task and the level of autonomy the system possesses

Process

- **Perception:** The robot collects data from its sensors to perceive its surroundings, including its own position, the location of objects, and the state of the environment.
- **Goal Specification:** The robot receives high-level instructions or objectives that it needs to achieve.
- **State Estimation:** The robot combines its sensory input with its internal model of the world to accurately estimate its current state and the state of its environment.
- **Path Planning:** The robot determines a feasible path or trajectory from its current state to the desired goal, considering factors like obstacles, terrain, and the robot's physical capabilities.
- **Motion Planning:** The robot generates specific motions or actions that it should execute to follow the planned path, ensuring that its movements are safe and efficient.

Advantages

- **Efficiency:** Planning allows AI systems to find optimal or near-optimal sequences of actions to achieve goals, minimizing time and resource consumption. Reacting in real-time enables robots to adapt swiftly to unexpected changes without waiting for a predefined plan to be executed.
- **Adaptability:** Planning enables AI systems to handle complex and dynamic environments by considering various possibilities and selecting the best course of action. Reacting allows robots to respond to unforeseen events and adjust their actions based on current conditions.
- **Learning:** Planning and reacting can facilitate machine learning. Planning can be informed by learned models of the environment, and reacting can provide data for updating these models through reinforcement learning or other techniques.

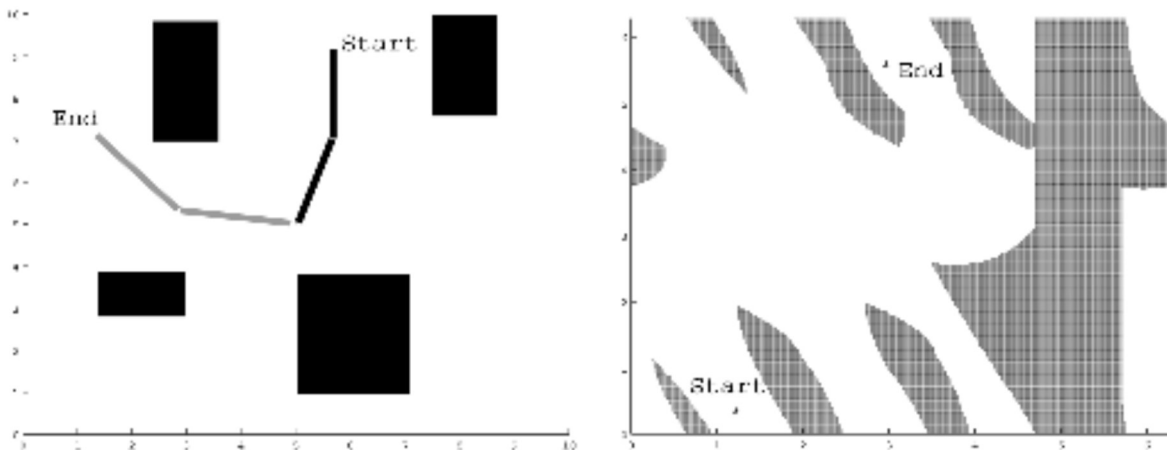
Disadvantages

- **Computational Complexity:** Developing and executing plans can be computationally intensive, especially for complex tasks or environments. Balancing real-time reacting with resource-intensive planning might lead to performance bottlenecks.
- **Uncertainty Handling:** Planning might struggle when dealing with high levels of uncertainty, as it relies on accurate information about the environment. Reacting, while flexible, might not always make the best decisions in uncertain or unfamiliar situations.
- **Over-Dependency on Predefined Plans:** Overreliance on preplanned sequences of actions could make the system less adaptable to unexpected changes.

Path Planning

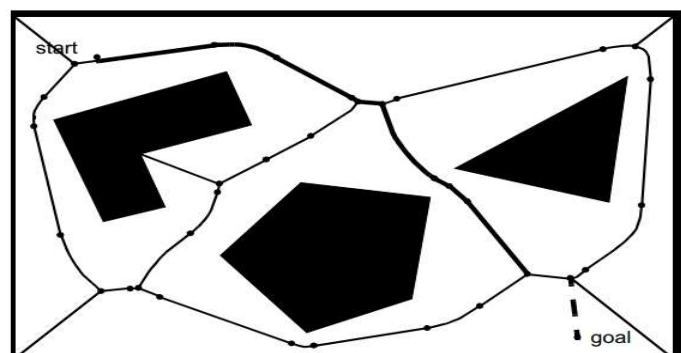
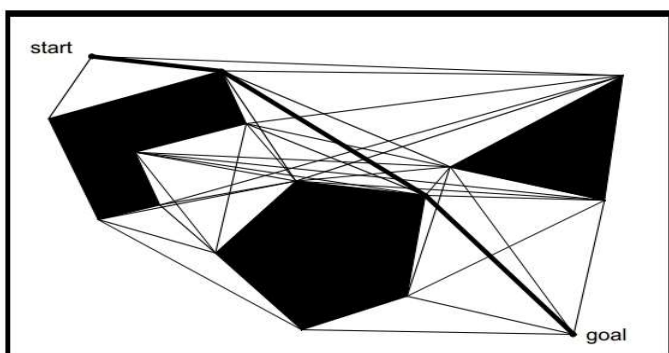
- Path planning is a computational problem to find a sequence of valid configurations that moves the object from the source to destination.

- The robot's environment representation can range from a continuous geometric description to a decomposition-based geometric map or even a topological map.
- The first step of any path-planning system is to transform this possibly continuous environmental model into a discrete map suitable for the chosen path-planning algorithm. Path planners differ as to how they effect this discrete decomposition.
- We can identify three general strategies for decomposition:
 - Road map: identify a set of routes within the free space.
 - Cell decomposition: discriminate between free and occupied cells.
 - Potential field: impose a mathematical function over the space.



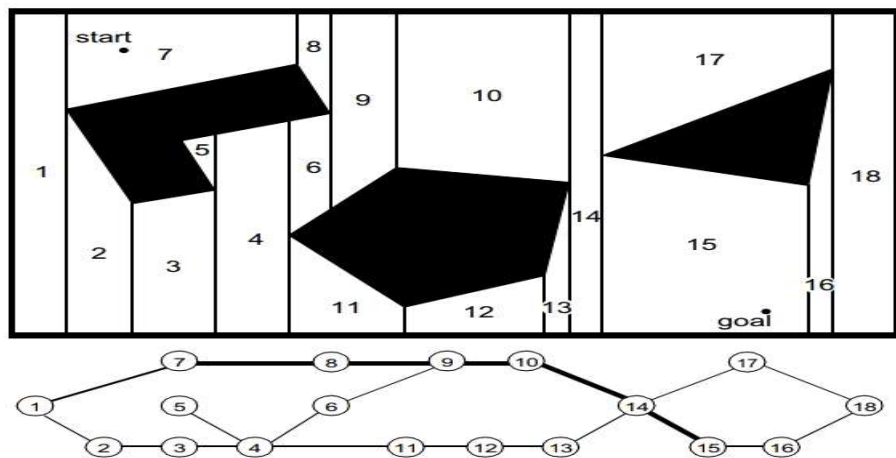
Road map path planning

- Road map approaches capture the connectivity of the robot's free space in a network of 1D curves or lines, called road maps.
- Once a road map is constructed, it is used as a network of road (path) segments for robot motion planning.
- Path planning is thus reduced to connecting the initial and goal positions of the robot to the road network, then searching for a series of roads from the initial robot position to its goal position.
- The road map is a decomposition of the robot's configuration space based specifically on obstacle geometry. The challenge is to construct a set of roads that together enable the robot to go anywhere in its free space, while minimizing the number of total roads.
- Generally, completeness is preserved in such decompositions as long as the true degrees of freedom of the robot have been captured with appropriate fidelity. We describe two road map approaches below that achieve this result with dramatically different types of roads.
- In the case of the visibility graph, roads come as close as possible to obstacles and resulting paths are minimum-length solutions. In the case of the Voronoi diagram, roads stay as far away as possible from obstacles.



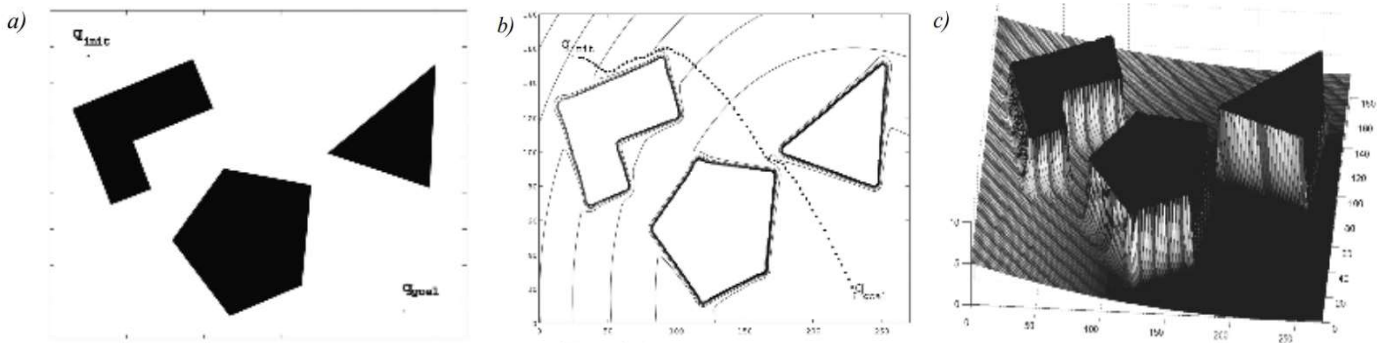
Cell decomposition path planning

- The idea behind cell decomposition is to discriminate between geometric areas, or cells, that are free and areas that are occupied by objects.
 - Divide F into simple, connected regions called “cells”.
 - Determine which open cells are adjacent and construct a “connectivity graph”.
 - Find the cells in which the initial and goal configurations lie and search for a path in the connectivity graph to join the initial and goal cell.
 - From the sequence of cells found with an appropriate searching algorithm, compute a path within each cell, for example, passing through the midpoints of the cell boundaries or by a sequence of wall-following motions and movements along straight lines.
- An important aspect of cell decomposition methods is the placement of the boundaries between cells. If the boundaries are placed as a function of the structure of the environment, such that the decomposition is lossless, then the method is termed exact cell decomposition.



Potential field path planning

- Potential field path planning creates a field, or gradient, across the robot's map that directs the robot to the goal position from multiple prior positions. This approach was originally invented for robot manipulator path planning and is used often and under many variants in the mobile robotics community.
- The potential field method treats the robot as a point under the influence of an artificial potential field. The robot moves by following the field, just as a ball would roll downhill. The goal (a minimum in this space) acts as an attractive force on the robot and the obstacles act as peaks, or repulsive forces.
- The superposition of all forces is applied to the robot, which, in most cases, is assumed to be a point in the configuration space. Such an artificial potential field smoothly guides the robot toward the goal while simultaneously avoiding known obstacles.

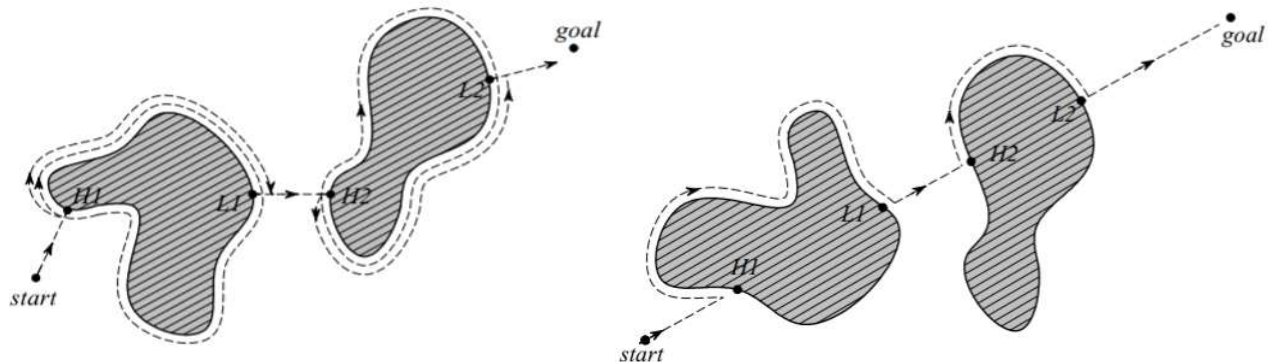


Obstacle Avoidance

- Local obstacle avoidance focuses on changing the robot's trajectory as informed by its sensors during robot motion.
- The resulting robot motion is both a function of the robot's current or recent sensor readings and its goal position and relative location to the goal position.
- The obstacle avoidance algorithms presented below depend to varying degrees on the existence of a global map and on the robot's precise knowledge of its location relative to the map.

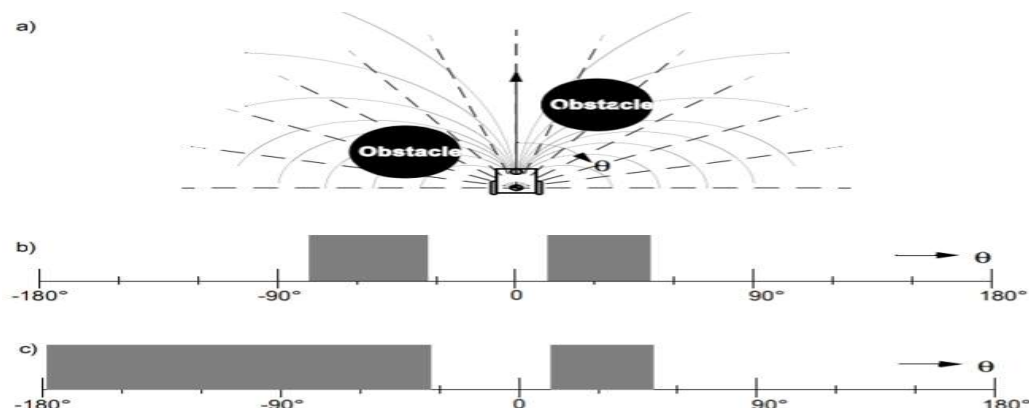
Bug algorithm

- The Bug algorithm is perhaps the simplest obstacle avoidance algorithm one could imagine. The basic idea is to follow the contour of each obstacle in the robot's way and thus circumnavigate it.
- With Bug1, the robot fully circles the object first, then departs from the point with the shortest distance toward the goal. This approach is, of course, very inefficient but guarantees that the robot will reach any reachable goal.
- With Bug2 the robot begins to follow the object's contour, but departs immediately when it is able to move directly toward the goal. In general, this improved Bug algorithm will have significantly shorter total robot travel, as shown in figure 6.8. However, one can still construct situations in which Bug2 is arbitrarily inefficient (i.e., nonoptimal).



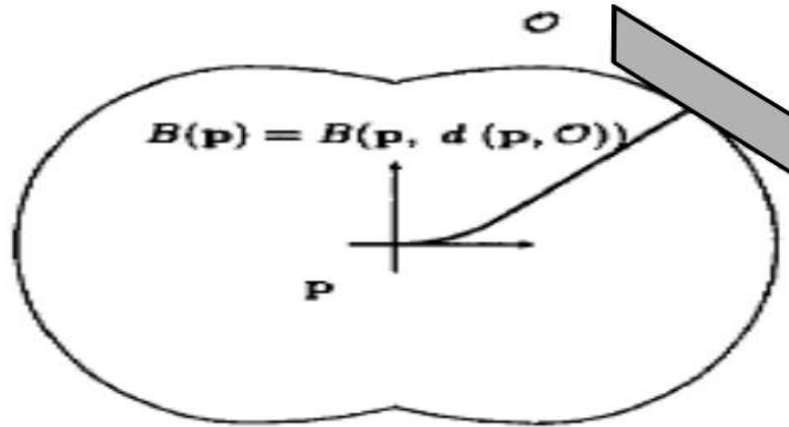
Vector field histogram

- One of the central criticisms of Bug-type algorithms is that the robot's behavior at each instant is generally a function of only its most recent sensor readings.
- This can lead to undesirable and yet preventable problems in cases where the robot's instantaneous sensor readings do not provide enough information for robust obstacle avoidance.
- The VFH techniques overcome this limitation by creating a local map of the environment around the robot.



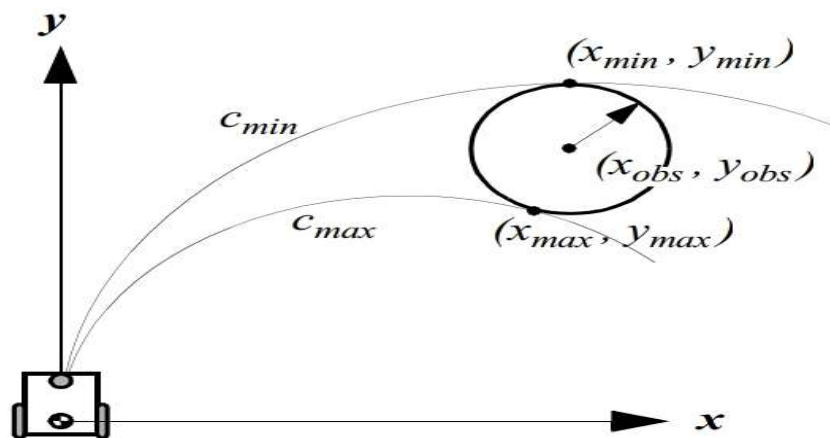
The bubble band technique

- A bubble is defined as the maximum local subset of the free space around a given configuration of the robot that which can be traveled in any direction without collision.
- The bubble is generated using a simplified model of the robot in conjunction with range information available in the robot's map.



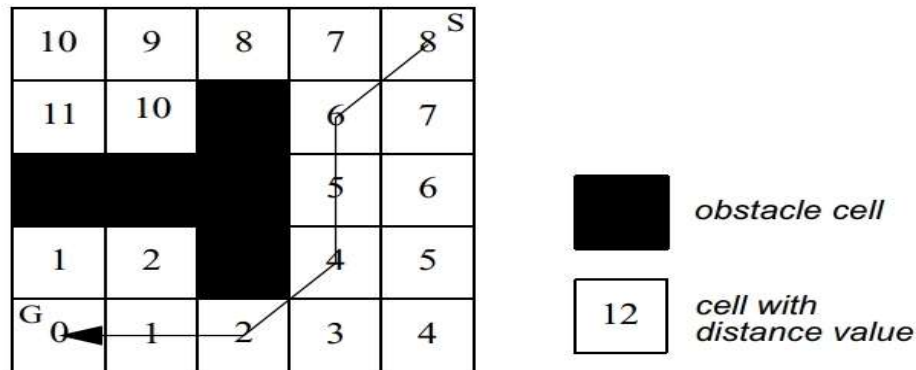
Curvature velocity techniques

- The curvature velocity approach (CVM) from Simmons enables the actual kinematic constraints and even some dynamic constraints of the robot to be taken into account during obstacle avoidance, which is an advantage over more primitive techniques.
- CVM begins by adding physical constraints from the robot and the environment to a velocity space.
- The velocity space consists of rotational velocity and w translational velocity, thus assuming that the robot only travels along arcs of circles with curvature $c = w/v$.



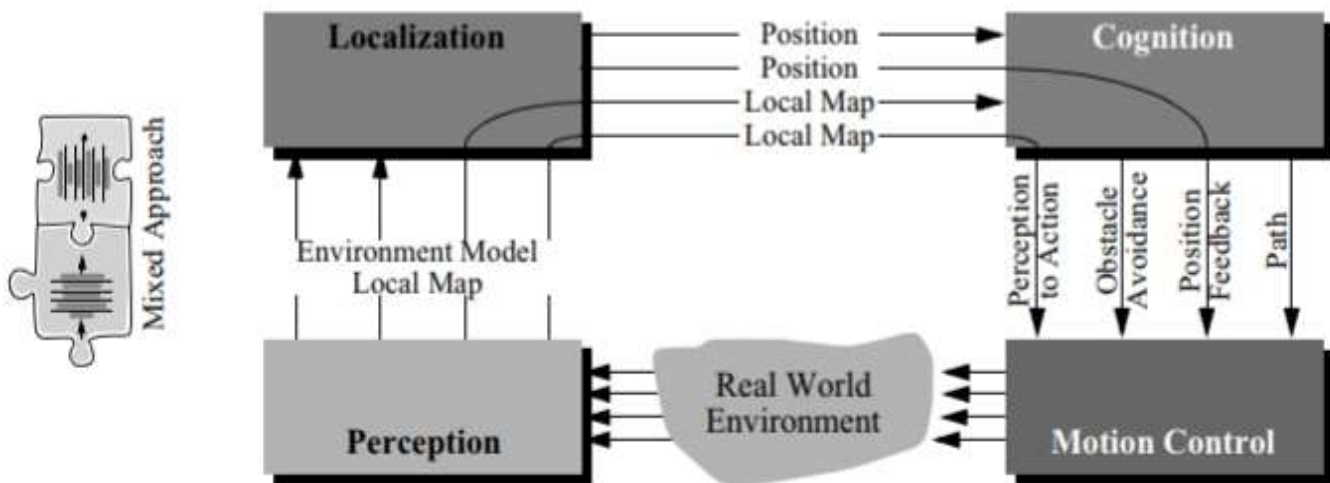
Dynamic window approaches

- In the local dynamic window approach the kinematics of the robot is taken into account by searching a well-chosen velocity space.
- The velocity space is all possible sets of tuples (v, w) where v is the velocity and w is the angular velocity.
- The approach assumes that robots move only in circular arcs representing each such tuple, at least during one timestamp.



Navigation Architectures

- The study of navigation architectures is the study of principled designs for the software modules that constitute a mobile robot navigation system.



Modularity for code reuse and sharing

- Basic software engineering principles embrace software modularity, and the same general motivations apply equally to mobile robot applications. But modularity is of even greater importance in mobile robotics because in the course of a single project the mobile robot hardware or its physical environmental characteristics can change dramatically, a challenge most traditional computers do not face.
- For example, one may introduce a Sick laser rangefinder to a robot that previously used only ultrasonic rangefinders. Or one may test an existing navigator robot in a new environment where there are obstacles that its sensors cannot detect, thereby demanding a new path-planning representation.

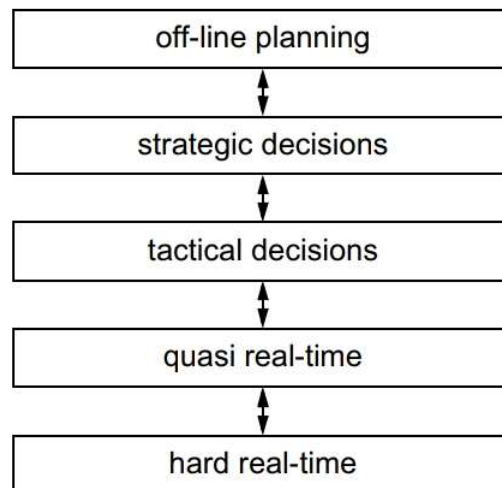
Control localization

- Localization of robot control is an even more critical issue in mobile robot navigation. The basic reason is that a robot architecture includes multiple types of control functionality (e.g., obstacle avoidance, path planning, path execution, etc.).
- By localizing each functionality to a specific unit in the architecture, we enable individual testing as well as a principled strategy for control composition.
- For example, consider collision avoidance. For stability in the face of changing robot software, as well as for focused verification that the obstacle avoidance system is correctly implemented, it is valuable to

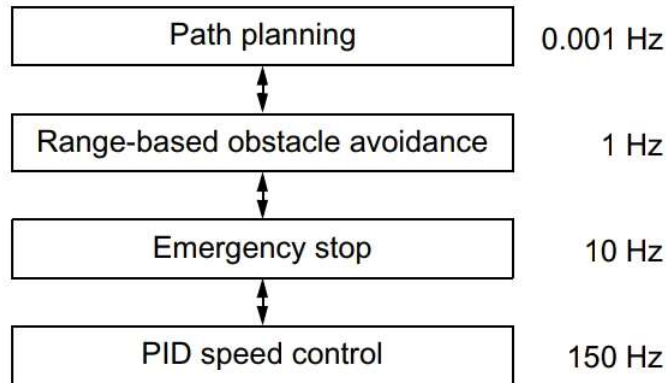
localize all software related to the robot's obstacle avoidance process. At the other extreme, high-level planning and task-based decision-making are required for robots to perform useful roles in their environment.

Techniques for decomposition

- Decompositions identify axes along which we can justify discrimination of robot software into distinct modules. Decompositions also serve as a way to classify various mobile robots into a more quantitative taxonomy.
 - **Temporal decomposition**
 - ✓ A temporal decomposition of robot software distinguishes between processes that have varying real-time and non real-time demands.
 - **Sensor response time**
 - ✓ A particular module's sensor response time can be defined as the amount of time between acquisition of a sensor-based event and a corresponding change in the output of the module.

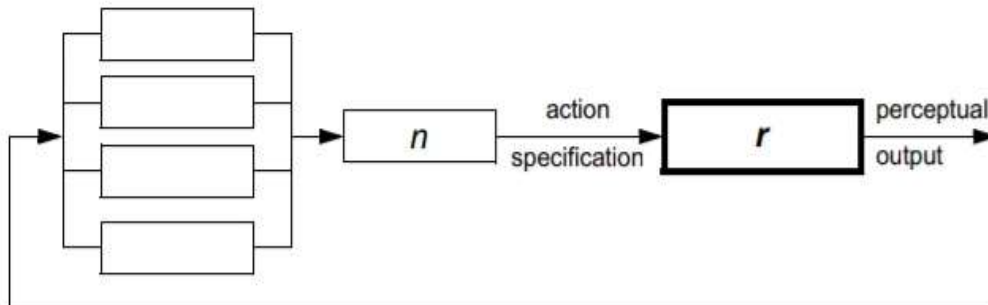


- **Temporal depth**
 - ✓ Temporal depth is a useful concept applying to the temporal window that affects the module's output, both backward and forward in time. Temporal horizon describes the amount of look ahead used by the module during the process of choosing an output.
- **Spatial locality**
 - ✓ Hand in hand with temporal span, the spatial impact of layers increases dramatically as one moves from low-level modules to high-level modules. Real-time modules tend to control wheel speed and orientation, controlling spatially localized behavior.
- **Context specificity**
 - ✓ A module makes decisions as a function not only of its immediate inputs but also as a function of the robot's context as captured by other variables, such as the robot's representation of the environment. Lowest-level modules tend to produce outputs directly as a result of immediate sensor inputs, using little context and therefore being relatively context insensitive.



Control decomposition

- Whereas temporal decomposition discriminates based on the time behavior of software modules, control decomposition identifies the way in which each module's output contributes to the overall robot control outputs. Presentation of control decomposition requires the evaluator to understand the basic principles of discrete systems representation and analysis.



UNIT – 5

ROBOT PLANNING AND NAVIGATION – QUESTION BANK

PART – A

- Define planning and Navigation.
- Define planning and reacting.
- What is the path planning technique?
- What is the road map path planning?
- What is the visibility graph?
- Define Voronoi diagram.
- Implement the technique of obstacles avoidance.
- Define bug algorithm.
- Define navigation architecture.
- What is the control navigation?
- List the technique for decomposition.

PART – B

- Explain the concepts of planning and navigation with an example.
- Describe the planning and reacting technique in robot navigation process.

3. What is the path planning? Explain the technique of path planning.
4. Develop the algorithm for obstacle avoidance technique for build a navigation robot.
5. Examine the navigation architecture with an example.

GNANAMANI COLLEGE OF TECHNOLOGY
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
AD8705-AI and Robotics - Question Bank

UNIT – I ROBOT LOCOMOTION

PART - A

1. Define AI.
2. Define Robotics.
3. Explain the usage of AI in Robotics.
4. List the application of AI.
5. List the application of Robotics?
6. Define Robot locomotion.
7. What are the key issues in robot locomotion?
8. List the types of mobile robot locomotion.
9. Define legged mobile robot. List its types.
10. Define leg configuration and stability in legged mobile robots.
11. Define wheeled mobile robots and list its types.
12. Explain the types of wheels are used in wheeled mobile robots.
13. Define drone.
14. List the applications of aerial mobile robots.
15. Write down the importance of drones.

PART – B

1. Explain in the details about robot locomotion.
2. Define legged mobile robots. Explain in the types of legged mobile robots.
3. Define wheeled mobile robots. Explain in the types of wheeled mobile robots.
4. Define Aerial Mobile robots. Write the application and importance of drones.

UNIT – II MOBILE ROBOT KINEMATICS

PART – A

1. Define kinematics models and constraints.
2. Explain the concepts of robot position.
3. What is the Forward kinematic model?
4. What is the wheel kinematic constraints?
5. Define robot maneuverability.
6. Define DOF.
7. Define degree of mobility.
8. Define Degree of steerability.
9. What is the holonomic robot?
10. Define motion control.
11. Define open loop control.
12. Define feedback control.
13. Explain the concepts of advanced kinematics.

PART – B

1. Explain the details about kinematic models and constraints.
2. Discuss the details about mobile robot maneuverability.
3. Analyze the mobile robot workshop constraints.
4. Examine the Advance kinematics efficiency compare to basic constraints.
5. Explain the details about motion control mechanism.

UNIT – III ROBOT PERCEPTION

PART – A

1. What is sensor?
2. What are the types of sensors used in mobile robot construction?
3. Examine the sensor classification.
4. List the characteristics of sensors.
5. Define wheel motor sensor with example.
6. Define heading sensor with example.
7. Define ground-based beacons with example.
8. Define active ranging with example.
9. Define motion sensor with example.
10. Define vision-based sensor with example.
11. Define computer vision for robots.
12. What is color channel histogram?
13. What are the techniques included in robot arm pic and place of object concepts?
14. What is the technique included in object segmentation process?
15. List the application of computer vision for robots.
16. Define image processing technique.
17. Explain the common image processing techniques in robotics.
18. Explain the advanced image processing techniques in robotics.
19. What is place recognition?
20. Define range data.

PART – B

1. Explain the sensors usage in robotics.
2. Examine the technique of computer vision for robotics.
3. Examine the concepts of image processing in robotics with applications.
4. Explain the concepts of place recognition in robot's technology.
5. Define range data. Examine the range data algorithm with example.

UNIT – IV MOBILE ROBOT LOCALIZATION

PART – A

1. Define localization.
2. Define noise and aliasing.
3. Define sensor noise with example.
4. Define sensor aliasing with example.
5. Define effector noise.
6. What is the localization-based navigation?

7. What is the belief representation?
8. List the types of hypothesis belief.
9. Define map representation.
10. Define probability map-based localization.
11. Define markov localization.
12. Define Kalman filter localization.
13. Define autonomous map building.
14. Define cyclic environments.
15. Define dynamic environments.

PART - B

1. Explain the details about noise and aliasing with an example.
2. Examine the localization-based navigation in robotics.
3. What is belief representation? Explain it with an example.
4. What is map representation? Explain it with an example.
5. Define probability map-based localization with example.
6. Describe the details about autonomous map building robots.

UNIT - V ROBOT PLANNING AND NAVIGATION

PART - A

1. Define planning and Navigation.
2. Define planning and reacting.
3. What is the path planning technique?
4. What is the road map path planning?
5. What is the visibility graph?
6. Define Voronoi diagram.
7. Implement the technique of obstacles avoidance.
8. Define bug algorithm.
9. Define navigation architecture.
10. What is the control navigation?
11. List the technique for decomposition.

PART - B

1. Explain the concepts of planning and navigation with an example.
2. Describe the planning and reacting technique in robot navigation process.
3. What is the path planning? Explain the technique of path planning.
4. Develop the algorithm for obstacle avoidance technique for build a navigation robot.
5. Examine the navigation architecture with an example.

PREPARED BY
FACULTY / AI&DS

APPROVED BY
HOD / AI&DS